Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Contents

Reference

Quick Reference

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 1 - Meet the X-Touch

Why "Project XTC"?

XTC stands for X-Touch Companion. This guide is designed to sit alongside the official Behringer, Bitwig and DrivenByMoss documentation, bringing everything together into one practical handbook. If the name raises a knowing smile, that's perfectly fine - but throughout this book, XTC always means X-Touch Companion.

If you've bought a Behringer X-Touch, you've probably already discovered two things. First, it feels like a serious piece of studio equipment. Second, learning to use it effectively with Bitwig can be surprisingly difficult.

The Behringer manual explains the hardware. Bitwig explains the DAW. DrivenByMoss explains the extension. What has been missing is a guide that explains how all three work together. That is the purpose of Project XTC.

Who This Book Is For

This guide assumes you already know the basics of using a computer and Bitwig Studio. It does not assume any previous knowledge of MCU, MIDI protocols or control surfaces. Every important idea is introduced only when it becomes useful.

What the X-Touch Is

The X-Touch is a hardware control surface. It does not process audio or replace Bitwig. It provides tactile control over track selection, transport, mixing, PLUG-IN, automation and much more.

The Four Principles

1. Bitwig is in charge.
2. The selected track matters.
3. Modes change meanings.
4. Feedback is as important as input.

Before You Continue...

Don't worry about memorising every button yet. In the next chapter we'll take a guided tour of the X-Touch so every control has a name and a purpose before we start using it.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 2 - A Tour of the Hardware

Before learning individual features, it's worth becoming familiar with the physical layout of the X-Touch. You don't need to memorise every control. The goal is simply to help you recognise where things are, so later chapters can refer to them naturally.

The Eight Channel Strips

The heart of the X-Touch consists of eight identical channel strips. Each strip includes a motor fader, a touch-sensitive V-Pot with an LED ring, a scribble strip display, and buttons such as SELECT, SOLO, MUTE and REC. Because every strip is identical, once you understand one, you understand them all.

The Master Section

To the right of the eight channels is the master section. It includes the master fader and additional navigation controls.

Assignment Buttons

The TRACK, SEND, PAN, PLUG-IN, EQ and INSTRUMENT buttons change what the V-Pots control.

GLOBAL VIEW

The USER button in GLOBAL VIEW opens Browser Mode when using DrivenByMoss.

Reality Check

If the number of buttons feels intimidating, remember that you rarely use them all at once.

Before You Continue...

The next chapter introduces the mental model that makes the X-Touch predictable.

Chapter 3 - The Mental Model

Most difficulties people experience with the X-Touch come from thinking of it as a mixer with fixed controls. It isn't. The X-Touch is better understood as a window into Bitwig.

Bitwig Is Always in Charge

The X-Touch follows Bitwig's current state. If Bitwig changes, the controller reflects that change.

Selection Comes First

Before adjusting anything ask: What is currently selected? Successful workflows nearly always begin with SELECT.

Modes Change Meaning

The same controls perform different jobs in different contexts. Understanding modes is more important than memorising combinations.

Always Read the Feedback

Motor faders, LED rings and scribble strips constantly tell you what Bitwig is doing.

A Simple Workflow

SELECT -> Observe -> Adjust -> Listen

Notice that adjustment comes third, not first.

> Reality Check
> If the X-Touch behaves unexpectedly, check the selected track, current mode and scribble strips first.

Field Note
The best X-Touch users don't memorise hundreds of functions. They ask, "What is Bitwig telling me right now?"

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 4 - Banks and Channels

Banks and Channels

The X-Touch has eight channel strips.

A Bitwig project can have far more than eight tracks.

So one of the first important ideas to understand is this:

The eight channel strips are a window onto the project.

They do not permanently belong to Tracks 1–8.

They represent whichever eight tracks are currently visible to the controller.

The Eight-Channel Window

Imagine a Bitwig project containing 24 tracks:

```
Project

 1   2   3   4   5   6   7   8
 9  10  11  12  13  14  15  16
17  18  19  20  21  22  23  24
```

The X-Touch can physically show eight of them at once.

Initially, that might be:

```
Bitwig Project
──────────────────────────────────────

┌───────────────────────────────┐
│  1   2   3   4   5   6   7   8│  │
└───────────────────────────────┘
              X-Touch
```

The faders, V-Pots, MUTE, SOLO, ARM and SELECT buttons all refer to those eight tracks.

But the window can move.

BANK — Move Eight Tracks at a Time

The BANK buttons move the track-bank focus by eight tracks.

Press:

```
BANK >
```

and the controller moves to the next group of eight.

So:

```
Before

┌───────────────────────────────┐
│  1   2   3   4   5   6   7   8  │
└───────────────────────────────┘
```

becomes:

```
After BANK >

┌───────────────────────────────┐
│  9  10  11  12  13  14  15  16 │
└───────────────────────────────┘
```

Press BANK > again:

```
┌───────────────────────────────┐
│ 17  18  19  20  21  22  23  24 │
└───────────────────────────────┘
```

BANK < moves in the opposite direction.

BANK Is the Large Movement

A useful mental model is:

```
BANK
  │
  ▼
move by one whole surface
```

Because the X-Touch has eight channel strips:

```
BANK
  │
  ▼
8 tracks
```

This is the fastest way to move through a large project.

CHANNEL — Move One Track at a Time

The CHANNEL buttons move the track-bank focus by one track.

Suppose the controller currently shows:

```
1   2   3   4   5   6   7   8
```

Press:

```
CHANNEL >
```

and the window moves by one:

```
2   3   4   5   6   7   8   9
```

Press it again:

```
3   4   5   6   7   8   9  10
```

CHANNEL < moves the window back one track.

CHANNEL Is the Small Movement

So the simplest distinction is:

```
BANK
   → move 8

CHANNEL
   → move 1
```

Or:

```
BANK
   → coarse navigation

CHANNEL
   → fine navigation
```

That is the fundamental relationship between the two pairs of buttons.

BANK and CHANNEL Work Together

Suppose you have 40 tracks and want to reach Track 19.

You could press CHANNEL > eighteen times.

That would work.

But it would be tedious.

Instead:

```
Tracks 1–8
    │
    │ BANK >
    ▼
Tracks 9–16
    │
    │ BANK >
    ▼
Tracks 17–24
```

Now Track 19 is visible.

BANK gets you into the right area.

CHANNEL lets you make smaller adjustments when necessary.

A useful rule is:

BANK for distance. CHANNEL for precision.

The Hardware Does Not Belong to Particular Tracks

This is worth emphasising.

Physical Channel Strip 1 is not:

Track 1.

It means:

The first track in the controller's current bank.

Likewise, Physical Channel Strip 8 means:

The eighth track in the current bank.

So:

```
Bank 1

Strip     1   2   3   4   5   6   7   8
Track     1   2   3   4   5   6   7   8
```

then:

```
Bank 2

Strip     1   2   3   4   5   6   7   8
Track     9  10  11  12  13  14  15  16
```

The physical strips remain where they are.

Their targets change.

Watch the Scribble Strips

This is why the scribble strips are so important.

After changing bank, do not continue thinking:

```
Fader 1 = previous Track 1
```

Look at the display.

It tells you what Fader 1 represents now.

For example:

```
Before BANK >

Kick   Snare   Hats   Bass   Pad   Lead   Vox   FX
```

then:

```
After BANK >

Room   Perc   Piano   Gtr1   Gtr2   BVox   Verb   Delay
```

The controls have not moved.

Their context has.

This gives us one of the most important habits in Project XTC:

Observe before you adjust.

Banking Does Not Select a Track

Moving the bank and selecting a track are different ideas.

BANK and CHANNEL change:

Which tracks are available on the eight channel strips?

SELECT changes:

Which particular track has focus?

So:

```
BANK / CHANNEL
      │
      ▼
move the window
```

whereas:

```
SELECT
   │
   ▼
choose something inside the window
```

This distinction becomes increasingly important later.

Selection Can Move with the Window

Suppose Track 6 is selected.

Your current bank is:

```
1   2   3   4   5  [6]  7   8
```

You then move to:

```
9  10  11  12  13  14  15  16
```

Track 6 still exists in the project.

It is simply no longer represented by one of the eight visible channel strips.

This distinction between:

```
selected track
```

and:

```
tracks currently exposed by the bank
```

is useful when understanding more advanced controller behaviour.

The Window Can Overlap

CHANNEL navigation shows us that banks are not necessarily isolated blocks.

For example:

```
1  2  3  4  5  6  7  8
```

followed by CHANNEL > gives:

```
2  3  4  5  6  7  8  9
```

So the controller window can overlap its previous position.

This is why it is better to think in terms of a movable eight-track window than a set of fixed pages.

A Visual Model

Imagine a long strip of tracks:

```
01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20
```

The X-Touch window begins here:

```
[01 02 03 04 05 06 07 08] 09 10 11 12 13 14 15 16 17 18 19 20
```

BANK >:

```
01 02 03 04 05 06 07 08 [09 10 11 12 13 14 15 16] 17 18 19 20
```

CHANNEL >:

```
01 02 03 04 05 06 07 08 09 [10 11 12 13 14 15 16 17] 18 19 20
```

BANK changes the window dramatically.

CHANNEL nudges it.

Banking and Large Projects

Suppose a project contains:

```
Drums
Bass
Guitar
Piano
Pad
Lead
Vocal
Backing Vocal
Percussion
Strings
FX 1
FX 2
...
```

With a mouse, you might scroll horizontally through the mixer.

With the X-Touch:

```
BANK >
```

is the equivalent large-scale movement.

The eight physical strips remain a stable workspace while the project moves through them.

This can become much faster than visually hunting for a track.

Track Order Matters

Because BANK and CHANNEL follow the Bitwig track bank, the order of tracks in the project affects the physical workflow.

For example, this:

```
Kick
Snare
Hats
Percussion
Bass
Guitar
Keys
Vocal
```

may make more physical sense than:

```
Kick
Vocal
Keys
Snare
Guitar
Hats
Bass
Percussion
```

if the first arrangement reflects how you naturally think about the mix.

This is not an X-Touch rule.

It is a workflow observation:

A logically organised Bitwig project is easier to navigate from hardware.

OPTION Changes BANK and CHANNEL

BANK and CHANNEL have another set of functions when used with OPTION.

These are important because they do not navigate the track bank.

Instead, they physically reorganise things in the Bitwig project.

The verified DrivenByMoss mappings are:

```
OPTION + BANK <
   → Move selected device left

OPTION + BANK >
   → Move selected device right
```

and:

```
OPTION + CHANNEL <
   → Move selected track left

OPTION + CHANNEL >
   → Move selected track right
```

This is a significant distinction.

Without OPTION:

```
BANK / CHANNEL
   → navigate
```

With OPTION:

```
BANK / CHANNEL
   → move project objects
```

OPTION + CHANNEL — Move the Selected Track

Suppose the project order is:

```
Drums   Bass   Guitar   Vocal
```

and Guitar is selected:

```
Drums   Bass  [Guitar]  Vocal
```

Use:

```
OPTION + CHANNEL <
```

and the selected track moves to the left.

Conceptually:

```
Before

Drums   Bass  [Guitar]  Vocal


OPTION + CHANNEL <


After

Drums  [Guitar]  Bass   Vocal
```

Likewise:

```
OPTION + CHANNEL >
```

moves the selected track to the right.

This is not scrolling.

It is reordering the project.

Navigation Versus Reorganisation

This difference deserves a clear mental separation:

```
CHANNEL >
   → show the next track position
```

but:

```
OPTION + CHANNEL >
   → move the selected track
     to the right
```

One changes your view.

The other changes the project.

Because moving tracks is consequential, use the modified command deliberately.

OPTION + BANK — Move the Selected Device

OPTION + BANK performs the corresponding operation at device level.

Suppose a track contains:

```
EQ  →  Compressor  →  Delay
```

and Compressor is selected.

Use:

```
OPTION + BANK <
```

to move the selected device left.

Conceptually:

```
Before

EQ  →  [Compressor]  →  Delay


OPTION + BANK <


After

[Compressor]  →  EQ  →  Delay
```

And:

```
OPTION + BANK >
```

moves the selected device to the right.

Again, this is not controller navigation.

The device itself is being moved within the Bitwig device chain.

Why BANK for Devices and CHANNEL for Tracks?

At first, this may seem slightly odd:

```
OPTION + BANK
   → move device

OPTION + CHANNEL
   → move track
```

But it becomes easier to remember once Device Mode is introduced.

BANK already acquires a strong relationship with devices there.

CHANNEL remains associated with movement at a finer level.

For now, it is enough to learn the verified mapping rather than trying to force every command into a perfect analogy.

Device Mode Is the Important Exception

So far we have said:

```
BANK
   → move 8 tracks

CHANNEL
   → move 1 track
```

That is correct during normal track-oriented operation.

But DrivenByMoss explicitly makes Device Mode an exception.

In Device Mode:

```
BANK <
   → Previous Device

BANK >
   → Next Device
```

while:

```
CHANNEL <
   → Previous Parameter Page

CHANNEL >
   → Next Parameter Page
```

So the same buttons change meaning according to context.

Normal Context Versus Device Context

Outside Device Mode:

```
BANK
   → 8 tracks

CHANNEL
   → 1 track
```

Inside Device Mode:

```
BANK
   → devices

CHANNEL
   → parameter pages
```

This is not an inconsistency.

It is an example of the X-Touch's contextual design.

The controls are reused for the kind of navigation appropriate to the current mode.

BANK Means "Larger Step"

There is a useful conceptual pattern here.

In ordinary track navigation:

```
BANK
   → large movement through tracks
```

In Device Mode:

```
BANK
   → movement between whole devices
```

Meanwhile:

```
CHANNEL
```

handles the finer-grained movement:

```
Normal
   → individual tracks

Device Mode
   → individual parameter pages
```

So even though the literal targets change, the broad relationship remains:

BANK moves at the larger structural level; CHANNEL moves at the finer level.

That is a useful idea to carry into later chapters.

Context Determines Meaning

This gives us an early example of one of the most important ideas in the entire guide.

A physical button does not necessarily have one permanent function.

Its meaning depends on context.

For BANK >:

```
Normal track context
       │
       ▼
move track bank by 8
```

but:

```
Device Mode
       │
       ▼
select next device
```

and:

```
OPTION + BANK >
       │
       ▼
move selected device right
```

One physical button.

Three related but distinct operations.

Do Not Memorise All Three at Once

At this stage of the guide, the most important pair is still:

```
BANK
   → 8 tracks

CHANNEL
   → 1 track
```

Learn that first.

The other meanings will become easier when we reach the relevant chapters.

For now, simply remember:

BANK and CHANNEL are contextual navigation controls, and modifiers can turn navigation into editing.

Banks and Hierarchy

Later we will introduce another dimension: hierarchical navigation.

A project might contain:

```
Project
   │
   ├── Drums
   │     ├── Kick
   │     ├── Snare
   │     └── Hats
   │
   ├── Bass
   └── Synths
```

When working hierarchically, entering the Drums Group changes the level represented by the controller.

BANK and CHANNEL then navigate the tracks available at that level.

So we eventually gain two kinds of movement:

```
BANK / CHANNEL
      → across the current level

SELECT navigation
      → between levels
```

This becomes extremely powerful in large projects.

We will explore it properly in Chapter 17.

The Window Model Still Holds

Even after adding Device Mode, Groups, Layers and other contexts, the original window idea remains useful.

The X-Touch has a limited number of physical controls.

DrivenByMoss decides what part of Bitwig those controls currently expose.

Conceptually:

```
Large Bitwig world
────────────────────────────────────

          ┌──────────────┐
          │   X-Touch    │
          │    window    │
          └──────────────┘
```

Sometimes that window contains tracks.

Sometimes devices.

Sometimes parameters.

Sometimes Layers or Drum Pads.

The hardware stays finite.

The thing it can explore does not.

A Practical Exercise

Create or open a Bitwig project with at least twelve tracks.

Start with the first eight visible on the X-Touch.

1. Press BANK >

Watch the scribble strips.

You should see the track bank move by eight positions.

2. Press BANK <

Return to the original area.

3. Press CHANNEL >

Watch the window move by one track.

4. Press CHANNEL > several more times

Notice that the eight-strip window overlaps its previous position.

5. Press CHANNEL <

Move back one track at a time.

Do this without touching the mouse.

The aim is to make:

```
BANK = large move
```

```
CHANNEL = small move
```

feel obvious rather than remembered.

A Second Exercise

Once the basic navigation is comfortable, select a track and try:

```
OPTION + CHANNEL <
```

and:

```
OPTION + CHANNEL >
```

Watch Bitwig carefully.

The selected track itself should move.

Then compare that with plain CHANNEL.

The distinction should become very clear:

```
CHANNEL
   → I move my view
```

```
OPTION + CHANNEL
   → I move the track
```

This is an excellent example of why modifiers deserve deliberate use.

The Important Idea

The X-Touch does not have eight permanent tracks.

It has an eight-channel window onto Bitwig.

In normal track-oriented operation:

```
BANK <
BANK >
   → move track-bank focus by 8

CHANNEL <
CHANNEL >
   → move track-bank focus by 1
```

With OPTION:

```
OPTION + BANK
   → move selected device left / right

OPTION + CHANNEL
   → move selected track left / right
```

And in Device Mode:

```
BANK
   → previous / next device

CHANNEL
   → previous / next parameter page
```

So the deeper mental model is:

```
Physical Control
       +
Current Context
       +
Modifier, if any
       =
Current Function
```

For now, though, remember the simplest version:

BANK for distance. CHANNEL for precision.

Coming Next

We now know how eight physical channel strips can move through a project containing many more than eight tracks.

But navigation alone does not tell us what the controls mean at any particular moment.

The X-Touch can change the role of its V-Pots and other controls depending on what we are trying to edit.

Next:

Modes.

Chapter 5 - Modes

The X-Touch is often described as having "lots of buttons". That is true, but it is only half the story.

The more important fact is that many of those buttons do different jobs at different times.

Understanding modes is the key to understanding the X-Touch.

Once this idea becomes familiar, the controller feels logical rather than mysterious.

What Is a Mode?

A mode changes the meaning of one or more controls.

The physical hardware never changes.

Instead, the X-Touch changes how it interprets your button presses, encoder turns and fader movements.

Think of the controller as speaking several different "languages".

Each mode uses the same hardware to perform a different set of tasks.

> Field Note
> A common mistake is to think that every button has exactly one purpose.
>
> On the X-Touch, many controls are intentionally multifunctional.

Figure 5.1 illustrates this idea. Although the hardware remains exactly the same, selecting a different mode changes the role of the controls rather than the controls themselves.

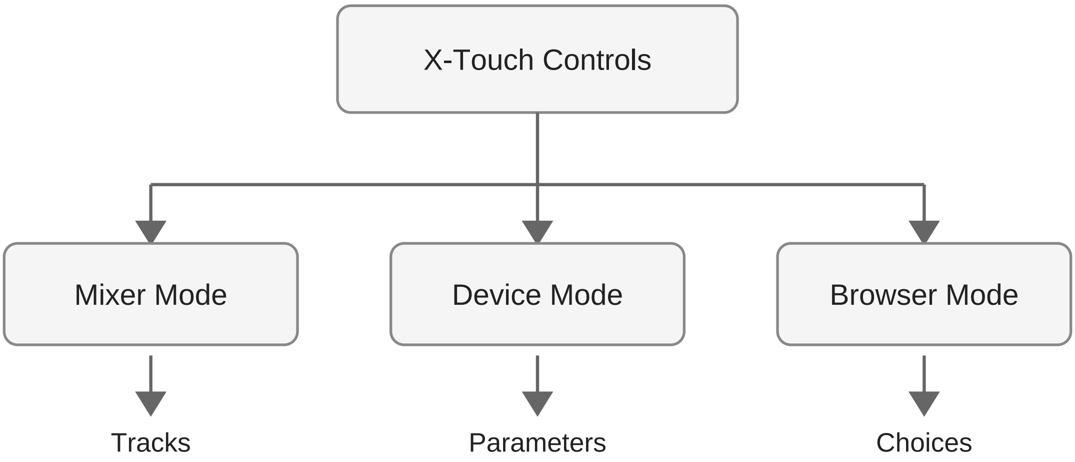

Figure 5.1 — The X-Touch uses the same physical controls in many different operating modes.

The Current Mode Is Always Visible

Fortunately, the X-Touch rarely leaves you guessing.

The scribble strips, LEDs and button illumination usually indicate the current operating mode.

Before pressing a control, take a quick look at the feedback the controller is already providing.

Experienced users develop this habit very quickly.

The Primary Modes

DrivenByMoss makes extensive use of the X-Touch's mode system.

The most frequently used modes include:

- TRACK
- SEND
- PAN/SURROUND
- PLUG-IN
- EQ
- INSTRUMENT
- USER

Each mode changes what the rotary encoders control and, in some cases, what appears on the scribble strips.

The important point is that these are not different layouts of the controller. They are simply different interpretations of the same hardware.

Throughout the remainder of this book we shall examine each of these modes in detail.

TRACK Mode

TRACK mode is the mode most users spend the majority of their time in.

The controller focuses on the Bitwig tracks themselves.

Typical operations include:

- selecting tracks
- adjusting volume
- muting
- soloing
- recording
- automation

Think of TRACK mode as the controller's "home".

Whenever you become unsure where you are, returning to TRACK mode is often a sensible starting point.

SEND Mode

SEND mode changes the rotary encoders so that they adjust send levels instead of pan.

Instead of controlling the stereo position of each track, the encoders now control how much of the signal is sent to an effects bus such as a reverb or delay.

Exactly which send is being adjusted depends upon the current Bitwig project and the selected send.

> Reality Check
> If turning an encoder suddenly changes a reverb or delay send instead of the pan position, the controller is almost certainly in SEND mode.

PAN/SURROUND Mode

PAN/SURROUND mode returns the rotary encoders to controlling the stereo (or surround) position of each track.

For most stereo projects this simply means moving sounds left or right within the mix.

PLUG-IN Mode

PLUG-IN mode is one of the X-Touch's most powerful capabilities.

Rather than controlling the mixer, the encoders are assigned to parameters exposed by the currently selected Bitwig device.

The scribble strips display parameter names, allowing the hardware to become a tactile extension of the software interface.

Later chapters explore this workflow in depth.

INSTRUMENT Mode

INSTRUMENT mode provides direct access to software instrument parameters.

Depending upon the selected Bitwig device, the encoders may control filter cutoff, resonance, envelopes, oscillator settings or many other synthesis controls.

The exact assignments depend entirely upon the instrument currently in focus.

USER Mode

USER mode deserves special attention.

Unlike the other operating modes, USER mode is not tied to a particular mixer function.

DrivenByMoss uses USER mode to expose additional Bitwig features.

For example, pressing USER immediately opens Bitwig's Browser. The scribble strips are repurposed to display browser information, allowing sounds and presets to be selected directly from the X-Touch.

> Field Note
> Because USER mode is completely software-defined, it is capable of doing far more than its name might suggest.
>
> Many users overlook it at first, yet it quickly becomes one of the most frequently used buttons on the controller.

Modes Are Temporary

Changing mode does not permanently alter the controller.

It simply changes what the controls do at that moment.

You are free to move between modes whenever your workflow requires.

The motor faders, scribble strips and LEDs update automatically to reflect the current context.

Thinking in Modes

One of the biggest steps towards mastering the X-Touch is learning to ask a simple question:

"What mode am I in?"

Many apparent "problems" disappear the moment you realise that the controller is behaving exactly as expected—it is simply operating in a different mode.

Once this becomes second nature, the X-Touch feels far less complicated than it first appears.

> Exercise
> Open any Bitwig project.
>
> Press each of the available mode buttons in turn and observe how the scribble strips, encoder assignments and button illumination change.
>
> Do not try to memorise every function immediately.
>
> Instead, become comfortable with the idea that the X-Touch continually adapts its controls to the task at hand.
>
> Recognising the current mode is far more valuable than memorising individual button assignments.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 6 - The SELECT Button

The SELECT Button

Every X-Touch channel strip has a SELECT button.

At first glance, its purpose seems obvious:

```
SELECT
   |
   ▼
Select Track
```

And that is indeed its most basic function.

But in DrivenByMoss, SELECT is considerably more important than that.

It can:

- select a track;
- enter a Group;
- leave a Group;
- enter Layers or Drum Pads;
- expand or collapse a Group;
- participate in multi-selection;
- stop a playing clip;
- open or close a Group folder;
- choose a new clip length;
- select a Send.

So SELECT is not merely:

Which track do I want?

It is one of the X-Touch's principal navigation and context controls.

Basic Selection

Suppose the X-Touch currently represents:

```
1       2       3       4       5       6       7       8

Kick    Snare   Hats    Bass    Pad     Lead    Vox     FX
```

Press SELECT beneath Bass:

```
Kick    Snare   Hats   [Bass]   Pad     Lead    Vox     FX
```

Bass becomes the selected track in Bitwig.

That selection matters because many other X-Touch operations act on:

the currently selected track

For example, Device Mode needs to know which track's devices you want to edit.

SELECT Establishes Focus

A useful mental model is:

```
Many Tracks
     │
     ▼
   SELECT
     │
     ▼
One Track Has Focus
```

The SELECT button answers:

Which track am I interested in right now?

This makes it different from BANK and CHANNEL.

BANK and CHANNEL move the eight-channel window.

SELECT chooses something within that window.

```
BANK / CHANNEL
      │
      ▼
Where am I looking?


SELECT
   │
   ▼
What am I working on?
```

That distinction is fundamental.

Selection and the Eight-Channel Window

Imagine the controller currently shows:

```
Kick   Snare   Hats   Bass   Pad   Lead   Vox   FX
```

You press SELECT beneath Vocal:

```
Kick   Snare   Hats   Bass   Pad   Lead  [Vox]  FX
```

The X-Touch now has a selected track.

If you subsequently move the bank, Vocal may disappear from the eight visible strips.

But that does not necessarily mean the selection itself has disappeared.

This is why it is useful to distinguish:

```
controller bank
```

from:

```
selected track
```

The bank determines what the surface currently exposes.

SELECT determines what has focus.

SELECT Has a Second Role

When hierarchical track navigation is enabled in DrivenByMoss, pressing SELECT on an already selected Group does something different.

It enters that Group.

The sequence is:

```
SELECT Group
      |
      ▼
Group becomes selected
      |
      ▼
SELECT same Group again
      |
      ▼
Enter Group
```

This is one of the most important navigation patterns on the X-Touch.

Selecting Versus Entering

Suppose the top level of the project contains:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

and Drums is a Group.

The first press:

```
SELECT Drums
```

means:

Select the Drums Group.

The second press:

```
SELECT Drums again
```

means:

Enter the Drums Group.

The controller might then show:

```
Kick   Snare   Hats   Toms   Perc   Room
```

The same physical channel strips now represent tracks inside the Group.

SELECT Creates Hierarchical Navigation

Conceptually:

```
Project
   │
   ├── Drums
   │     │
   │     ├── Kick
   │     ├── Snare
   │     ├── Hats
   │     └── Percussion
   │
   ├── Bass
   ├── Guitar
   └── Vocal
```

At the top level:

```
Drums   Bass   Guitar   Vocal
```

Select Drums:

```
[Drums]   Bass   Guitar   Vocal
```

Select Drums again:

```
Kick   Snare   Hats   Percussion
```

The X-Touch has moved down one level in the project hierarchy.

Leaving a Group

To leave the current Group in hierarchical navigation:

Long-press any track SELECT button.

Conceptually:

```
Inside Drums

Kick   Snare   Hats   Perc
             │
             │ long-press SELECT
             ▼

Project Level

Drums   Bass   Guitar   Vocal
```

Notice the wording:

any SELECT button

You do not need to find a special Back button.

The long press itself means:

Leave this level.

A Compact Navigation Vocabulary

This gives us a remarkably small set of gestures:

```
SELECT
   │
   ▼
Focus

SELECT again
   │
   ▼
Enter

Long-press SELECT
   │
   ▼
Leave
```

Or, even more compactly:

```
SELECT
   → focus

SELECT again
   → descend

Long SELECT
   → ascend
```

This pattern becomes extremely useful later.

SELECT and Flat Navigation

DrivenByMoss also supports Flat track navigation.

In Flat mode, the controller does not use Groups as hierarchical levels in the same way.

Instead, all tracks are presented in a flat navigation structure.

That changes what happens when an already selected Group track is selected again.

In Flat mode:

```
SELECT selected Group again
          │
          ▼
Toggle Group expanded state
```

So the same gesture has two related meanings depending on the Track Navigation preference.

Hierarchical Versus Flat

In Hierarchical navigation:

```
SELECT Group
      ↓
SELECT Group again
      ↓
Enter Group
```

In Flat navigation:

```
SELECT Group
      ↓
SELECT Group again
      ↓
Expand / Collapse Group
```

This is an important example of configuration changing controller behaviour.

If two X-Touch users describe different results from pressing SELECT twice on a Group, both may be correct.

Their DrivenByMoss Track Navigation setting may differ.

SELECT Also Enters Layers and Drum Pads

Groups are not the only structures that SELECT can enter.

If the selected track contains an instrument with Layers or Drum Pads, pressing its SELECT button again can enter the corresponding Layer / Drum Pad mode.

Conceptually:

```
Instrument Track
      │
      │ SELECT
      ▼
Selected
      │
      │ SELECT again
      ▼
Layers / Drum Pads
```

The eight channel strips can then represent the contents of the instrument.

For example:

```
Kick   Snare   Hat   Clap   Rim   Tom   Shaker   Perc
```

rather than eight ordinary project tracks.

Leaving Layers or Drum Pads

The exit gesture remains the same:

```
Long-press any SELECT
          │
          ▼
Leave Layer / Drum Pad mode
```

So the hierarchical vocabulary remains consistent:

```
SELECT again
   → enter

Long SELECT
   → leave
```

This consistency is one reason SELECT is worth understanding early in the guide.

SELECT Is Contextual

We can now see that a normal SELECT press does not have only one possible effect.

Its meaning depends on what is currently selected and what that track contains.

For example:

```
Ordinary Track
      │
      ▼
SELECT
      │
      ▼
Select Track
```

but:

```
Selected Group
      │
      ▼
SELECT again
      │
      ▼
Enter Group
```

and:

```
Selected Track
containing Layers
      │
      ▼
SELECT again
      │
      ▼
Enter Layers
```

and similarly for Drum Pads.

The physical button stays the same.

The context determines what the action means.

SHIFT + SELECT — Multi-Selection

SELECT also participates in modifier combinations.

DrivenByMoss documents:

```
SHIFT + SELECT
      │
      ▼
Multi-select Tracks
```

where multi-selection is supported by the DAW.

So normal SELECT says:

Work with this track.

SHIFT + SELECT says:

Add this track to, or otherwise participate in, a multiple-track selection.

Conceptually:

```
SELECT Track 2

Track 2
   │
   ▼
selected
```

then:

```
SHIFT + SELECT Track 4
```

allows a multi-track selection rather than simply replacing the original selection.

SHIFT + SELECT Has Another Important Context

There is another documented use of SHIFT + the Track SELECT buttons:

Selecting the New Clip Length.

The eight SELECT buttons correspond to:

```
SHIFT + SELECT 1   → 16 bars

SHIFT + SELECT 2   → 8 bars

SHIFT + SELECT 3   → 4 bars

SHIFT + SELECT 4   → 2 bars
SHIFT + SELECT 5   → 1 bar
```

```
SHIFT + SELECT 6   → 2 beats

SHIFT + SELECT 7   → 1 beat

SHIFT + SELECT 8   → 32 bars
```

This means SHIFT + SELECT is another contextual combination.

In an appropriate selection context it participates in track multi-selection.

It is also used by DrivenByMoss to choose the length used when creating new clips.

We will return to New Clip Length in the recording chapters.

OPTION + SELECT — Stop a Clip

OPTION gives SELECT a very different purpose.

DrivenByMoss documents:

```
OPTION + SELECT
       │
       ▼
Stop the playing clip
on that track
```

Suppose several Launcher clips are playing:

```
Drums    Bass    Keys    Vocal
  ▶        ▶       ▶       ▶
```

Use:

```
OPTION + SELECT
```

on Keys.

The command targets the playing clip on that specific track.

This is a performance-oriented function rather than a selection function.

The Modifier Changes the Question

Without OPTION:

```
SELECT
   │
   ▼
Which track do I want?
```

With OPTION:

```
OPTION + SELECT
       │
       ▼
Which track's playing clip
do I want to stop?
```

This is a useful example of the modifier system.

The physical location still identifies the track.

The modifier changes the action performed on it.

CONTROL + SELECT — Open or Close a Group

CONTROL gives SELECT another Group-related function.

DrivenByMoss documents:

```
CONTROL + SELECT
        │
        ▼
Open / Close Group Folder
```

when the corresponding track is a Group.

This should not be confused with hierarchical entry.

Entering and Opening Are Different

With hierarchical navigation:

```
SELECT selected Group again
          │
          ▼
Enter Group
```

means:

Make the Group's contents the controller's current navigation level.

Whereas:

```
CONTROL + SELECT
```

means:

Open or close the Group folder in Bitwig.

Conceptually:

```
SELECT again
   → controller navigation

CONTROL + SELECT
   → Group folder state
```

These are related ideas, but they are not the same operation.

ALT + SELECT — Set New Clip Length

DrivenByMoss also documents:

```
ALT + SELECT
     │
```

```
      ▼
Set the length of a new clip
```

This associates the eight SELECT buttons with clip-length choices.

The exact clip-length choices documented for the Track SELECT row are:

```
1   → 16 bars

2   → 8 bars

3   → 4 bars

4   → 2 bars

5   → 1 bar

6   → 2 beats

7   → 1 beat

8   → 32 bars
```

This is particularly useful for Launcher-oriented recording.

Instead of opening a settings dialog before creating a clip, the surface can participate directly in choosing its length.

SEND + SELECT — Choose the Send

SELECT also works with the SEND mode button.

DrivenByMoss documents:

```
SEND + SELECT 1
   → Send 1

SEND + SELECT 2
   → Send 2

...

SEND + SELECT 8
   → Send 8
```

So the SELECT row becomes a direct Send selector.

This is an important pattern.

Normally the eight SELECT buttons correspond to eight tracks.

But when SEND is held:

```
SELECT row
     │
     ▼
Send choices 1–8
```

The same physical row is temporarily repurposed.

A Row of Eight Choices

This gives us a broader way to think about the SELECT buttons.

Sometimes they mean:

```
Track 1
Track 2
Track 3
...
Track 8
```

But with another control held, the same row can become:

```
Choice 1
Choice 2
Choice 3
...
Choice 8
```

For example:

```
SEND + SELECT
      │
      ▼
Send 1–8
```

or when setting clip length:

```
modifier + SELECT
       │
       ▼
one of eight clip lengths
```

The SELECT row is therefore not merely eight copies of a Track Select command.

It is also an eight-choice command surface.

Why SELECT Is So Powerful

The X-Touch has eight identical channel strips.

That gives DrivenByMoss a ready-made row of eight physical targets.

SELECT naturally identifies:

```
this one
```

So the software can reuse that gesture in many contexts.

For example:

```
SELECT
   → this track

OPTION + SELECT
   → stop clip on this track
CONTROL + SELECT
```

```
   → open/close this Group

SEND + SELECT
   → choose this Send number

clip-length modifier + SELECT
   → choose this clip length
```

The physical position provides the number or target.

The context provides the meaning.

SELECT and Fader Touch

There is another relationship worth understanding.

DrivenByMoss can be configured so that touching a motor fader automatically selects its track.

With that preference enabled:

```
Touch Fader
     │
     ▼
Select Track
```

So there are two physical ways of establishing track focus:

```
SELECT button
```

or:

```
touch fader
```

The SELECT button remains the explicit choice.

Fader-touch selection can make mixing workflows faster because simply reaching for a channel can establish its focus.

We will discuss the preference itself in Chapter 21.

The Master Fader Can Select Too

The same principle applies to the Master fader.

DrivenByMoss documents that touching the Master fader selects the Master track.

Conceptually:

```
Touch Master Fader
        │
        ▼
Master Track Selected
```

That selection can then place the controller into the Master editing context.

So selection is not limited strictly to the eight channel SELECT buttons.

But those eight buttons remain the controller's primary explicit selection mechanism.

SELECT Before Acting

A useful X-Touch habit is:

Establish focus before performing a contextual action.

For example:

```
SELECT Synth
     ↓
DEVICE
     ↓
edit synth
```

or:

```
SELECT Vocal
     ↓
SEND
     ↓
adjust delay
```

or:

```
SELECT Drum Group
     ↓
SELECT again
     ↓
work inside Drums
```

The sequence is often:

```
Focus
   ↓
Context
   ↓
Action
```

SELECT frequently performs the first step.

Avoid Accidental Second Presses

Because SELECT can have a second-stage meaning, it is worth being deliberate when pressing an already selected track.

On an ordinary track, another press may have little consequence.

But on a Group, Layer-capable instrument or Drum Machine, a second press may change the controller's navigation context.

So:

```
first SELECT
   → select
```

and:

```
second SELECT
   → potentially enter
```

are worth thinking of as separate gestures.

If the Surface Suddenly Changes

Suppose you press SELECT and the scribble strips unexpectedly change from:

```
Drums   Bass   Guitar   Keys   Vocal
```

to:

```
Kick   Snare   Hats   Perc   Room
```

The controller has probably not malfunctioned.

You may simply have entered a Group or Layer/Drum Pad context.

The recovery gesture is:

```
Long-press any SELECT
```

to leave that context.

This is worth remembering because it gives you a simple escape route when learning hierarchical navigation.

ENTER and CANCEL Are Not Hierarchy Controls

It is particularly important to clarify one thing.

The X-Touch has ENTER and CANCEL buttons.

It might seem natural to assume:

```
ENTER
   → go into Group

CANCEL
   → go back
```

But that is not how DrivenByMoss documents hierarchical Track navigation.

Hierarchy uses:

```
SELECT again
   → enter

Long SELECT
   → leave
```

ENTER and CANCEL have other roles.

For example, in Browser Mode:

```
ENTER
   → confirm Browser selection
```

```
CANCEL
   → discard Browser selection
```

Outside the Browser they behave like the corresponding computer keyboard keys.

Keeping these concepts separate prevents a great deal of confusion.

Hierarchical Navigation as a Tree

The SELECT behaviour becomes particularly intuitive if you imagine the project as a tree.

```
Project
   │
   ├── Drums
   │     │
   │     ├── Kick
   │     ├── Snare
   │     └── Hats
   │
   ├── Bass
   │
   └── Synths
         │
         ├── Pad
         └── Lead
```

BANK and CHANNEL move:

```
across
```

the current level.

SELECT again moves:

```
down
```

into a level.

Long SELECT moves:

```
up
```

out of it.

So:

```
BANK / CHANNEL
      → horizontal navigation

SELECT / Long SELECT
      → vertical navigation
```

This is a powerful mental model for large projects.

SELECT as Navigation

We can therefore expand our original definition.

At first:

```
SELECT
   → choose a track
```

Now:

```
SELECT
   → establish focus

SELECT again
   → enter available structure

Long SELECT
   → leave structure
```

And with modifiers:

```
SHIFT + SELECT
   → additional selection / clip-length function

OPTION + SELECT
   → stop playing clip on track

CONTROL + SELECT
   → open / close Group folder

ALT + SELECT
   → set new clip length

SEND + SELECT
   → select Send 1–8
```

That is a great deal of functionality from one row of buttons.

Do Not Memorise the Modifier Table Yet

At this point, you do not need to memorise every SELECT combination.

The most important behaviour to learn is:

```
SELECT
   → focus
```

followed by:

```
SELECT again
   → enter
```

and:

```
Long SELECT
   → leave
```

Once that is comfortable, the modifier functions can be added gradually.

The dedicated Modifiers chapter will give them a more systematic treatment.

A Practical Exercise

Create or open a project containing several ordinary tracks and at least one Group.

For example:

```
Drums
Bass
Guitar
Keys
Vocal
```

with Drums containing:

```
Kick
Snare
Hats
Percussion
```

Make sure DrivenByMoss Track Navigation is set to Hierarchical.

1. Select Bass

Press its SELECT button.

Watch Bitwig and the X-Touch feedback.

2. Select Drums

Press SELECT beneath the Drums Group.

The Group should become selected.

3. Press Drums SELECT again

The controller should enter the Group.

The scribble strips should now represent its contents.

4. Long-press a SELECT button

The controller should leave the Group.

5. Repeat the process

Do it several times until:

```
SELECT
SELECT again
Long SELECT
```

feels like:

```
focus
enter
leave
```

rather than three commands you have to remember.

A Modifier Exercise

Once normal hierarchical navigation is comfortable, try a few modified SELECT operations separately.

For example:

```
OPTION + SELECT
```

on a track with a playing Launcher clip.

Then try:

```
CONTROL + SELECT
```

on a Group.

Finally, enter Send Mode and try:

```
SEND + SELECT
```

to select different Send channels.

The purpose is not to memorise everything.

It is to notice the recurring pattern:

The SELECT button identifies a position; the modifier or current mode determines what that position means.

The Important Idea

SELECT is one of the most important controls on the X-Touch.

Its basic function is:

```
SELECT
   → select track
```

But it also provides hierarchical navigation:

```
SELECT selected Group again
   → enter Group

Long-press any SELECT
   → leave Group
```

and access to Layers and Drum Pads:

```
SELECT selected containing track again
   → enter Layers / Drum Pads

Long-press any SELECT
   → leave
```

Its modifier functions include:

```
SHIFT + SELECT
   → multi-selection
     / New Clip Length context

OPTION + SELECT
   → stop playing clip on track

CONTROL + SELECT
   → open / close Group folder

ALT + SELECT
   → set New Clip Length

SEND + SELECT
   → choose Send 1–8
```

So SELECT is much more than a track-selection button.

A useful mental model is:

```
SELECT
   │
   ▼
"This one."
```

The current mode, modifier and context determine what “this one” means.

Coming Next

We now know how to move the X-Touch's eight-channel window through the project and how SELECT establishes focus within it.

But the controller still needs to tell us what those physical controls currently represent.

That feedback comes from several places:

• the scribble strips;
• the assignment display;
• LEDs;
• motorised faders;
• V-Pot rings.

Next:

Displays and Feedback.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 7 - Displays and Feedback

Displays and Feedback

The X-Touch does not merely send commands to Bitwig.

Bitwig also sends information back.

That two-way communication is one of the most important differences between the X-Touch and a simple MIDI controller.

A basic MIDI controller might work like this:

```
Turn Knob
    │
    ▼
Bitwig Changes
```

The X-Touch can work like this:

```
Turn Knob
    │
    ▼
Bitwig Changes
    │
    ▼
X-Touch Updates
```

The controller therefore becomes part of the user interface.

It does not merely control Bitwig.

It also tells you what Bitwig is doing.

Feedback Is Part of the Control Surface

The X-Touch provides several forms of feedback:

```
Scribble Strips

V-Pot LED Rings

Button LEDs

Motor Faders

VU Meters

Assignment Display
Segment Display
```

Each communicates a different kind of information.

Together they answer questions such as:

```
What am I controlling?

Which track is selected?

What mode am I in?

What is the current value?

Is this function active?

What is the current level?

Where is the play cursor?
```

This is why learning to read the X-Touch is just as important as learning to operate it.

The Scribble Strips

Above the eight channel strips are the scribble-strip displays.

These are among the most useful parts of the X-Touch.

In a normal mixer context they may show track information such as:

```
Kick    Snare   Hats    Bass    Keys    Lead    Vocal   FX
```

Change bank:

```
BANK >
```

and the displays change with the faders:

```
Perc    Room    Piano   Gtr1    Gtr2    BVox    Verb    Delay
```

The scribble strips tell you what the eight physical channel strips currently represent.

The Displays Follow Context

The same display can mean different things in different modes.

In a mixer context:

```
Track Name
```

In Send Mode:

```
Send Information
```

In Device Mode:

```
Parameter Name
Parameter Value
```

In Browser Mode:

```
Browser Choice
```

In Marker Mode:

```
Marker Information
```

So the displays are contextual.

They should not be thought of as permanent track-name labels.

They are the X-Touch's way of answering:

What do these controls mean right now?

Read Before You Touch

This gives us one of the most useful habits for working with the X-Touch:

Read before you touch.

Before turning a V-Pot:

```
Look at Display
      ↓
Identify Parameter
      ↓
Turn V-Pot
```

Before moving a fader after changing context:

```
Look at Display
      ↓
Identify Channel
      ↓
Move Fader
```

This becomes increasingly important as the controller moves beyond simple mixing.

Abbreviated Names

The displays have limited width.

Long track or parameter names may therefore be shortened.

For example:

```
Supermassive Reverb
```

cannot appear in full.

You may see something more like:

```
SupRvrb
```

The exact abbreviation depends on what Bitwig and DrivenByMoss send.

This is normal.

Over time, frequently used names become surprisingly easy to recognise.

Naming Tracks for Hardware

Because the scribble strips have limited space, sensible track naming helps.

A track named:

```
Main Lead Vocal Double Processed
```

may not communicate very much on a small display.

A shorter name such as:

```
Ld Vox
```

may be much more useful.

Likewise:

```
Kick
Snare
Hat
Bass
Pad
Lead
Vox
Delay
```

work particularly well on a control surface.

You do not have to rename an entire project for the X-Touch.

But concise names can make hardware navigation much easier.

Display Colours

The X-Touch supports coloured scribble-strip backgrounds.

DrivenByMoss can use these as another layer of feedback.

Colour can help distinguish tracks or contexts without requiring you to read every label.

For example, a project might visually separate:

```
Drums
```

```
Bass
```

```
Instruments
```

```
Vocals
```

```
Effects
```

through colour.

The exact colour behaviour depends on the DrivenByMoss hardware and display configuration.

Chapter 21 covers that configuration.

Display Mode

The X-Touch includes a:

```
DISPLAY MODE
```

control.

DrivenByMoss maps this to:

```
DISPLAY MODE
   → Toggle Track Names
     in the First Display
```

This changes what the first display row presents.

It gives you a choice between different kinds of contextual information.

Conceptually:

```
Current Display Information
          │
          │ DISPLAY MODE
          ▼
Track Names
```

and back again.

Why Display Mode Is Useful

There are two questions the surface frequently needs to answer:

```
What tracks am I controlling?
```

and:

```
What mode or parameters am I controlling?
```

Those needs can compete for limited display space.

DISPLAY MODE gives you a way to change the emphasis.

If you momentarily lose track of which channels are represented:

```
DISPLAY MODE
      ↓
Track Names
```

can restore that orientation.

This is especially useful after changing banks or moving into a more specialised controller context.

The Assignment Display

The X-Touch also contains a small assignment display.

DrivenByMoss can use this to show the current controller mode.

For example, the surface may be operating in a context such as:

```
Panorama
```

```
Send
```

```
Device
```

```
EQ
```

```
Browser
```

The assignment display provides another clue to the controller's current state.

This matters because the same physical V-Pot may perform very different jobs in those contexts.

The Segment Display

The large segment display in the Transport area can show transport-related information.

DrivenByMoss can use it to display the play position.

Depending on configuration, this can be represented in forms such as:

```
Time
```

or:

```
Measures / Beats
```

The final digits can also display additional information.

TEMPO / TICKS

DrivenByMoss maps the MCU:

```
TEMPO / TICKS
```

control to toggling the final part of the segment display between:

```
Tempo
```

and:

```
Ticks
```

Conceptually:

```
Segment Display
       │
       ├── Main Position
       │
```

```
    └── Final Digits
            │
            │ TEMPO / TICKS
            ▼
   Tempo ↔ Ticks
```

This lets the same limited display area provide two different kinds of timing information.

Which Display Is More Useful?

There is no universally correct choice.

Tempo may be useful when:

```
setting project speed

matching another piece of music

making fine tempo adjustments
```

Ticks may be more useful when:

```
working with precise musical position
```

The important point is that the hardware display is configurable rather than fixed.

V-Pot LED Rings

Each V-Pot is surrounded by LEDs.

These provide visual feedback about the current parameter value.

For example, a Pan control might appear approximately as:

```
Left      Centre      Right

●●○○○      ○○●○○      ○○○●●
```

A level-style parameter may instead appear more like:

```
Low

●○○○○○○○○○○
```

versus:

```
High

●●●●●●●●●○○
```

The exact LED-ring representation depends on the parameter.

The important point is:

The encoder has no fixed physical position, so the LEDs provide the position information.

Why Endless Encoders Need Feedback

A normal knob has a physical endpoint.

You can look at it and see:

```
about 2 o'clock
```

An endless encoder has no such permanent position.

It can rotate forever.

So its meaning has to come from elsewhere:

```
Display
   +
LED Ring
```

This makes the combination much more flexible.

The same encoder can represent:

```
Pan
```

```
Send Level
```

```
Device Parameter
```

```
Browser Choice
```

```
Master Parameter
```

without needing to physically jump to a new position.

Button LEDs

Many X-Touch buttons illuminate.

These LEDs often indicate whether a function is active.

For example:

```
MUTE
SOLO
REC
SELECT
REPEAT
```

may provide immediate visual confirmation of state.

This gives us another useful pattern:

```
Press Button
     ↓
Bitwig State Changes
     ↓
LED Reflects State
```

The light is not merely decoration.

It is confirmation.

State Versus Action

Some buttons represent an ongoing state.

For example:

```
MUTE
```

can be:

```
On
```

or:

```
Off
```

An illuminated button is therefore useful.

Other controls represent an action rather than a persistent state.

For example:

```
SAVE
```

performs something and finishes.

There is no long-term "Save Mode" that needs to remain illuminated.

Understanding whether a control represents:

```
State
```

or:

```
Action
```

helps explain the feedback you should expect.

VU Meters

The channel strips include level-meter feedback.

This lets you see signal activity without relying entirely on Bitwig's mixer.

Conceptually:

```
Audio Signal
     │
     ▼
Bitwig Level
     │
     ▼
DrivenByMoss
     │
     ▼
X-Touch VU Meter
```

The meter is another example of information travelling from the DAW back to the hardware.

EDIT / GLOBAL VIEW — Toggle VU Meters

DrivenByMoss maps the MCU Global View / EDIT control to:

```
EDIT / GLOBAL VIEW
   → Toggle VU Meters
```

This lets you switch the meter display on or off from the surface.

Conceptually:

```
VU Meters Off
     │
     │ EDIT
     ▼
VU Meters On
```

and:

```
VU Meters On
     │
     │ EDIT
     ▼
VU Meters Off
```

If the channel displays appear different after pressing EDIT, remember that this control may have changed the meter feedback rather than the mixer itself.

Why Toggle the VU Meters?

Meters are useful when you want to see:

```
Signal Presence
```

```
Relative Level
```

```
Activity Across Tracks
```

But display space is limited.

There may be times when other feedback is more useful than continuous level information.

The toggle lets the surface adapt to the current task.

Motor Faders Are Displays Too

A motor fader is not merely an input control.

Its physical position is also feedback.

Suppose Bitwig reports:

```
Track Volume = -6 dB
```

The fader moves to the corresponding position.

So:

```
Bitwig Value
     │
     ▼
Motor
     │
     ▼
Physical Position
```

The fader itself has become a display.

Changing Banks Demonstrates Feedback

Banking makes this particularly obvious.

Suppose the first bank contains:

```
Kick    Snare   Hats    Bass    Pad     Lead    Vox     FX
```

with eight different volume settings.

Press:

```
BANK >
```

The controller now represents another set of tracks.

The scribble strips change.

The faders move.

The button LEDs change.

Possibly the VU activity changes.

In other words:

```
BANK >
   │
   ▼
New Controller Context
   │
   ├── New Names
   ├── New Fader Positions
   ├── New Button States
   └── New Meter Activity
```

The whole surface updates as one system.

Layout Controls

DrivenByMoss also maps three MCU controls to Bitwig's main application layouts:

```
AUX
   → Arrange Layout
```

```
BUSSES
   → Mix Layout

OUTPUTS
   → Edit Layout
```

These controls do not merely change the X-Touch.

They change the layout shown by Bitwig itself.

AUX — Arrange Layout

Press:

```
AUX
```

to switch Bitwig to:

```
Arrange Layout
```

This is useful when the main task is working with the timeline and the overall song arrangement.

Conceptually:

```
AUX
  ↓
Arrange
```

BUSSES — Mix Layout

Press:

```
BUSSES
```

to switch Bitwig to:

```
Mix Layout
```

This gives the mixer greater visual emphasis.

Conceptually:

```
BUSSES
   ↓
Mix
```

This is a good example of a hardware label inherited from MCU being repurposed for a Bitwig-specific operation.

OUTPUTS — Edit Layout

Press:

```
OUTPUTS
```

to switch Bitwig to:

```
Edit Layout
```

Conceptually:

```
OUTPUTS
    ↓
Edit
```

Again, the printed hardware label is less important than the DrivenByMoss assignment.

Three Physical Routes into Bitwig's Main Views

Together:

```
AUX
   → Arrange

BUSSES
   → Mix

OUTPUTS
   → Edit
```

give the X-Touch direct access to three major Bitwig working layouts.

A useful mental model is:

```
             Bitwig
                │
      ┌─────────┼─────────┐
      │         │         │
   Arrange     Mix       Edit
      ▲         ▲         ▲
      │         │         │
     AUX      BUSSES    OUTPUTS
```

This can reduce another common reason for reaching for the mouse.

Hardware Labels Can Be Misleading

The layout controls provide an excellent example of why Project XTC concentrates on the DrivenByMoss mapping, not simply the text printed on the X-Touch.

A new user seeing:

```
AUX

BUSSES

OUTPUTS
```

might reasonably assume that these buttons navigate mixer channel types.

In DrivenByMoss they instead mean:

```
Arrange

Mix
```

```
Edit
```

So once again:

Do not assume that the printed MCU label describes the Bitwig operation.

The X-Touch supplies the physical controls.

DrivenByMoss supplies their Bitwig meaning.

Feedback and Layout Work Together

The layout buttons reveal an important distinction.

Some X-Touch controls change:

```
what the hardware controls
```

Others change:

```
what Bitwig displays
```

And some operations cause both interfaces to update.

This gives us two related but distinct ideas:

```
Controller Context
```

and:

```
Bitwig Layout
```

Do not assume that changing one necessarily changes the other.

The Screen Is Still Useful

Project XTC aims toward a Mouse-Lite workflow.

That does not mean:

Never look at the computer.

The screen remains extremely useful for:

- detailed waveform editing;
- device interfaces;
- Browser exploration;
- arrangement overview;
- visual automation;
- complex modulation.

The X-Touch's feedback complements the screen.

It does not need to reproduce it.

Hardware Feedback Reduces Screen Checking

The benefit is that many simple questions can be answered without looking away from the controller.

Instead of asking the screen:

```
Which track is this?
```

```
Is it muted?
```

```
What is its level?
```

```
What parameter is this?
```

```
What mode am I in?
```

the X-Touch can often answer directly.

That keeps your attention closer to the controls you are physically using.

Feedback Makes Context Safe

A contextual controller would be dangerous without feedback.

Imagine one V-Pot controlling:

```
Pan
```

then:

```
Send
```

then:

```
Filter Cutoff
```

with no indication of which one was active.

That would be unusable.

Feedback makes context practical:

```
Context Changes
      ↓
Display Changes
      ↓
You Read
      ↓
You Act
```

This is the basic interaction loop of the X-Touch.

Trust the Surface — But Read It

Once the controller is configured correctly, the hardware feedback is there to help you.

If the faders suddenly move after a bank change:

```
That is information.
```

If the scribble strips change after entering Device Mode:

```
That is information.
```

If a button lights:

```
That is information.
```

If the assignment display changes:

```
That is information.
```

The controller is continually telling you what state it is in.

A Practical Display Exercise

Open a project containing at least eight tracks.

1. Read the Scribble Strips

Without looking at Bitwig's mixer, identify the tracks represented by the eight channel strips.

2. Press BANK >

Watch:

```
Track Names
```

```
Fader Positions
```

```
Button LEDs
```

```
VU Activity
```

change.

3. Press BANK <

Watch the previous state return.

4. Press DISPLAY MODE

Observe how the first display changes.

Press it again and compare the information shown.

The aim is to start treating the displays as an active part of navigation rather than passive decoration.

A Practical VU Exercise

Play a section containing several active tracks.

Watch the channel meters.

Now press:

```
EDIT / GLOBAL VIEW
```

Observe the change in VU-meter display.

Press it again.

The important relationship is:

```
EDIT / GLOBAL VIEW
      ↓
VU Feedback Toggle
```

not whatever the printed label might initially suggest.

A Practical Layout Exercise

With a project open, try:

```
AUX
```

then:

```
BUSSES
```

then:

```
OUTPUTS
```

Observe Bitwig move between:

```
Arrange
```

```
Mix
```

```
Edit
```

Repeat the sequence several times.

The goal is to associate the physical button position with the Bitwig layout rather than relying on the printed MCU name.

A Practical Segment-Display Exercise

Watch the Transport segment display.

Press:

```
TEMPO / TICKS
```

Observe the final digits.

Press it again.

The aim is simply to recognise that this part of the display is switchable and to learn which representation is most useful for your normal workflow.

If the Feedback Looks Wrong

If the X-Touch displays or LEDs do not seem to match Bitwig, ask:

```
Am I in the mode I think I am?
```

```
Did I change bank?
```

```
Did I change track?
```

```
Did I change device?
```

```
Is FLIP active?
```

```
Is DISPLAY MODE showing the view I expect?
```

```
Are VU meters enabled?
```

```
Is the controller receiving MIDI feedback?
```

```
Is the correct DrivenByMoss profile configured?
```

Feedback problems are not always display problems.

Sometimes the controller is correctly showing a context you did not realise you had entered.

Configuration Matters

Several aspects of feedback can be configured in DrivenByMoss.

These include things such as:

```
Display Setup
```

```
Display Colours
```

```
VU Behaviour
```

```
Segment Display
```

```
Track Names
```

```
Startup Mode
```

We will deal with those settings in Chapter 21.

For now, the important principle is:

If the X-Touch's feedback differs from what you expect, configuration may be part of the explanation.

A Useful Mental Model

Think of the X-Touch as having two directions of communication.

```
                YOUR ACTIONS
                     |
```

```
                  ▼

               X-Touch
                  │
                  ▼

                Bitwig
```

and:

```
               BITWIG

                  │
                  ▼

               X-Touch
                  │
        ┌─────────┼─────────┐
        │         │         │
        ▼         ▼         ▼
    Displays    Motors     LEDs
```

The first direction is control.

The second is feedback.

A good control surface requires both.

The Important Idea

The X-Touch is not simply a collection of physical inputs.

It is a two-way interface.

Its feedback systems include:

```
Scribble Strips

V-Pot LED Rings

Button LEDs

VU Meters

Motor Faders

Assignment Display

Segment Display
```

DrivenByMoss also gives several physical controls direct display or view functions:

```
DISPLAY MODE
   → Toggle Track Names
     in First Display

TEMPO / TICKS
   → Toggle Tempo / Ticks
```

```
EDIT / GLOBAL VIEW
   → Toggle VU Meters
```

and direct access to Bitwig's main layouts:

```
AUX
   → Arrange

BUSSES
   → Mix

OUTPUTS
   → Edit
```

The central habit is:

```
Context Changes
      ↓
Read Feedback
      ↓
Understand State
      ↓
Act
```

So perhaps the most important rule in this chapter is:

Do not merely operate the X-Touch. Read it.

The surface is continually telling you what it represents.

Learning to notice that information is what makes the controller's increasingly complex contextual behaviour manageable.

Coming Next

Feedback tells us what the controller is doing.

But DrivenByMoss can dramatically change what a control does when a modifier is held.

The same button, encoder or Jog Wheel may acquire an entirely different role through:

```
SHIFT
```

```
OPTION
```

```
CONTROL
```

```
ALT
```

Next:

Modifiers.

Chapter 8 - Modifiers: SHIFT, OPTION, CONTROL and ALT

Modifiers: SHIFT, OPTION, CONTROL and ALT

By now, an important pattern should be becoming familiar.

The X-Touch does not have enough physical controls for every possible function to exist as a dedicated button.

Instead, DrivenByMoss reuses controls according to context.

One of the main ways it does this is through four modifier buttons:

`SHIFT`

`OPTION`

`CONTROL`

`ALT`

Hold one of these while pressing, turning or moving another control, and the meaning of that control may change.

The underlying idea is simple:

A modifier temporarily changes the context of another control.

A Modifier Is a Temporary Context

Suppose a control normally does this:

```
Control
   │
   ▼
Primary Function
```

Hold a modifier:

```
Modifier + Control
       │
       ▼
Alternative Function
```

Release the modifier:

```
Control
   │
```

```
   ▼
Primary Function again
```

So modifiers behave rather like temporary modes.

They extend the controller without requiring more hardware.

Why Modifiers Matter

Imagine the X-Touch without modifiers.

Every secondary function would require:

- another physical button;
- another mode;
- another menu;
- or another trip to the mouse.

Modifiers allow one control to support several related operations.

For example:

```
MARKER
   → enter Marker Mode
```

while:

```
OPTION + MARKER
   → create a marker
```

The same physical button participates in two related tasks.

That is exactly what modifiers are for.

SHIFT

SHIFT is one of the most frequently used modifiers.

It often provides:

- finer adjustment;
- a related secondary function;
- reverse movement;
- access to a Launcher-oriented variant;
- or another function closely related to the unmodified control.

For example:

```
Jog Wheel
   → move play position
```

while:

```
SHIFT + Jog Wheel
   → finer movement
```

Likewise:

```
SEND
   → next Send
```

while:

```
SHIFT + SEND
   → previous Send
```

So SHIFT often means:

Do the same kind of thing, but differently.

SHIFT and Precision

One recurring SHIFT pattern is fine adjustment.

For example:

```
Turn parameter control
      │
      ▼
Normal movement
```

with SHIFT:

```
SHIFT + Turn
      │
      ▼
Finer movement
```

The exact control depends on the mode.

But once you recognise the pattern, SHIFT becomes easier to remember.

SHIFT and Transport

SHIFT also modifies several transport controls.

Verified DrivenByMoss examples include:

```
SHIFT + PLAY
   → Toggle Repeat
```

and:

```
SHIFT + RECORD
   → Toggle Launcher overdub
```

and:

```
SHIFT + OVR
   → Toggle Launcher overdub
```

So SHIFT does not always mean precision.

Its actual meaning remains contextual.

SHIFT and REDO

The UNDO button also follows a familiar modifier pattern:

```
UNDO
   → Undo
```

while:

```
SHIFT + UNDO
   → Redo
```

This is a particularly easy combination to remember because the two actions form an obvious pair.

SHIFT and the Metronome

DrivenByMoss also provides:

```
SHIFT + METRONOME
   → Toggle metronome ticks
```

and:

```
SHIFT + Master Fader
   → Metronome volume
```

These two controls belong to the same general recording and timing workflow.

OPTION

OPTION frequently gives a control an alternative action.

Examples include:

```
OPTION + MARKER
   → Create marker

OPTION + REWIND
   → Previous marker

OPTION + FORWARD
   → Next marker

OPTION + TRACK
   → Pin cursor track

OPTION + DEVICE
   → Pin cursor device
```

OPTION often changes not merely the degree of an operation, but what the operation does.

A useful shorthand is:

OPTION often means: give me the alternative operation.

OPTION + BANK and CHANNEL

Chapter 4 introduced an especially important distinction.

Normally:

```
BANK
   → move through track bank by 8
```

```
CHANNEL
   → move through track bank by 1
```

With OPTION:

```
OPTION + BANK
   → move selected device left / right
```

and:

```
OPTION + CHANNEL
   → move selected track left / right
```

This is a major change in meaning.

Without OPTION:

```
I move my view
```

With OPTION:

```
I move an object in the project
```

That is exactly the kind of difference worth recognising before pressing the modifier casually.

OPTION and the Jog Wheel

DrivenByMoss also gives OPTION a clear Jog Wheel role:

```
OPTION + Jog Wheel
   → Change Tempo
```

Add SHIFT:

```
OPTION + SHIFT + Jog Wheel
   → Fine Tempo adjustment
```

This illustrates how modifiers can combine.

OPTION determines what is being controlled.

SHIFT modifies how precisely it is being controlled.

CONTROL

CONTROL often exposes another structural or specialised function.

For example:

```
CONTROL + Jog Wheel
   → Change Loop Start
```

and:

```
CONTROL + SHIFT + Jog Wheel
   → Fine Loop Start adjustment
```

Again:

```
CONTROL
   → choose the parameter

SHIFT
   → refine the movement
```

CONTROL and Devices

In Device Mode, CONTROL becomes especially useful.

Hold CONTROL and DrivenByMoss exposes the devices on the selected track.

Conceptually:

```
Hold CONTROL
      │
      ▼
Devices shown on V-Pots
      │
      ▼
Press V-Pot
      │
      ▼
Select Device
```

So CONTROL can act as a temporary direct-selection context.

This is much faster than stepping device-by-device when you already know which one you want.

CONTROL + SELECT

CONTROL also modifies the channel SELECT buttons.

DrivenByMoss documents:

```
CONTROL + SELECT
   → Open / Close Group folder
```

This is distinct from hierarchical Group navigation.

Hierarchical navigation uses:

```
SELECT again
   → enter Group
```

whereas:

```
CONTROL + SELECT
   → open / close Group folder
```

changes the Group's expanded state.

ALT

ALT is another modifier used for specialised parameter operations.

A clear example is the Jog Wheel:

```
ALT + Jog Wheel
   → Change Loop Length
```

and:

```
ALT + SHIFT + Jog Wheel
   → Fine Loop Length adjustment
```

So the Jog Wheel now has several modifier layers:

```
Jog Wheel
   → Position

SHIFT + Jog Wheel
   → Fine Position

OPTION + Jog Wheel
   → Tempo

OPTION + SHIFT + Jog Wheel
   → Fine Tempo

CONTROL + Jog Wheel
   → Loop Start

CONTROL + SHIFT + Jog Wheel
   → Fine Loop Start

ALT + Jog Wheel
   → Loop Length

ALT + SHIFT + Jog Wheel
   → Fine Loop Length
```

This is one of the clearest examples of the modifier system.

One Control, Several Parameters

Without modifiers:

```
Jog Wheel
   → Position
```

With modifiers:

```
OPTION
   → Tempo

CONTROL
   → Loop Start

ALT
   → Loop Length
```

and then:

```
SHIFT
   → finer movement
```

The Jog Wheel has not changed.

Its context has.

V-Pot Press Modifiers

The V-Pots provide another excellent modifier pattern.

DrivenByMoss documents these general press behaviours:

```
Press V-Pot
   → Reset parameter to default
```

```
SHIFT + Press V-Pot
   → Centre value
```

```
CONTROL + Press V-Pot
   → Minimum value
```

```
ALT + Press V-Pot
   → Maximum value
```

And when the V-Pot controls a Send:

```
OPTION + Press V-Pot
   → Toggle Send on / off
```

This gives the V-Pot press a compact family of related functions.

A Useful V-Pot Mental Model

Think:

```
PRESS
   → default
```

then:

```
SHIFT
   → centre
```

```
CONTROL
   → minimum
```

```
ALT
   → maximum
```

```
OPTION
   → context-specific alternate action
```

This pattern is especially useful because it gives physical access to values that might otherwise require precise mouse work.

OPTION + V-Pot Is Contextual

OPTION + V-Pot press is particularly important because its meaning depends on the current assignment.

For a Send:

```
OPTION + Press
   → toggle Send on / off
```

In other contexts, OPTION can provide another context-specific action.

So do not learn OPTION as:

OPTION always means X.

Instead:

OPTION asks for the alternative action appropriate to the current context.

SELECT Modifiers

The SELECT row is one of the most heavily modified parts of the X-Touch.

Verified DrivenByMoss uses include:

```
SHIFT + SELECT
   → Multi-select tracks
```

```
OPTION + SELECT
   → Stop playing clip on that track
```

```
CONTROL + SELECT
   → Open / Close Group folder
```

```
ALT + SELECT
   → Set New Clip Length
```

and:

```
SEND + SELECT
   → Select Send 1–8
```

We looked at these in Chapter 6.

The important point here is that the modifiers do not merely change a button's label.

They change the question the button answers.

The Same SELECT Button, Different Questions

Normal:

```
SELECT
   → Which track do I want?
```

OPTION:

```
OPTION + SELECT
```

```
   → Which track's playing clip
     do I want to stop?
```

CONTROL:

```
CONTROL + SELECT
   → Which Group do I want
     to open or close?
```

ALT:

```
ALT + SELECT
   → Which New Clip Length
     do I want?
```

SEND:

```
SEND + SELECT
   → Which Send do I want?
```

The physical row remains the same.

The context changes the meaning.

Modifiers Can Be Combined

Modifiers do not necessarily operate one at a time.

For example:

```
OPTION + SHIFT + PLAY
   → Toggle Punch Out
```

while:

```
OPTION + PLAY
   → Toggle Punch In
```

Likewise:

```
OPTION + SHIFT + Jog Wheel
```

combines OPTION's Tempo context with SHIFT's fine-adjustment behaviour.

So the pattern can be:

```
Modifier 1
   → choose alternate function

Modifier 2
   → refine that function
```

This is one reason the X-Touch can expose so much functionality without becoming physically enormous.

Order of Pressing

When this guide writes:

```
OPTION + MARKER
```

it means:

1. hold OPTION;
2. press MARKER;
3. release the buttons.

Likewise:

```
CONTROL + SHIFT + Jog Wheel
```

means:

1. hold CONTROL;
2. hold SHIFT;
3. turn the Jog Wheel;
4. release the modifiers.

The `+` symbol means:

Use these controls together.

It does not mean you should perform them as a rapid sequential shortcut.

Modifier Buttons and Modes Work Together

Modifiers do not replace modes.

They operate inside them.

For example, Device Mode already changes what the V-Pots mean.

Then CONTROL can temporarily change that Device Mode context again:

```
DEVICE Mode
     │
     ▼
V-Pots = Device Parameters
```

then:

```
Hold CONTROL
     │
     ▼
V-Pots = Device Choices
```

Similarly:

```
Hold OPTION
     │
     ▼
V-Pots = Parameter Page Choices
```

So the complete meaning of a control can depend on:

```
Physical Control
      +
Current Mode
      +
```

```
Current Focus
      +
Held Modifier
      =
Current Function
```

This is the core architecture of the X-Touch + DrivenByMoss workflow.

Modifiers Often Change Scale

Another useful pattern is that modifiers can change the scale of an operation.

For example:

```
ARM
   → this track
```

while:

```
SHIFT + ARM
   → record-arm across the active bank
```

Likewise:

```
CHANNEL
   → move the view
```

while:

```
OPTION + CHANNEL
   → move the track itself
```

The modifier can therefore make an operation broader, deeper or more consequential.

Some Modified Commands Are Consequential

Not every modifier command is harmless.

For example:

```
OPTION + CHANNEL
   → move selected track
```

and:

```
OPTION + BANK
   → move selected device
```

actually change the Bitwig project structure.

Likewise:

```
OPTION + RECORD
```

creates a clip and enables overdub.

So one good habit is:

Know whether a modified command changes the view, changes a value, or changes the project itself.

That distinction helps prevent surprises.

Modifiers Are Easier to Learn in Context

It would be possible to create an enormous table containing:

```
SHIFT + every control
```

```
OPTION + every control
```

```
CONTROL + every control
```

```
ALT + every control
```

That would be complete.

It would also be difficult to learn.

A better approach is:

Learn each modified command with the job it performs.

So:

```
OPTION + MARKER
```

belongs with markers.

```
OPTION + RECORD
```

belongs with recording.

```
CONTROL + V-Pot
```

belongs with device selection.

```
ALT + Jog Wheel
```

belongs with loop manipulation.

This reduces the feeling that the controller contains hundreds of unrelated shortcuts.

Modifier Patterns Are Tendencies, Not Rules

There are some useful patterns:

```
SHIFT
   → fine / secondary / reverse
```

```
OPTION
   → alternate operation
```

```
CONTROL
   → structural / direct-selection operation
```

```
ALT
```

```
   → specialised parameter or maximum-type operation
```

But these are not universal rules.

For example:

```
OPTION + MARKER
   → create marker
```

cannot be derived mechanically from a universal definition of OPTION.

You simply learn it as part of the Marker workflow.

Patterns reduce memory load.

They do not replace actual mappings.

The Displays Are Part of the Modifier System

Because modifiers can change what controls mean, feedback becomes especially important.

Suppose a V-Pot normally controls:

```
Pan
```

but after changing mode and holding a modifier it now represents:

```
Device 4
```

The hardware did not physically move.

Without feedback, that would be confusing.

So when using modifiers:

Look at what the controller tells you.

A useful cycle is:

```
Hold Modifier
      ↓
Assignments change
      ↓
Displays / LEDs update
      ↓
Confirm context
      ↓
Act
```

The X-Touch is meant to be read as well as touched.

SHIFT and Launcher-Oriented Work

DrivenByMoss contains a preference called:

```
Flip arranger and clip record / automation
```

This can reverse the normal-versus-SHIFT relationship for certain Arranger and Clip recording/automation functions.

That means commands such as:

```
RECORD
```

and:

```
SHIFT + RECORD
```

may behave differently for users who have deliberately enabled that preference.

This is an important reminder:

A modifier mapping can be affected by configuration.

Project XTC describes the normal mapping unless otherwise stated.

Chapter 21 discusses the configuration option itself.

Don't Memorise Everything at Once

This chapter contains a lot of examples.

You do not need to learn all of them now.

Start with a small core:

```
SHIFT + control
   → related / fine function
```

```
OPTION + control
   → alternative function
```

Then add practical combinations as you encounter them.

For example:

```
SHIFT + UNDO
   → Redo
```

```
OPTION + MARKER
   → Create marker
```

```
OPTION + REWIND / FORWARD
   → Marker navigation
```

```
OPTION + DEVICE
   → Pin device
```

The repeated use will turn them into muscle memory.

A Practical Jog Wheel Exercise

The Jog Wheel is an excellent way to experience the modifier system.

Start with:

```
Jog Wheel
   → change position
```

Then try:

```
SHIFT + Jog Wheel
   → fine position
```

Then:

```
OPTION + Jog Wheel
   → tempo
```

Then:

```
CONTROL + Jog Wheel
   → loop start
```

Then:

```
ALT + Jog Wheel
   → loop length
```

Finally add SHIFT to those modifier combinations for finer adjustment.

The point of the exercise is not merely to manipulate the transport.

It is to feel one physical control becoming several different controls through context.

A Practical V-Pot Exercise

Choose a parameter controlled by a V-Pot.

Try:

```
Press
```

to reset it to default.

Then:

```
SHIFT + Press
```

for centre.

Then:

```
CONTROL + Press
```

for minimum.

Then:

```
ALT + Press
```

for maximum.

If the current V-Pot is controlling a Send, try:

```
OPTION + Press
```

to toggle the Send.

Again, the aim is to feel the pattern physically rather than memorise a table.

A Useful Mental Model

Earlier we had:

```
Physical Control
      │
      ▼
Current Function
```

Then modes added:

```
Physical Control
      +
Current Mode
      =
Current Function
```

Now modifiers extend the model:

```
Physical Control
      +
Current Focus
      +
Current Mode
      +
Modifier, if held
      =
Current Function
```

This explains how one V-Pot, button or wheel can participate in so many workflows without the surface becoming physically enormous.

The Important Idea

Modifiers do not create a second controller hidden underneath the first.

They temporarily reshape the controller you already know.

The four main modifiers are:

```
SHIFT
OPTION
CONTROL
ALT
```

Some useful recurring patterns are:

```
SHIFT
   → finer / related / reverse

OPTION
   → alternate operation

CONTROL
   → structural or direct-selection operation
```

```
ALT
   → specialised parameter operation
```

But always remember:

The exact meaning depends on context.

The most useful approach is not to memorise every possible combination.

Learn the modifier vocabulary.

Then learn individual combinations as part of real tasks.

That way:

```
OPTION + MARKER
```

is not an arbitrary shortcut.

It belongs to the Marker workflow.

```
OPTION + RECORD
```

belongs to recording.

```
CONTROL + Jog Wheel
```

belongs to loop editing.

```
ALT + Press V-Pot
```

belongs to parameter control.

The job gives the shortcut meaning.

Coming Next

Now that we understand the modifier vocabulary, we can apply it to one of the most important physical controls on the X-Touch:

the V-Pots.

Their rotary movement, push action, LED rings and modifier behaviours make them far more capable than a simple row of knobs.

Next:

V-Pots (Rotary Encoders).

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 9 - V-Pots (Rotary Encoders)

V-Pots (Rotary Encoders)

Across the top of the eight X-Touch channel strips is a row of rotary encoders.

These are usually called:

V-Pots

They may look like ordinary knobs.

They are not.

Each V-Pot combines three important things:

```
Turn
   +
Press
   +
LED Ring
```

DrivenByMoss makes extensive use of all three.

More importantly, the parameter controlled by a V-Pot changes according to the current edit mode.

So a better mental model is not:

```
V-Pot = Pan Knob
```

but:

```
V-Pot
   │
   ▼
Current Parameter
```

The V-Pots are eight general-purpose parameter controls whose meaning changes with context.

Three Parts of a V-Pot

Each V-Pot gives us:

```
        Turn
         ↕
      ┌─────┐
      │     │
      │  ●  │  ← Press
```

```
      │     │
      └─────┘
       ╭───╮
     LED Ring
```

The three parts have different jobs.

Turning changes a value.

Pressing performs an action associated with that value.

The LED ring provides feedback.

This combination makes a V-Pot considerably more capable than a conventional rotary knob.

Why Encoders Rather Than Ordinary Knobs?

An ordinary potentiometer has a fixed physical position.

For example:

```
minimum       centre       maximum
   |             |            |
   ▼             ▼            ▼
   7 o'clock    12 o'clock    5 o'clock
```

That works well when the knob always controls the same parameter.

But the X-Touch constantly changes context.

A single V-Pot might control:

```
Pan
```

then:

```
Send Level
```

then:

```
Device Parameter
```

then:

```
Track Volume
```

An endless rotary encoder has no fixed physical position.

So it can inherit whatever value belongs to the parameter it currently represents.

That is exactly what a contextual controller needs.

The LED Ring Shows the Current Value

Because the V-Pot itself has no fixed pointer position, the LED ring provides visual feedback.

Conceptually:

```
V-Pot
  │
  ├── Turn
  │     ↓
  │   Change value
  │
  └── LED ring
        ↓
      Show value
```

When the assignment changes, the ring can immediately represent the new parameter.

You do not need to physically reposition the knob.

The Meaning Comes from the Current Mode

The most important thing to understand about the V-Pots is:

Their function is contextual.

For example, in Panorama Mode:

```
V-Pot 1
   → Pan Track 1

V-Pot 2
   → Pan Track 2

...

V-Pot 8
   → Pan Track 8
```

In Send Mode:

```
V-Pot 1
   → Send level Track 1

V-Pot 2
   → Send level Track 2

...

V-Pot 8
   → Send level Track 8
```

In Device Mode:

```
V-Pot 1
   → Device Parameter 1

V-Pot 2
   → Device Parameter 2

...
V-Pot 8
```

```
   → Device Parameter 8
```

The hardware remains unchanged.

Its assignment changes.

Turn to Change the Value

The most obvious V-Pot gesture is:

```
Turn V-Pot
      │
      ▼
Change Current Parameter
```

Exactly which parameter changes depends on the current mode.

For example:

```
Pan Mode
   → Panorama

Send Mode
   → Send Level

Device Mode
   → Device Parameter
```

The scribble strip and LED ring help tell you what the V-Pot currently represents.

SHIFT for Fine Adjustment

DrivenByMoss supports fine adjustment of V-Pot-controlled parameters with SHIFT.

So:

```
Turn V-Pot
      │
      ▼
Normal Adjustment
```

while:

```
SHIFT + Turn V-Pot
          │
          ▼
Fine Adjustment
```

This is especially useful when a small parameter change matters.

For example:

```
Send Level
   → broad adjustment
```

then:

```
SHIFT + V-Pot
```

```
   → precise adjustment
```

Rather than trying to make an extremely small movement with the encoder, hold SHIFT and make the same physical gesture.

Press to Reset

The V-Pots are also push switches.

DrivenByMoss gives the normal press a particularly useful function:

```
Press V-Pot
      │
      ▼
Reset Parameter
to Default Value
```

This applies to the current parameter.

So if a Pan value has been changed:

```
Pan = 37% Right
```

pressing the V-Pot can restore its default value.

Likewise, a changed device parameter can be returned to its default.

This makes experimentation much less intimidating.

You can change something and quickly return it to its normal value.

Press Is Not the Same as Turn

It is useful to treat these as two distinct gestures:

```
TURN
   → edit

PRESS
   → reset
```

That distinction becomes even more useful once modifiers are added.

SHIFT + Press — Centre

DrivenByMoss documents:

```
SHIFT + Press V-Pot
          │
          ▼
Set Parameter to Centre
```

This differs subtly from resetting to default.

The parameter's default value and its centre value are not necessarily the same thing.

So:

```
Press
   → default
```

while:

```
SHIFT + Press
   → centre
```

For a bipolar parameter such as Panorama, centre is immediately meaningful.

For other parameter types, the usefulness depends on the parameter.

CONTROL + Press — Minimum

CONTROL provides another direct-value gesture:

```
CONTROL + Press V-Pot
            │
            ▼
Set Parameter to Minimum
```

Instead of turning the encoder all the way down:

```
turn
turn
turn
turn
turn
...
```

you can jump directly to the minimum.

Conceptually:

```
Current Value
     │
     │ CONTROL + Press
     ▼
   Minimum
```

ALT + Press — Maximum

ALT provides the opposite operation:

```
ALT + Press V-Pot
         │
         ▼
Set Parameter to Maximum
```

So the four press gestures form a useful family:

```
Press
   → Default

SHIFT + Press
   → Centre

CONTROL + Press
   → Minimum
ALT + Press
```

```
   → Maximum
```

This is one of the cleanest modifier patterns on the X-Touch.

Four Useful Destinations

Think of the parameter range as:

```
MINIMUM -------- CENTRE -------- MAXIMUM
                    |
                 DEFAULT?
```

Depending on the parameter, its default may be at the centre.

But it does not have to be.

The V-Pot press gestures therefore provide four conceptually different destinations:

```
Default
Centre
Minimum
Maximum
```

without requiring precise turning.

OPTION + Press in Send Mode

OPTION has a special role when the V-Pot controls a Send level.

DrivenByMoss documents:

```
OPTION + Press V-Pot
          |
          ▼
Toggle Send On / Off
```

This is particularly useful because Send level and Send state are related but distinct things.

Turning controls:

```
How much?
```

while OPTION + Press controls:

```
On or off?
```

So one V-Pot gives direct access to both aspects of the Send.

A Send V-Pot as a Complete Control

In Send Mode:

```
Turn
   → Send Level

SHIFT + Turn
   → Fine Send Level
```

```
Press
   → Reset Send Level

OPTION + Press
   → Toggle Send On / Off
```

This is an excellent example of how much functionality can be packed into a single physical encoder.

For dub-style mixing, this is particularly useful.

You can treat the V-Pot not simply as a level control but as a compact performance control for the Send itself.

Track Edit Mode

Track Edit Mode gives the eight V-Pots a different arrangement.

DrivenByMoss assigns them to properties of the selected track.

The available controls are:

```
Volume
Panorama
Crossfader
Send 1
Send 2
Send 3
Send 4
Send 5
```

So rather than:

```
8 V-Pots
   → same parameter
     across 8 tracks
```

Track Edit Mode gives us:

```
8 V-Pots
   → different parameters
     for 1 selected track
```

This is a fundamental change in perspective.

Across Tracks Versus Across Parameters

Compare Panorama Mode:

```
          Track
V-Pot     Target

  1         1
  2         2
  3         3
  4         4
  5         5
  6         6
```

```
  7         7
  8         8
```

All eight knobs perform the same kind of operation.

Now compare Track Edit Mode:

```
V-Pot     Parameter

  1       Volume
  2       Panorama
  3       Crossfader
  4       Send 1
  5       Send 2
  6       Send 3
  7       Send 4
  8       Send 5
```

All eight knobs refer to the same selected track, but control different things.

This gives us two important V-Pot layouts:

```
one parameter
across many tracks
```

and:

```
many parameters
on one track
```

Recognising which layout you are currently using is crucial.

Track Mode Can Expose Six Sends

DrivenByMoss provides a preference that can hide the Crossfader parameter from Track Edit Mode.

With that option enabled, the freed V-Pot is used for another Send.

So instead of:

```
Volume
Pan
Crossfader
Send 1
Send 2
Send 3
Send 4
Send 5
```

you can have:

```
Volume
Pan
Send 1
Send 2
Send 3
Send 4
Send 5
Send 6
```

This can be particularly attractive in a Send-heavy workflow.

We will return to configuration choices in Chapter 21.

Volume Edit Mode

Press TRACK twice to enter Volume Edit Mode.

Here the relationship is simple:

```
V-Pot 1
   → Volume Track 1

V-Pot 2
   → Volume Track 2

...

V-Pot 8
   → Volume Track 8
```

This may initially seem redundant because the X-Touch already has motor faders.

But it demonstrates an important principle:

The V-Pots are general-purpose controls, not permanently assigned Pan knobs.

And depending on how the surface is configured or how you are working, alternative physical access to a parameter can be useful.

Panorama Edit Mode

Press PAN to enter Panorama Edit Mode.

Now:

```
V-Pot 1
   → Panorama Track 1

V-Pot 2
   → Panorama Track 2

...

V-Pot 8
   → Panorama Track 8
```

This is probably the role most people instinctively associate with the rotary controls on a mixing surface.

But on the X-Touch it is only one of many assignments.

Send Edit Mode

Press SEND to enter Send Edit Mode.

The V-Pots now control a Send across the current bank of tracks.

For example, with Send 1 selected:

```
Track       1     2     3     4     5     6     7     8
            │     │     │     │     │     │     │     │
            ▼     ▼     ▼     ▼     ▼     ▼     ▼     ▼
V-Pot       1     2     3     4     5     6     7     8

            Send 1 levels
```

This is an extremely useful mixer-oriented arrangement.

Rather than visiting each track and finding its Send individually, the entire Send appears across the physical surface.

Think Across the Mix

Suppose Send 1 feeds a delay.

In Send Mode:

```
Kick    Snare   Hats    Bass    Keys    Lead    Vox     FX
 │        │       │       │       │       │       │       │
 ▼        ▼       ▼       ▼       ▼       ▼       ▼       ▼
Delay Send Amount
```

Now you can shape:

How much of the entire mix is feeding this delay?

using one row of knobs.

That is a very different way of thinking from:

What are all the parameters on this track?

Both are useful.

The mode determines which viewpoint the hardware gives you.

Selecting Sends

Repeated presses of SEND step forward through Sends 1–8.

Conceptually:

```
SEND
  ↓
Send 1
  ↓
SEND
  ↓
Send 2
  ↓
SEND
  ↓
Send 3
```

and so on.

SHIFT + SEND moves backwards.

You can also use:

```
SEND + SELECT 1–8
```

to choose a Send directly.

Once the desired Send is selected, the V-Pots control its level across the tracks.

Device Mode

Device Mode is where the general-purpose nature of the V-Pots becomes particularly powerful.

Press DEVICE.

The eight V-Pots now control the currently selected eight device parameters.

Conceptually:

```
Selected Device

Parameter 1   Parameter 2   Parameter 3   Parameter 4
     │             │             │             │
     ▼             ▼             ▼             ▼
  V-Pot 1       V-Pot 2       V-Pot 3       V-Pot 4


Parameter 5   Parameter 6   Parameter 7   Parameter 8
     │             │             │             │
     ▼             ▼             ▼             ▼
  V-Pot 5       V-Pot 6       V-Pot 7       V-Pot 8
```

The X-Touch has effectively become a hardware parameter editor.

Parameter Pages

A device may expose far more than eight parameters.

The X-Touch only has eight V-Pots.

So DrivenByMoss organises device parameters into pages.

For example:

```
Page 1
Parameters 1–8

Page 2
Parameters 9–16

Page 3
Parameters 17–24
```

In Device Mode:

```
CHANNEL <
   → Previous Parameter Page

CHANNEL >
   → Next Parameter Page
```

Each page repopulates the eight V-Pots with another group of parameters.

Again, the eight knobs form a movable window.

Direct Parameter-Page Selection

DrivenByMoss provides a faster way to choose parameter pages.

Hold OPTION in Device Mode.

The V-Pots show the available parameter pages.

Then:

```
Press V-Pot
      │
      ▼
Select that Parameter Page
```

Conceptually:

```
Hold OPTION

Page 1   Page 2   Page 3   Page 4   ...
  │        │        │        │
  ▼        ▼        ▼        ▼
Pot 1    Pot 2    Pot 3    Pot 4
```

Press the V-Pot corresponding to the page you want.

This avoids stepping through pages one at a time.

Direct Device Selection

CONTROL provides a corresponding shortcut for devices.

Hold CONTROL in Device Mode.

DrivenByMoss shows the devices on the selected track across the V-Pots.

Then:

```
Press V-Pot
      │
      ▼
Select that Device
```

For example:

```
Selected Track

EQ      Comp     Saturator    Chorus     Delay     Reverb
│        │           │           │          │         │
```

```
▼       ▼          ▼          ▼         ▼         ▼
1       2          3          4         5         6
```

If you want the Delay:

```
Hold CONTROL
      ↓
Press V-Pot 5
      ↓
Delay Selected
```

This can be much faster than repeatedly pressing BANK to move through devices.

Device Navigation Has Two Speeds

So Device Mode gives us two navigation methods.

Sequential:

```
BANK <
BANK >
   → Previous / Next Device
```

Direct:

```
Hold CONTROL
      ↓
Press Device V-Pot
```

Likewise for parameter pages:

Sequential:

```
CHANNEL <
CHANNEL >
   → Previous / Next Page
```

Direct:

```
Hold OPTION
      ↓
Press Page V-Pot
```

This gives the workflow both precision and speed.

EQ Mode

EQ Mode works like Device Mode, but targets the track's equalizer device.

DrivenByMoss specifically supports Bitwig's EQ+ in this mode.

An especially interesting behaviour is:

If EQ Mode is activated on a track that does not yet contain an equalizer device, DrivenByMoss automatically adds one.

The V-Pots then control the EQ device parameters in the same general manner as Device Mode.

So the workflow can become:

```
Select Track
     ↓
EQ
     ↓
EQ+ available
     ↓
V-Pots edit EQ
```

This makes the hardware feel less like a remote control for an existing screen layout and more like an active part of the mixing workflow.

Instrument Mode

The INSTRUMENT assignment selects the instrument device edit mode.

Like the other device-oriented contexts, the V-Pots become parameter controls for the relevant device.

The underlying mental model remains:

```
Select context
      ↓
Eight parameters appear
      ↓
Turn V-Pots
      ↓
Edit
```

The particular parameters depend on the device and its mappings.

Project and Track Parameters

Pressing DEVICE again enters the Project/Track Parameter edit mode.

DrivenByMoss labels these contexts:

```
PP
```

or:

```
tP
```

depending on the active parameter context.

The eight device knobs control the currently selected eight parameters.

Again:

```
8 physical V-Pots
       │
       ▼
8 current parameters
```

The mechanism is the same even though the target has changed.

Layers and Drum Pads

The V-Pots also participate in Layer and Drum Pad editing.

Once you enter Layers or Drum Pads using the SELECT behaviour described earlier, the mode buttons can select different Layer edit modes.

The V-Pots can then edit:

```
Volume
Pan
Sends
```

for the Layers or Drum Pads.

The channel-strip MUTE and SOLO buttons also follow the Layer or Drum Pad context.

So a row that previously represented:

```
eight tracks
```

can now represent:

```
eight layers
```

or:

```
eight drum pads
```

while the V-Pots retain the same broad role:

Edit the current parameter across the eight current targets.

Master Mode

Touching the Master fader selects the Master track and enters Master Edit Mode.

The V-Pots acquire a very different set of functions.

DrivenByMoss documents:

```
V-Pot 1
   → Master Volume

V-Pot 2
   → Master Panorama
```

Pressing either resets its parameter.

The remaining V-Pots include project-level actions.

DrivenByMoss documents presses on V-Pots 3–5 for toggling the project's audio engine, while:

```
Press V-Pot 7
   → Previous Project
```

and:

```
Press V-Pot 8
   → Next Project
```

This is another reminder that a V-Pot press is not universally a parameter reset.

Its meaning can be overridden by the current mode.

Context Comes Before the Gesture

Earlier we learned:

```
Press V-Pot
   → reset parameter
```

That is the common behaviour for parameter controls.

But Master Mode shows why we must always remember the complete model:

```
Current Mode
      +
Current Assignment
      +
Gesture
      =
Function
```

For V-Pot 8 in Master Mode:

```
Master Mode
      +
V-Pot 8
      +
Press
      =
Next Project
```

The context wins.

Browser Mode

The V-Pots take on another role in Browser Mode.

Here they are used for navigating Browser columns.

DrivenByMoss documents:

```
Turn V-Pot
   → Navigate Browser column
```

and:

```
Press V-Pot
   → Enter filter or result
```

Pressing again confirms the selection at that level.

So the same gesture that normally edits a numeric parameter can now navigate a list.

From Continuous Values to Choices

This is an important conceptual shift.

Normally:

```
Turn V-Pot
      │
      ▼
Continuous Value
```

For example:

```
0% ─────────────── 100%
```

But in Browser Mode:

```
Turn V-Pot
      │
      ▼
Discrete Choices
```

For example:

```
Bass
Keys
Lead
Pad
Pluck
Strings
```

The encoder is equally comfortable doing both because it has no fixed physical endpoint.

Marker Mode

V-Pots also participate in Marker Mode.

Press MARKER to enter Marker Mode.

DrivenByMoss documents:

```
Press V-Pot
      │
      ▼
Start Playback
from Marker Position
```

So the V-Pot row becomes a set of marker destinations.

Conceptually:

```
Intro   Verse   Chorus   Break   Drop   Outro
  │       │       │        │       │      │
  ▼       ▼       ▼        ▼       ▼      ▼
Pot 1   Pot 2   Pot 3   Pot 4   Pot 5  Pot 6
```

Pressing a V-Pot chooses the corresponding marker position for playback.

This is another excellent example of the V-Pots acting as:

eight contextual choices

rather than simply eight knobs.

The V-Pots Have Two Broad Personalities

After seeing these modes, we can identify two broad V-Pot roles.

Parameter Controls

```
Turn
   → change value

Press
   → parameter action
```

Examples:

```
Pan
Volume
Send Level
Device Parameter
EQ Parameter
```

Choice Controls

```
Turn
   → navigate choices

Press
   → choose
```

or simply:

```
Press
   → select item
```

Examples:

```
Browser columns
Devices
Parameter Pages
Markers
Projects
```

This is a useful distinction.

Eight Knobs Can Mean Eight Tracks

In modes such as Panorama or Send:

```
V-Pot 1  → Track 1
V-Pot 2  → Track 2
```

```
V-Pot 3  → Track 3
...
V-Pot 8  → Track 8
```

The parameter is common.

The targets differ.

Eight Knobs Can Mean Eight Parameters

In Device Mode:

```
V-Pot 1  → Parameter 1
V-Pot 2  → Parameter 2
V-Pot 3  → Parameter 3
...
V-Pot 8  → Parameter 8
```

The target device is common.

The parameters differ.

Eight Knobs Can Mean Eight Choices

While holding CONTROL in Device Mode:

```
V-Pot 1  → Device 1
V-Pot 2  → Device 2
V-Pot 3  → Device 3
...
V-Pot 8  → Device 8
```

While holding OPTION:

```
V-Pot 1  → Page 1
V-Pot 2  → Page 2
V-Pot 3  → Page 3
...
V-Pot 8  → Page 8
```

The same physical row therefore supports three different mental layouts:

```
one parameter across eight targets
```

```
eight parameters on one target
```

```
eight choices
```

That is one of the keys to understanding the X-Touch.

Read Before You Turn

Because the V-Pots are so contextual, it is especially important not to assume what a knob currently controls.

Before turning one:

```
Look
  ↓
```

```
Identify Context
  ↓
Confirm Assignment
  ↓
Turn
```

The scribble strips, mode indicators and LED rings are all part of the interaction.

A useful habit is:

Read before you turn.

This becomes increasingly important as you use more advanced modes.

Why This Matters in a Mouse-Lite Workflow

With a mouse, you often work like this:

```
Find parameter visually
      ↓
Move pointer
      ↓
Click
      ↓
Drag
```

With the X-Touch:

```
Choose context
      ↓
Parameter appears on V-Pot
      ↓
Turn
```

The second approach becomes fast when you stop thinking primarily in terms of screen coordinates.

Instead of:

Where is the control on the screen?

you think:

Which controller context exposes the control?

That is a fundamental shift towards a hardware-centred workflow.

V-Pots and Dub-Style Mixing

Send Mode provides a particularly good example.

Suppose:

```
Send 1 → Delay
```

```
Send 2 → Reverb
```

Select Send 1.

Now the eight V-Pots represent:

```
Delay amount
across eight tracks
```

Instead of opening eight mixer channels and adjusting their Sends individually, you have eight physical controls side by side.

You can think:

```
Kick delay
Snare delay
Hat delay
Bass delay
Keys delay
Lead delay
Vocal delay
FX delay
```

as one physical performance surface.

Switch to Send 2 and the same knobs become the Reverb sends.

That is a much more musical way to approach effects than thinking of each Send as a tiny control buried inside an individual mixer strip.

A Practical Pan Exercise

Start in Panorama Mode.

Use the eight V-Pots to change the Pan positions of several tracks.

Then try:

```
SHIFT + Turn
```

and notice the finer adjustment.

Now deliberately change one Pan value and press its V-Pot:

```
Press
   → Default
```

Then try:

```
SHIFT + Press
   → Centre
```

Try:

```
CONTROL + Press
   → Minimum
```

and:

```
ALT + Press
   → Maximum
```

Watch Bitwig while doing this.

The aim is to make the press-modifier family physically memorable.

A Practical Send Exercise

Create or use a project with a Delay effect track.

Enter Send Mode and select the corresponding Send.

The V-Pots should now represent the Send amount across the current bank.

Try:

```
Turn
```

to change Send level.

Then:

```
SHIFT + Turn
```

for finer adjustment.

Finally try:

```
OPTION + Press
```

to toggle the Send on or off.

This is worth practising because it turns a fairly abstract modifier combination into an obvious musical action.

A Practical Device Exercise

Select a track containing several devices.

Press DEVICE.

Turn the V-Pots and watch the corresponding parameters in Bitwig.

Then use:

```
CHANNEL >
```

to move to another parameter page.

Next hold:

```
OPTION
```

and observe how the V-Pot assignments change.

Press one of the V-Pots to select a parameter page directly.

Then hold:

```
CONTROL
```

and observe the device choices.

Press the V-Pot corresponding to another device.

The sequence demonstrates three different uses of exactly the same hardware:

```
Normal Device Mode
   → edit parameters

OPTION held
   → choose parameter page

CONTROL held
   → choose device
```

Do Not Fight the Context

If a V-Pot appears to be controlling the wrong thing, resist the temptation to keep turning it until the result makes sense.

Instead ask:

```
Which mode am I in?

Which track is selected?

Which device is selected?

Which parameter page is active?

Is a modifier being held?
```

Usually the V-Pot is doing exactly what its current context tells it to do.

The challenge is understanding that context.

A Useful Mental Model

The complete V-Pot model is now:

```
              Current Mode
                   │
                   ▼
Current Target → V-Pot ← Modifier
                   │
                   ▼
              Current Action
```

Or more simply:

```
V-Pot
  +
Context
  +
Gesture
  =
Function
```

The gesture may be:

```
Turn
Press
SHIFT + Turn
SHIFT + Press
CONTROL + Press
ALT + Press
OPTION + Press
```

The context decides what that gesture acts upon.

The Important Idea

A V-Pot is not simply a knob.

It is a:

```
Rotary Encoder
      +
Push Button
      +
LED Display
```

and DrivenByMoss uses it as a general-purpose contextual control.

Its common parameter gestures are:

```
Turn
   → Change value

SHIFT + Turn
   → Fine adjustment

Press
   → Reset to default

SHIFT + Press
   → Centre

CONTROL + Press
   → Minimum

ALT + Press
   → Maximum
```

For Sends:

```
OPTION + Press
   → Toggle Send on / off
```

But the V-Pot row can also become:

```
8 track controls
```

or:

```
8 device parameters
```

or:

```
8 device choices
```

or:

```
8 parameter-page choices
```

or:

```
Browser navigation
```

or:

```
Marker destinations
```

The important question is therefore never merely:

What does this knob do?

It is:

What does this knob do in the context I am in now?

Once that becomes instinctive, the V-Pots stop feeling like eight mysterious multifunction controls.

They become one of the most flexible parts of the X-Touch.

Coming Next

The V-Pots provide flexible contextual control.

The motor faders provide something different:

a physical control that can move itself to reflect Bitwig's current state.

That creates a particularly strong connection between software and hardware.

Next:

Motor Faders.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 10 - Motor Faders

Motor Faders

The eight channel faders are probably the most immediately impressive part of the X-Touch.

They are:

- touch-sensitive;
- motorised;
- capable of receiving position information from Bitwig;
- able to represent different tracks as the controller changes bank.

The Master fader provides the same physical style of control for the master channel.

But the motorisation is not merely a visual effect.

It solves one of the fundamental problems of a controller whose physical controls can represent many different things.

A Fader Has Two Jobs

A normal MIDI fader sends information in one direction:

```
Move Fader
    │
    ▼
Software Changes
```

A motor fader adds the return journey:

```
Software Changes
      │
      ▼
Fader Moves
```

So the complete relationship becomes:

```
Physical Fader
      ⇅
Bitwig Parameter
```

The fader is both:

```
Input
```

and:

```
Feedback
```

This is one of the defining features of the X-Touch.

Why Motorisation Matters

Suppose Track 1 is at:

```
-3 dB
```

and Track 9 is at:

```
-14 dB
```

The first physical fader may represent Track 1.

Press:

```
BANK >
```

and that same physical fader may now represent Track 9.

Without motorisation, its physical position would still show Track 1's old value.

With motorisation:

```
BANK >
   |
   ▼
Track Assignment Changes
   |
   ▼
Bitwig Sends New Value
   |
   ▼
Fader Moves to -14 dB
```

The hardware immediately reflects the new context.

The Faders Follow the Track Bank

In normal mixer operation:

```
Fader 1  → Track 1
Fader 2  → Track 2
Fader 3  → Track 3
...
Fader 8  → Track 8
```

Change bank:

```
BANK >
```

and the mapping becomes:

```
Fader 1  → Track 9
Fader 2  → Track 10
Fader 3  → Track 11
```

```
...
Fader 8  → Track 16
```

The faders move automatically to the values of those newly represented tracks.

This is why eight physical faders can control a much larger project without losing positional feedback.

CHANNEL Movement Also Updates the Faders

CHANNEL movement shifts the eight-track window by one track.

For example:

```
Tracks 1–8
```

then:

```
CHANNEL >
```

gives:

```
Tracks 2–9
```

Each physical fader may therefore acquire a new track assignment.

Again, the motors move to reflect the new values.

The sequence is:

```
Change Controller Window
        ↓
Assignments Change
        ↓
Faders Reposition
```

This should become so normal that you barely notice it happening.

Do Not Fight the Motors

When changing banks or modes, the faders may move by themselves.

That is expected.

Do not hold them in place while they are repositioning.

Let the surface update.

Think:

```
Motor Movement
   → Feedback
```

not:

```
Motor Movement
   → Something has gone wrong
```

The movement is the controller telling you:

These are the current values for the things I now represent.

Touch Sensitivity

The faders know when you touch them.

This is different from merely detecting movement.

Conceptually:

```
Finger Touches Fader
        ↓
Touch State
        ↓
DrivenByMoss / Bitwig
```

That touch information can be useful for:

- track-selection behaviour;
- automation;
- temporary controller-mode behaviour.

Exactly what happens on fader touch can depend on the DrivenByMoss preferences.

We will examine those options in Chapter 21.

Fader Touch Can Select the Track

DrivenByMoss can be configured so that touching a fader selects its corresponding track.

With that preference enabled:

```
Touch Fader
     ↓
Track Selected
```

This can make mixing very fluid.

Instead of:

```
SELECT
   ↓
Move Fader
```

you may simply:

```
Touch Fader
     ↓
Adjust
```

But this behaviour is configurable.

Do not assume that touching a fader must always change the selected track.

Why Touch-to-Select Can Be Useful

Suppose you hear that the Vocal is too loud.

Your hand goes naturally to the Vocal fader.

With touch selection enabled:

```
Touch Vocal Fader
        ↓
Vocal Selected
        ↓
Move Fader
```

The physical action of reaching for the channel also establishes controller focus.

That can be very efficient.

Why Touch-to-Select Can Be Unwanted

Suppose the Synth track is selected because you are editing one of its devices.

At the same time, you want to reduce the Vocal level.

If touching the Vocal fader automatically selects Vocal:

```
Synth Device Context
        ↓
Touch Vocal Fader
        ↓
Selection Changes
```

that may interrupt the workflow.

So DrivenByMoss makes this behaviour optional.

The important lesson is:

Fader touch is information. What DrivenByMoss does with that information can be configured.

The Master Fader

The Master fader is separate from the eight channel-strip faders.

In normal operation it provides direct access to the master level.

Conceptually:

```
Channel Faders
   → Track Levels

Master Fader
   → Master Level
```

Unlike the eight channel faders, the Master fader does not normally move through the track bank.

It remains a stable physical destination for the overall output level.

That consistency makes it especially useful.

SHIFT + Master Fader — Metronome Volume

DrivenByMoss also gives the Master fader a modified function.

Hold:

```
SHIFT
```

while moving the Master fader.

The fader controls:

```
Metronome Volume
```

So:

```
Master Fader
   → Master Volume
```

while:

```
SHIFT + Master Fader
   → Metronome Volume
```

This is a useful example of a modifier temporarily changing the meaning of an otherwise stable physical control.

Why Metronome Volume Belongs on a Fader

Metronome level is the sort of value you may want to change quickly while recording.

You might think:

```
I need the click louder.
```

or:

```
The click is distracting me.
```

Instead of opening a software control:

```
SHIFT
   +
Master Fader
```

provides an immediate physical adjustment.

Release SHIFT and the Master fader returns to its normal role.

Faders and Automation

Touch sensitivity becomes especially important during automation.

Imagine writing a volume movement.

```
Touch Fader
     ↓
Move Fader
     ↓
Automation Written
     ↓
Release Fader
```

The system knows not merely that the value changed, but also when your hand took control and when it released control.

This allows automation modes such as Touch and Latch to behave in musically useful ways.

We will explore those modes properly in Chapter 16.

Motorised Automation Playback

Motorisation makes automation visible and physical.

Suppose a track contains volume automation.

During playback:

```
Automation Data
      ↓
Bitwig Volume
      ↓
X-Touch Motor
      ↓
Fader Moves
```

You can literally watch the mix being performed.

This is not merely entertaining.

It tells you:

```
where the automated value currently is
```

before you decide whether to touch the fader and intervene.

The Fader Is a Moving Value Display

This leads to an important idea.

A motor fader is effectively a physical meter for a parameter.

For example:

```
Low Position
   → Lower Value

High Position
   → Higher Value
```

When Bitwig changes the value, the display moves.

When you change the display, Bitwig changes the value.

So the distinction between:

```
control
```

and:

```
display
```

almost disappears.

FLIP

The X-Touch includes a:

```
FLIP
```

button.

FLIP exchanges or redirects assignments between the V-Pots and faders according to the current mode.

Conceptually:

```
Before FLIP

V-Pots
   → Current Rotary Assignment

Faders
   → Track Volume
```

then:

```
FLIP
```

and the current rotary assignment can move to the faders.

The exact result depends on the active controller context.

Why FLIP Is Powerful

Some parameters are naturally comfortable on a rotary encoder.

Others benefit from the long physical travel of a fader.

Suppose a V-Pot currently controls:

```
Send Level
```

FLIP can put that assignment onto the fader.

Instead of:

```
Turn Encoder
```

you can use:

```
Move Long-Throw Fader
```

This gives you:

- more physical travel;
- a visible position;
- touch sensitivity;
- motorised feedback.

FLIP Does Not Mean One Fixed Assignment

It is important not to memorise FLIP as:

Faders become Sends.

That may be what happens in one context.

In another context, the V-Pots may represent:

```
Pan
```

or:

```
Device Parameters
```

or another assignment.

The more useful rule is:

FLIP moves the current rotary-control relationship onto the faders.

So always ask:

```
What are the V-Pots controlling now?
```

before asking:

```
What will FLIP do?
```

FLIP in Send Mode

Suppose the V-Pots are controlling Send levels.

Normally:

```
V-Pots
   → Sends

Faders
   → Track Volumes
```

Press:

```
FLIP
```

and the Send assignments can be brought onto the faders.

Conceptually:

```
Send 1
Send 2
Send 3
...
```

become physically performable with the motor faders.

This can be especially useful for effects-heavy mixing.

FLIP in Device Mode

FLIP becomes even more interesting in Device Mode.

Suppose the eight V-Pots currently control:

```
Cutoff
Resonance
Drive
Mix
Attack
Release
Feedback
Tone
```

Press:

```
FLIP
```

and those assignments can be moved onto the faders.

A parameter such as:

```
Filter Cutoff
```

can now be performed with a long physical gesture.

For some parameters this feels much more expressive than turning an encoder.

FLIP Changes the Physical Character of a Parameter

This is worth emphasising.

FLIP does not merely move a parameter from one control to another.

It changes how that parameter feels.

Compare:

```
Encoder
   → compact rotary gesture
```

with:

```
Fader
   → long linear gesture
```

The software parameter may be identical.

The physical interaction is not.

That can matter when the controller is being used performatively.

SHIFT + FLIP — Normal Tracks and Effect Tracks

FLIP itself changes the relationship between the V-Pots and faders.

DrivenByMoss also gives the FLIP button a second function when used with SHIFT.

Press:

```
SHIFT + FLIP
```

to toggle the track bank between:

```
Instrument / Audio / Hybrid Tracks
```

and:

```
Effect Tracks
```

Conceptually:

```
Instrument / Audio / Hybrid Tracks
              │
              │ SHIFT + FLIP
              ▼
         Effect Tracks
              │
              │ SHIFT + FLIP
              ▼
Instrument / Audio / Hybrid Tracks
```

This is not the same operation as FLIP by itself.

FLIP and SHIFT + FLIP Do Different Jobs

The distinction is important.

Press:

```
FLIP
```

and you change which physical controls operate the current assignments.

For example:

```
V-Pot Assignment
       │
       ▼
     Fader
```

But press:

```
SHIFT + FLIP
```

and you change which kind of tracks appear in the track bank.

So:

```
FLIP
   → Change Control Assignment
```

while:

```
SHIFT + FLIP
   → Change Track-Bank Type
```

The two operations share a button, but they work at completely different levels.

Why Effect-Track Access Is Useful

Effect tracks often sit slightly outside the main flow of ordinary track navigation.

But when mixing, they can be extremely important.

A project might contain ordinary tracks such as:

```
Kick
Snare
Bass
Keys
Vocal
Guitar
```

and separate Effect tracks such as:

```
Room
Plate
Dub Delay
Long Reverb
```

Press:

```
SHIFT + FLIP
```

and the X-Touch can move its attention from the Instrument / Audio / Hybrid track bank to the Effect tracks.

That gives you direct physical access to the returns that shape the mix.

A Particularly Useful Mixing Workflow

Suppose the eight channel strips currently represent:

```
Kick   Snare   Hats   Bass   Keys   Gtr   Vox   Perc
```

You are happy with their levels, but now want to adjust the Effect tracks themselves.

Press:

```
SHIFT + FLIP
```

The track bank changes to the Effect tracks.

You can now work directly with controls such as:

```
Room
```

```
Plate
```

```
Delay
```

```
Reverb
```

When finished:

```
SHIFT + FLIP
```

returns to the Instrument / Audio / Hybrid track bank.

So the workflow becomes:

```
Mix Source Tracks
       ↓
SHIFT + FLIP
       ↓
Mix Effect Tracks
       ↓
SHIFT + FLIP
       ↓
Return to Source Tracks
```

This can be considerably faster than navigating through a large project to find the Effect tracks manually.

Don't Confuse This with Send Mode

There is an important distinction between:

```
SEND
```

and:

```
SHIFT + FLIP
```

SEND lets you control:

How much of a source track is being sent to an effect.

SHIFT + FLIP lets you reach:

The Effect track receiving those sends.

Conceptually:

```
Source Track
     │
     │ SEND
     ▼
Send Amount
     │
     ▼
Effect Track
```

```
     │
     │ SHIFT + FLIP
     ▼
Effect Track Controls
```

These are two sides of the same signal-flow relationship.

Source and Destination

Suppose a Vocal is feeding a Delay Effect track.

To change how much Vocal reaches the delay:

```
Select Vocal
     ↓
SEND
     ↓
Adjust Send Level
```

To change the level or other track-level properties of the Delay Effect track itself:

```
SHIFT + FLIP
      ↓
Find Delay Effect Track
      ↓
Adjust
```

Think:

```
SEND
   → source side

SHIFT + FLIP
   → destination side
```

That mental model makes the two workflows much easier to distinguish.

Returning from FLIP

Press FLIP again to return the controls to their normal relationship.

Conceptually:

```
Normal
   ↓
FLIP
   ↓
Flipped
   ↓
FLIP
   ↓
Normal
```

The faders then return to the values appropriate to their normal assignment.

Again, let the motors move.

Their movement is feedback.

FLIP and Motor Recall

Motorisation is particularly important when using FLIP.

Suppose a fader normally represents:

```
Track Volume = -8 dB
```

You press FLIP and it now represents:

```
Send Level = -20 dB
```

The motor moves.

Press FLIP again.

The fader returns to:

```
Track Volume = -8 dB
```

Without motorisation, the physical position would become meaningless every time the assignment changed.

With motorisation, the hardware follows the context.

Do Not Assume the Fader Still Means Volume

This is one of the most important safety habits around FLIP.

After pressing FLIP:

Do not assume that moving a fader changes track volume.

Read the display.

Check the active mode.

Then move the control.

The sequence should be:

```
Change Context
      ↓
Read Feedback
      ↓
Understand Assignment
      ↓
Move Fader
```

This is the same principle we introduced in Chapter 7.

Faders as Performance Controls

Motor faders become especially interesting when the X-Touch is treated as an instrument rather than merely a mixer.

Imagine several effect parameters assigned to the faders:

```
Delay Send
```

```
Reverb Send
```

```
Filter Cutoff
```

```
Feedback
```

```
Effect Mix
```

Now several parameters can be moved simultaneously with multiple fingers.

That is difficult to reproduce with a mouse.

The controller becomes a performance surface.

Multiple Faders at Once

One of the great advantages of physical controls is simultaneity.

A mouse generally manipulates one parameter at a time.

With faders you can:

```
Raise Track 1
Lower Track 2
Increase Track 3
```

simultaneously.

Or, after FLIP:

```
Increase Delay
Reduce Reverb
Open Filter
```

with several fingers.

This is one of the places where hardware control is not merely an alternative to the mouse.

It can enable a different style of interaction.

Motor Faders and Muscle Memory

Because the faders always occupy the same physical positions, your hands begin to learn the surface.

For example:

```
Channel 1
Channel 2
Channel 3
...
Channel 8
```

remain physically stable even though their track assignments change.

The scribble strips tell you what the channels represent.

The motors tell you their values.

Your hand learns where the controls are.

This combination of:

```
Fixed Physical Position
        +
Dynamic Assignment
        +
Motor Feedback
```

is one of the X-Touch's strongest design ideas.

A Practical Motor-Fader Exercise

Open a project containing more than eight tracks with noticeably different volume settings.

1. Observe the Faders

Do not touch anything.

Look at the eight physical positions.

2. Press BANK >

Watch all eight faders move to the values of the next bank.

3. Press BANK <

Watch the original positions return.

4. Move a Fader in Bitwig

Use the mouse to change one track's volume.

Watch the corresponding physical fader follow.

5. Move the Physical Fader

Confirm that Bitwig follows the hardware.

The purpose is to establish this relationship:

```
Hardware
   ⇄
Software
```

A Practical FLIP Exercise

Choose a mode in which the V-Pots have a clear assignment.

For example:

```
SEND
```

Observe the V-Pot assignments.

Now press:

```
FLIP
```

Watch the faders reposition.

Move one carefully and observe what changes in Bitwig.

Press:

```
FLIP
```

again.

Watch the faders return.

The aim is to make this principle instinctive:

```
FLIP
   → current rotary assignment
     moves to faders
```

A Practical SHIFT + FLIP Exercise

Open a project containing both ordinary tracks and Effect tracks.

Begin with the normal Instrument / Audio / Hybrid track bank visible.

Press:

```
SHIFT + FLIP
```

Observe the track names on the scribble strips.

The surface should now represent the Effect tracks.

Adjust an Effect-track level if appropriate.

Then press:

```
SHIFT + FLIP
```

again.

Observe the normal track bank return.

The aim is to distinguish clearly between:

```
FLIP
```

and:

```
SHIFT + FLIP
```

They share a physical button.

They do not perform variations of the same operation.

A Useful Mental Model

Think of the motor faders as:

```
Eight Physical Value Displays
          +
Eight Touch-Sensitive Controls
```

Their meaning comes from the current context.

Normally:

```
Faders
   → Track Volume
```

With another assignment flipped:

```
Faders
   → Current V-Pot Function
```

And independently:

```
SHIFT + FLIP
   → Normal Track Bank
     ↔ Effect Track Bank
```

So there are two distinct ideas:

```
FLIP
   → What do the faders control?
```

and:

```
SHIFT + FLIP
   → Which class of tracks
     does the bank contain?
```

The Important Idea

The motor faders are one of the clearest examples of the X-Touch's two-way relationship with Bitwig.

They do not merely send values.

They receive them.

```
You Move Fader
      ↓
Bitwig Changes
```

and:

```
Bitwig Changes
      ↓
Fader Moves
```

That feedback allows the same eight physical controls to represent many different tracks and parameters without losing their current values.

FLIP extends this flexibility:

```
FLIP
   → Move Current Rotary Assignment
     onto the Faders
```

while DrivenByMoss gives the same button a separate modified function:

```
SHIFT + FLIP
   → Toggle
     Instrument / Audio / Hybrid Tracks
     ↔ Effect Tracks
```

This gives us an especially useful signal-flow relationship:

```
SEND
   → Control how much signal
     goes to an Effect track

SHIFT + FLIP
   → Reach the Effect track itself
```

And throughout all of these context changes, the motors provide the same reassurance:

The physical position shows the current value of whatever the fader represents now.

Coming Next

The faders give us long-throw, touch-sensitive physical control.

The next part of the surface handles something different:

```
Play
```

```
Stop
```

```
Record
```

```
Navigate
```

```
Loop
```

```
Scrub
```

Next:

Transport Controls.

Chapter 11 - Transport Controls

Transport Controls

The Transport section of the X-Touch is one of the most immediately familiar parts of the surface.

It contains controls for:

`REWIND`

`FORWARD`

`STOP`

`PLAY`

`RECORD`

along with:

`Jog Wheel`

`Arrow Keys`

`ZOOM`

`SCRUB`

`NUDGE`

At first, these look like ordinary tape-machine controls.

DrivenByMoss makes them considerably more capable.

The basic transport remains simple.

But modifiers and repeated presses add access to:

- Repeat;
- Punch In;
- Punch Out;
- marker navigation;
- tempo;
- loop start;
- loop length;
- fine adjustment;

• editing modes;
• zoom;
• Tap Tempo.

So the Transport area is not merely:

Play, Stop and Record.

It is one of the X-Touch's principal navigation and timing surfaces.

PLAY

Press PLAY to start playback.

DrivenByMoss documents the button as:

```
PLAY
   → Start / Stop Playback
```

So PLAY behaves as a playback toggle.

If the project is stopped:

```
PLAY
  ↓
Playback Starts
```

If it is already playing:

```
PLAY
  ↓
Playback Stops
```

This makes PLAY useful even when your hand is already sitting over the transport area and you do not want to move to STOP.

Double-Press PLAY

DrivenByMoss also documents:

```
Double-press PLAY
   → Move Play Cursor to Start of Song
```

So PLAY contains two related operations:

```
Single Press
   → Playback

Double Press
   → Start of Song
```

This is useful when beginning another run-through from the very start.

SHIFT + PLAY — Repeat

Hold SHIFT and press PLAY:

```
SHIFT + PLAY
   → Toggle Repeat
```

This controls Bitwig's repeat state.

Conceptually:

```
Repeat Off
    │
    │ SHIFT + PLAY
    ▼
Repeat On
```

and the same command toggles it back off.

This is one of the simpler modifier combinations because PLAY and Repeat are both closely related to playback.

OPTION + PLAY — Punch In

DrivenByMoss documents:

```
OPTION + PLAY
   → Toggle Punch In
```

Punch In determines whether recording begins automatically when playback reaches the configured punch-in boundary.

Conceptually:

```
Playback
   │
   ▼
Punch-In Point
   │
   ▼
Recording Begins
```

OPTION + PLAY toggles that behaviour.

OPTION + SHIFT + PLAY — Punch Out

Add SHIFT:

```
OPTION + SHIFT + PLAY
   → Toggle Punch Out
```

Punch Out determines whether recording ends automatically at the configured punch-out boundary.

So the pair is:

```
OPTION + PLAY
   → Punch In

OPTION + SHIFT + PLAY
   → Punch Out
```

This is a useful example of modifiers building a family of related transport operations.

Punching as a Recording Workflow

Suppose only one phrase needs replacing.

Rather than manually pressing RECORD at exactly the right instant:

```
Set Punch In
      ↓
Set Punch Out
      ↓
Begin Playback
      ↓
Bitwig Records Only
Inside the Punch Region
```

The X-Touch therefore participates not only in starting recording but in defining how recording interacts with playback.

Chapter 19 deals with recording in more detail.

STOP

Press STOP:

```
STOP
  ↓
Stop Playback
```

That is the obvious behaviour.

But repeated STOP presses add useful navigation.

DrivenByMoss documents:

```
STOP
   → Stop Playback
```

then:

```
Press STOP again
   → Move Play Cursor to Start of Song
```

So after stopping, another press gives you a quick return to the beginning.

Double-Press STOP

There is another STOP gesture:

```
Double-press STOP
   → Move Play Cursor to End of Song
```

So STOP has three useful roles:

```
STOP
   → Stop
```

```
STOP again
   → Start of Song
```

```
Double STOP
   → End of Song
```

That gives surprisingly rich navigation from one familiar transport button.

REWIND and FORWARD

The ordinary functions are straightforward.

```
REWIND
   → Move Play Cursor Left
```

```
FORWARD
   → Move Play Cursor Right
```

These controls move through the Arranger timeline.

Think:

```
REWIND
   ← time

FORWARD
   time →
```

They provide coarse transport navigation without needing the mouse.

OPTION + REWIND / FORWARD — Marker Navigation

Markers give REWIND and FORWARD another role.

DrivenByMoss documents:

```
OPTION + REWIND
   → Move to closest marker
     before current position
```

and:

```
OPTION + FORWARD
   → Move to closest marker
     after current position
```

So the same physical controls provide two kinds of navigation.

Without OPTION:

```
REWIND / FORWARD
   → Move through Time
```

With OPTION:

```
OPTION + REWIND / FORWARD
   → Move through Structure
```

That distinction becomes especially useful in large arrangements.

Time Versus Structure

Imagine the timeline contains:

```
Intro      Verse      Chorus      Breakdown      Outro
  |          |           |           |              |
  ▼          ▼           ▼           ▼              ▼
──●──────────●───────────●───────────●──────────────●──
```

Ordinary REWIND and FORWARD ask:

How far through time should I move?

OPTION + REWIND / FORWARD ask:

Which musical landmark should I move to?

That is a very different way of navigating a song.

Chapter 15 explores Marker Mode in detail.

RECORD

Press RECORD:

```
RECORD
   → Start / Stop Recording
```

This is the main recording command.

But RECORD also has modifier functions:

```
SHIFT + RECORD
   → Toggle Launcher Overdub
```

and:

```
OPTION + RECORD
   → Create a new clip
     on selected track and slot,
     start playback,
     enable overdub
```

These are important commands, but Chapter 19 is their proper teaching home.

For now, remember that RECORD is another transport control whose modifiers change the recording context.

The Jog Wheel

The large Jog Wheel is one of the most flexible controls in the Transport section.

Normally:

```
Jog Wheel
   → Move Play Cursor
```

Turn left:

```
← earlier
```

Turn right:

```
later →
```

This is particularly useful for positioning the cursor without dragging the timeline with a mouse.

SHIFT + Jog Wheel — Fine Position

Hold SHIFT:

```
SHIFT + Jog Wheel
   → Fine Play-Cursor Adjustment
```

So:

```
Jog Wheel
   → normal movement
```

while:

```
SHIFT + Jog Wheel
   → finer movement
```

This follows the modifier pattern introduced in Chapter 8.

SHIFT often refines an existing continuous operation.

OPTION + Jog Wheel — Tempo

Hold OPTION:

```
OPTION + Jog Wheel
   → Change Tempo
```

Now the same wheel no longer moves through the song.

It changes the project's tempo.

Conceptually:

```
Jog Wheel
   → Time Position
```

becomes:

```
OPTION + Jog Wheel
   → Tempo
```

The physical gesture remains identical.

The modifier changes what is being controlled.

Fine Tempo Adjustment

Add SHIFT:

```
OPTION + SHIFT + Jog Wheel
   → Fine Tempo Adjustment
```

So:

```
OPTION
   → choose Tempo
```

and:

```
SHIFT
   → make the adjustment finer
```

This is a particularly clean example of two modifiers working together.

CONTROL + Jog Wheel — Loop Start

Hold CONTROL:

```
CONTROL + Jog Wheel
   → Change Loop Start
```

The wheel now moves the beginning of Bitwig's loop region.

Conceptually:

```
Loop
┌───────────────────────────┐
│                           │
└───────────────────────────┘
▲
│
Loop Start
```

CONTROL + Jog Wheel moves that left boundary.

Fine Loop-Start Adjustment

Add SHIFT:

```
CONTROL + SHIFT + Jog Wheel
   → Fine Loop-Start Adjustment
```

Again:

```
CONTROL
   → choose Loop Start

SHIFT
   → finer adjustment
```

ALT + Jog Wheel — Loop Length

Hold ALT:

```
ALT + Jog Wheel
   → Change Loop Length
```

Now the wheel changes the size of the loop.

Conceptually:

```
Short Loop

┌─────┐
│     │
└─────┘
```

versus:

```
Longer Loop

┌─────────────────────┐
│                     │
└─────────────────────┘
```

The Jog Wheel directly changes that length.

Fine Loop-Length Adjustment

Add SHIFT:

```
ALT + SHIFT + Jog Wheel
   → Fine Loop-Length Adjustment
```

So the full Jog Wheel map becomes:

```
Jog Wheel
   → Play Position

SHIFT + Jog Wheel
   → Fine Play Position

OPTION + Jog Wheel
   → Tempo

OPTION + SHIFT + Jog Wheel
   → Fine Tempo

CONTROL + Jog Wheel
   → Loop Start

CONTROL + SHIFT + Jog Wheel
   → Fine Loop Start

ALT + Jog Wheel
   → Loop Length

ALT + SHIFT + Jog Wheel
   → Fine Loop Length
```

That is a great deal of control from one wheel.

One Wheel, Four Dimensions

The Jog Wheel can therefore manipulate four distinct things:

```
Position
Tempo
```

```
Loop Start

Loop Length
```

The modifiers choose the dimension.

SHIFT chooses precision.

Conceptually:

```
             Jog Wheel
                 │
     ┌───────────┼───────────┐
     │           │           │
   OPTION      CONTROL      ALT
     │           │           │
    Tempo      Loop Start   Loop Length

No Modifier
     │
     ▼
   Position
```

This is one of the clearest demonstrations of DrivenByMoss's modifier design.

The Arrow Keys

The X-Touch includes four directional arrow buttons.

DrivenByMoss documents them as behaving like the arrow keys on the computer keyboard:

```
←   →   ↑   ↓
```

So they provide direct keyboard-style navigation from the surface.

Their exact effect depends on what currently has focus in Bitwig.

That is important.

The arrows do not necessarily represent one fixed DAW function.

They pass the familiar directional command into the current context.

Why Keyboard-Style Arrows Are Useful

The arrow keys can help with tasks where Bitwig already has keyboard-navigation behaviour.

Conceptually:

```
Current Bitwig Focus
        │
        ▼
Arrow Press
        │
        ▼
Move / Navigate
```

```
according to that focus
```

This is another example of the X-Touch complementing rather than replacing Bitwig's own interaction model.

ZOOM

Press ZOOM to change the meaning of the arrow keys.

When ZOOM is active:

```
← / →
   → Zoom Arranger Horizontally
```

The horizontal arrows control the timeline scale.

So:

```
←
   → one horizontal zoom direction

→
   → the opposite horizontal zoom direction
```

The precise visual result is shown immediately in Bitwig.

ZOOM and Track Height

DrivenByMoss also documents:

```
ZOOM active
   +
↑ / ↓
   → Toggle Track Height
```

Both vertical arrows toggle the track height.

So ZOOM changes the arrow-key context from ordinary navigation to display manipulation.

Conceptually:

```
Arrow Keys
    │
    ├── normal
    │      → keyboard-style navigation
    │
    └── ZOOM active
          ├── left/right
          │      → horizontal zoom
          │
          └── up/down
                 → track height
```

Why This Matters

Zooming is a good example of a task that often sends your hand back to the mouse.

DrivenByMoss gives the X-Touch a physical alternative.

If you need to see a broader section of the arrangement:

```
ZOOM
   ↓
Arrow
```

may be faster than finding and dragging a graphical zoom control.

As always, use whichever interface is more natural for the task.

SCRUB

The X-Touch also has a SCRUB button.

DrivenByMoss documents:

```
SCRUB
   → Toggle Between Editing Modes
```

This wording is important.

SCRUB is not documented here as simply enabling a traditional tape-style audio scrub mode.

Instead, DrivenByMoss uses the MCU SCRUB button to toggle Bitwig editing modes.

So once again:

Trust the DrivenByMoss mapping, not assumptions based only on the printed hardware label.

Hardware Labels Versus Bitwig Functions

The X-Touch inherits its labels from the Mackie Control design.

That means a button may be labelled:

```
SCRUB
```

because that is the MCU control being transmitted.

DrivenByMoss is free to map that control to the most useful Bitwig function.

The same principle appears elsewhere with controls such as:

```
TRIM
```

```
DROP
```

```
USER
```

So:

```
Printed Label
      ≠
```

```
Guaranteed Literal Bitwig Function
```

The actual DrivenByMoss mapping is what matters.

NUDGE — Tap Tempo

DrivenByMoss maps:

```
NUDGE
   → Tap Tempo
```

This means repeated presses can establish the project tempo by tapping the beat.

Conceptually:

```
Tap
Tap
Tap
Tap
```

gives Bitwig timing information from which it can derive tempo.

This can be particularly convenient when trying to match a project to something you are hearing or playing.

Tap Tempo as a Musical Gesture

Instead of thinking:

What BPM number should this be?

you can think:

This fast.

Then tap:

```
1      2      3      4
●      ●      ●      ●
```

NUDGE turns physical timing into tempo information.

That makes the control rather more musically intuitive than its printed name might suggest.

The REPEAT Button

The dedicated MCU REPEAT control also toggles Repeat.

So DrivenByMoss provides:

```
REPEAT
   → Toggle Repeat
```

and:

```
SHIFT + PLAY
```

```
→ Toggle Repeat
```

These are two physical routes to the same state.

This kind of redundancy is not necessarily wasteful.

One route may make more sense depending on where your hand currently is.

Multiple Routes Can Be Useful

Suppose your fingers are already around PLAY.

Then:

```
SHIFT + PLAY
```

may be natural.

If your hand is near REPEAT:

```
REPEAT
```

may be easier.

A control surface does not need to force every function through one unique route.

Sometimes a second route improves fluency.

Transport Is More Than Playback

At this point, the Transport section can manipulate:

```
Playback

Recording

Timeline Position

Markers

Repeat

Punch In

Punch Out

Tempo

Loop Start

Loop Length

Zoom

Track Height

Editing Mode
Tap Tempo
```

That is much broader than the word "Transport" initially suggests.

A Navigation Workflow

Suppose you want to move to a chorus marked later in the project.

You could:

```
OPTION + FORWARD
       ↓
Next Marker
```

If that is the Chorus marker:

```
PLAY
```

and playback starts from there.

If the cursor needs a small correction:

```
SHIFT + Jog Wheel
```

gives fine positioning.

The workflow becomes:

```
Structural Navigation
       ↓
Fine Position
       ↓
Playback
```

all from the Transport area.

A Loop-Editing Workflow

Suppose you want to work repeatedly on one section.

You can:

```
CONTROL + Jog Wheel
       ↓
Set Loop Start
```

then:

```
ALT + Jog Wheel
       ↓
Set Loop Length
```

then:

```
SHIFT + PLAY
       ↓
Enable Repeat
```

Now the section loops.

If either boundary needs fine adjustment:

```
SHIFT
```

can be added to the corresponding Jog Wheel modifier.

This gives the X-Touch a surprisingly complete loop-positioning workflow.

A Punch-Recording Workflow

Suppose a short section needs rerecording.

Configure the punch region in Bitwig.

Then:

```
OPTION + PLAY
   → Punch In
```

```
OPTION + SHIFT + PLAY
   → Punch Out
```

Start playback before the region.

Bitwig can then enter and leave recording according to those boundaries.

This can reduce the need to manually time the RECORD button.

A Tempo Workflow

Suppose you know roughly how fast the song should feel but do not know the BPM.

Start with:

```
NUDGE
   → Tap Tempo
```

Tap the pulse.

Then use:

```
OPTION + Jog Wheel
```

to refine the tempo.

If you need very small changes:

```
OPTION + SHIFT + Jog Wheel
```

gives finer control.

So:

```
Tap
   ↓
Approximate Tempo
   ↓
Jog
   ↓
Refine
```

is possible entirely from the X-Touch.

Transport and Mouse-Lite Working

Transport is one of the easiest places to reduce dependence on the mouse.

Compare:

```
Find play cursor
      ↓
Click timeline
      ↓
Find Play button
      ↓
Click
```

with:

```
Jog Wheel
    ↓
PLAY
```

Or:

```
Find marker visually
      ↓
Click marker
```

with:

```
OPTION + FORWARD
```

The hardware route often maps more directly to the intention.

The Transport Area Rewards Muscle Memory

Transport controls are particularly suitable for muscle memory because their physical positions do not change.

After enough use:

```
PLAY
STOP
RECORD
REWIND
FORWARD
```

become locations rather than commands you have to search for.

The same can gradually become true of:

```
SHIFT + PLAY
```

```
OPTION + PLAY
```

```
NUDGE
```

```
ZOOM
```

This is where a hardware transport can feel much faster than an on-screen equivalent.

Do Not Learn Every Modifier Immediately

There is a lot in this chapter.

Start with the core:

```
PLAY
STOP
REWIND
FORWARD
Jog Wheel
```

Then add:

```
SHIFT + PLAY
   → Repeat
```

Then perhaps:

```
OPTION + REWIND / FORWARD
   → Markers
```

Then:

```
OPTION / CONTROL / ALT + Jog Wheel
```

Once the basic Transport area is automatic, the advanced functions have somewhere sensible to attach in memory.

A Practical Transport Exercise

Open a project containing several minutes of material.

1. Use PLAY and STOP

Become comfortable starting and stopping without looking at the computer controls.

2. Press STOP again

Observe the play cursor return to the beginning.

3. Double-press STOP

Observe the play cursor move to the end.

4. Use REWIND and FORWARD

Move through the timeline.

5. Use the Jog Wheel

Position the cursor.

6. Add SHIFT

Make a finer movement.

The goal is to make the transport mechanics physically familiar.

A Practical Jog Wheel Exercise

Now try the four Jog Wheel contexts:

```
Normal
   → Position

OPTION
   → Tempo

CONTROL
   → Loop Start

ALT
   → Loop Length
```

For each one, add SHIFT and observe the finer adjustment.

This single exercise teaches one of the most reusable modifier patterns on the entire X-Touch.

A Practical Marker Exercise

Add several markers to the project.

Then use:

```
OPTION + REWIND
```

and:

```
OPTION + FORWARD
```

to navigate between them.

Compare the experience with manually moving the cursor.

The aim is to feel the difference between:

```
navigate by time
```

and:

```
navigate by structure
```

If Transport Behaviour Seems Unexpected

Check the current modifiers and modes.

For example:

```
Jog Wheel moving tempo?
```

Perhaps OPTION is held.

`Arrow keys zooming?`

Perhaps ZOOM is active.

`REWIND jumping to markers?`

Perhaps OPTION is held.

The hardware is contextual.

Unexpected behaviour often means:

`different context`

rather than:

`something is broken`

A Useful Mental Model

The Transport section can be understood in layers.

Layer 1 — Basic Transport

`PLAY`

`STOP`

`REWIND`

`FORWARD`

`RECORD`

Layer 2 — Navigation

`Jog Wheel`

`Markers`

`Arrow Keys`

Layer 3 — Timing

`Repeat`

`Tempo`

`Loop Start`

`Loop Length`

`Tap Tempo`

Layer 4 — Recording Support

```
Punch In

Punch Out

Overdub
```

Layer 5 — View / Edit Control

```
ZOOM

SCRUB
```

The hardware is easier to understand when these operations are grouped by purpose rather than memorised as one long button table.

The Important Idea

The X-Touch Transport section begins with familiar controls:

```
PLAY

STOP

REWIND

FORWARD

RECORD
```

But DrivenByMoss extends them considerably.

The verified mappings include:

```
PLAY
   → Start / Stop Playback

Double PLAY
   → Start of Song

SHIFT + PLAY
   → Toggle Repeat

OPTION + PLAY
   → Toggle Punch In

OPTION + SHIFT + PLAY
   → Toggle Punch Out
STOP
   → Stop

STOP again
   → Start of Song

Double STOP
   → End of Song
```

```
REWIND / FORWARD
   → Move through timeline

OPTION + REWIND / FORWARD
   → Previous / Next Marker
```

The Jog Wheel provides:

```
Normal
   → Position

OPTION
   → Tempo

CONTROL
   → Loop Start

ALT
   → Loop Length

SHIFT
   → Fine adjustment
```

And the remaining controls add:

```
NUDGE
   → Tap Tempo

SCRUB
   → Toggle Editing Modes

ZOOM + ← / →
   → Horizontal Arranger Zoom

ZOOM + ↑ / ↓
   → Toggle Track Height

Arrow Keys
   → Keyboard-style directional navigation
```

So perhaps the most useful way to think about this area is:

Transport controls move not only through playback, but through time, structure and timing.

Once these controls become familiar, a surprising amount of basic navigation can happen without the mouse.

Coming Next

Transport lets us move around the project and control playback.

The next chapter moves from:

```
Where are we?
```

to:

```
What device are we controlling?
```

DrivenByMoss gives the X-Touch a particularly rich Device Mode, including device navigation, parameter pages, direct selection and pinning.

Next:

Device Mode.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 12 - Device Mode

Device Mode

A Bitwig track can contain far more than a volume control and a Pan knob.

It may contain:

- instruments;
- audio effects;
- note effects;
- modulators;
- utility devices;
- whole chains of devices.

The X-Touch gives us eight V-Pots.

So how can eight physical knobs control a device that may expose dozens of parameters?

DrivenByMoss solves this with:

```
Device Selection
      +
Parameter Pages
      +
Direct Selection
```

This is the heart of Device Mode.

Entering Device Mode

Press:

```
DEVICE
```

The X-Touch enters Device Mode for the currently selected track.

The eight V-Pots now control the current page of parameters on the selected device.

Conceptually:

```
Selected Track
      │
      ▼
Selected Device
      │
```

```
      ▼
Current Parameter Page
      │
      ▼
V-Pots 1–8
```

The channel strip has not changed.

The controller's focus has moved deeper into the track.

Device Mode Needs a Selected Track

Device Mode acts on the currently selected track.

So a common workflow is:

```
SELECT Track
     ↓
DEVICE
     ↓
Edit Device
```

This is why SELECT is so important.

It establishes the track context.

DEVICE then moves the X-Touch into the device context for that track.

The First Eight Parameters

Suppose the selected device exposes:

```
Parameter 1
Parameter 2
Parameter 3
Parameter 4
Parameter 5
Parameter 6
Parameter 7
Parameter 8
```

The V-Pots map directly:

```
V-Pot 1  → Parameter 1
V-Pot 2  → Parameter 2
V-Pot 3  → Parameter 3
V-Pot 4  → Parameter 4
V-Pot 5  → Parameter 5
V-Pot 6  → Parameter 6
V-Pot 7  → Parameter 7
V-Pot 8  → Parameter 8
```

Turn a V-Pot:

```
Turn
   ↓
Change Parameter
```

Press it:

```
Press
   ↓
Reset Parameter
```

The modifier behaviours from Chapter 9 also apply where appropriate.

More Than Eight Parameters

Many devices expose far more than eight parameters.

DrivenByMoss therefore groups them into pages.

For example:

```
Page 1
   Parameters 1–8

Page 2
   Parameters 9–16

Page 3
   Parameters 17–24

Page 4
   Parameters 25–32
```

The X-Touch only needs eight V-Pots because those eight controls can be repopulated with another page whenever necessary.

CHANNEL Navigates Parameter Pages

In Device Mode, the CHANNEL buttons change meaning.

Normally:

```
CHANNEL <
CHANNEL >
   → move through tracks
```

In Device Mode:

```
CHANNEL <
   → Previous Parameter Page

CHANNEL >
   → Next Parameter Page
```

This is one of the most important context changes in the X-Touch.

The same physical buttons now navigate inside the selected device.

BANK Navigates Devices

The BANK buttons also change meaning in Device Mode.

Normally:

```
BANK <
```

```
BANK >
   → move track bank by 8
```

In Device Mode:

```
BANK <
   → Previous Device

BANK >
   → Next Device
```

So Device Mode gives us two navigation axes:

```
BANK
   → Devices

CHANNEL
   → Parameter Pages
```

That distinction is worth learning.

A Useful Device-Mode Map

Think:

```
Track
  │
  ▼
Device Chain
  │
  ├── Device 1
  ├── Device 2
  ├── Device 3
  └── Device 4
```

Use BANK to move between devices.

Then, inside the selected device:

```
Device
  │
  ├── Page 1
  ├── Page 2
  ├── Page 3
  └── Page 4
```

Use CHANNEL to move between parameter pages.

So:

```
BANK
   → horizontal movement through device chain

CHANNEL
   → horizontal movement through parameter pages
```

The physical layout stays the same.

The navigation target changes.

BANK Is the Larger Structural Step

There is a useful pattern here.

BANK handles the larger structural object:

```
whole device
```

CHANNEL handles the finer-grained object:

```
parameter page
```

This echoes the broader relationship introduced in Chapter 4:

BANK moves at the larger level; CHANNEL moves at the finer level.

The literal targets change, but the hierarchy of movement remains understandable.

Direct Device Selection with CONTROL

Stepping through devices with BANK is useful.

But if a track contains many devices, repeated BANK presses can become tedious.

DrivenByMoss provides a faster route.

Hold:

```
CONTROL
```

The V-Pots temporarily display the devices on the selected track.

Conceptually:

```
Hold CONTROL
      │
      ▼
Device 1   Device 2   Device 3   Device 4   ...
   │          │          │          │
   ▼          ▼          ▼          ▼
Pot 1      Pot 2      Pot 3      Pot 4
```

Press the V-Pot corresponding to the device you want.

Choosing a Device Directly

Suppose the selected track contains:

```
EQ
Compressor
Saturator
Chorus
Delay
Reverb
```

Hold CONTROL.

The V-Pots now represent:

```
1  EQ
2  Compressor
3  Saturator
4  Chorus
5  Delay
6  Reverb
```

Press V-Pot 5:

```
Hold CONTROL
      ↓
Press V-Pot 5
      ↓
Delay Selected
```

This is much faster than stepping through five devices one at a time.

Direct Parameter-Page Selection with OPTION

OPTION provides the corresponding shortcut for parameter pages.

Hold:

```
OPTION
```

DrivenByMoss temporarily displays the available parameter pages across the V-Pots.

Conceptually:

```
Hold OPTION
      │
      ▼
Page 1   Page 2   Page 3   Page 4   ...
  │        │        │        │
  ▼        ▼        ▼        ▼
Pot 1    Pot 2    Pot 3    Pot 4
```

Press the V-Pot corresponding to the page you want.

Two Ways to Navigate Devices

So we now have two device-navigation methods.

Sequential:

```
BANK <
BANK >
   → Previous / Next Device
```

Direct:

```
Hold CONTROL
      ↓
Press Device V-Pot
```

The first is ideal when moving one step.

The second is ideal when jumping directly to a known device.

Two Ways to Navigate Parameter Pages

Likewise, parameter pages can be navigated sequentially:

```
CHANNEL <
CHANNEL >
   → Previous / Next Page
```

or directly:

```
Hold OPTION
      ↓
Press Page V-Pot
```

So Device Mode supports both:

```
step through
```

and:

```
jump directly
```

That is a very useful design.

Direct Selection Reduces Button Repetition

Suppose a device has eight parameter pages and you want Page 7.

Sequentially:

```
CHANNEL >
CHANNEL >
CHANNEL >
CHANNEL >
CHANNEL >
CHANNEL >
```

Directly:

```
Hold OPTION
      ↓
Press V-Pot 7
```

Similarly, if the desired device is far along a device chain:

```
Hold CONTROL
      ↓
Press its V-Pot
```

The direct-selection modifiers turn the V-Pot row into a temporary menu.

The V-Pot Row Becomes Eight Choices

This is one of the recurring patterns we identified in Chapter 9.

Normally, the eight V-Pots are:

```
8 parameters
```

With CONTROL held:

```
8 device choices
```

With OPTION held:

```
8 page choices
```

So the same physical row alternates between:

```
editing
```

and:

```
selection
```

depending on context.

OPTION + DEVICE — Pin the Cursor Device

DrivenByMoss also provides:

```
OPTION + DEVICE
   → Pin Cursor Device
```

Pinning is useful when you want the controller to stay attached to the current device rather than following another device-selection change elsewhere in Bitwig.

Conceptually:

```
Current Device
      │
      │ OPTION + DEVICE
      ▼
Pinned Device
```

This can make Device Mode more predictable in workflows where Bitwig's cursor device would otherwise change automatically.

Why Pinning Matters

Suppose you are repeatedly adjusting the same delay device while selecting or manipulating other things in the project.

Without pinning:

```
Bitwig focus changes
      ↓
Cursor Device may change
      ↓
X-Touch follows
```

With the device pinned:

```
OPTION + DEVICE
      ↓
Device remains target
```

The controller can remain focused on the thing you actually care about.

Pinning Is a Focus Tool

It is useful to think of pinning as:

Hold this context steady.

Earlier we learned:

```
SELECT
   → establish focus
```

Pinning adds:

```
OPTION + DEVICE
   → keep this device focus
```

This is another way in which DrivenByMoss helps reduce repeated navigation.

Device Feedback

Because Device Mode is highly contextual, feedback is essential.

The X-Touch needs to tell you:

- which device is selected;
- which parameter page is active;
- what each V-Pot currently controls;
- what the current parameter values are.

So the complete interaction is:

```
Choose Device
     ↓
Choose Page
     ↓
Read Scribble Strips
     ↓
Turn V-Pot
     ↓
Read LED Ring / Bitwig
```

The controller is not merely receiving commands.

It is continually reporting the current assignment back to you.

Read Before You Turn

This principle is especially important in Device Mode.

A V-Pot may control:

```
Cutoff
```

then after a page change:

```
Resonance
```

then after a device change:

```
Delay Feedback
```

The physical knob did not move.

Its assignment did.

So:

Read before you turn.

That habit prevents a great deal of accidental parameter editing.

Parameter Names Can Be Abbreviated

The X-Touch scribble strips have limited space.

A Bitwig parameter name may therefore appear abbreviated.

For example:

```
Feedback
```

might appear as:

```
Feedbk
```

or another shortened form.

The display does not need to reproduce Bitwig's full graphical label.

It needs to give you enough information to identify the parameter confidently.

Familiarity makes these abbreviations much easier to read.

FLIP in Device Mode

FLIP becomes particularly interesting in Device Mode.

Normally:

```
V-Pots
   → Device Parameters

Faders
   → Track Volumes
```

Press FLIP:

```
Faders
   → Device Parameters
```

The exact exchanged assignment follows the current context.

This can turn a rotary parameter into a long-throw, touch-sensitive, motorised fader control.

Why Put Device Parameters on Faders?

Some parameters are easier to perform physically with a fader.

For example:

```
Filter Cutoff
```

```
Effect Mix
```

```
Feedback
```

```
Macro Amount
```

may benefit from a longer gesture.

A fader also provides:

- visible physical position;
- motorised recall;
- touch sensitivity.

So FLIP changes not merely the control location.

It changes the physical character of the interaction.

Device Automation

Once a device parameter is exposed on the X-Touch, it can participate in automation.

Conceptually:

```
Device Parameter
      │
      ▼
V-Pot or Fader
      │
      ▼
Automation
```

This means a filter sweep or effect movement can be performed physically rather than drawn first.

Chapter 16 covers the automation workflow itself.

The important point here is that Device Mode provides the parameter access required for that performance.

DEVICE Is a Mode, Not a One-Shot Command

When you press DEVICE, you are not merely selecting one device operation.

You are changing the controller's working context.

While Device Mode is active:

```
BANK
   → Devices

CHANNEL
   → Parameter Pages

V-Pots
   → Parameters

CONTROL
   → Direct Device Selection

OPTION
   → Direct Page Selection
```

So DEVICE creates an entire temporary control environment.

Leaving Device Mode

To leave Device Mode, select another edit mode.

For example:

```
PAN

SEND

TRACK

USER
```

or another appropriate mode button.

The V-Pots and navigation controls then acquire the assignments belonging to that new context.

This is another reason to think of modes as:

views onto the controller

rather than commands that merely happen once.

Device Mode and the Mouse-Lite Idea

Device Mode can substantially reduce mouse dependence for routine parameter work.

A graphical route might be:

```
Find Track
    ↓
Click Track
    ↓
Find Device
    ↓
Click Device
    ↓
Find Parameter
```

```
    ↓
Drag
```

A hardware route can become:

```
SELECT Track
    ↓
DEVICE
    ↓
Choose Device
    ↓
Choose Page
    ↓
Turn V-Pot
```

Once the controller workflow is familiar, this can become much faster.

When the Mouse Is Still Better

Device Mode does not replace the graphical device interface.

Some devices expose:

- complex visual editors;
- modulation graphs;
- sequencers;
- spectral displays;
- drag-and-drop structures;
- parameter relationships that are easier to understand visually.

For those tasks, use the screen.

The point of Device Mode is not:

Never look at the device GUI.

It is:

Routine parameter access should not always require the GUI.

That is the Mouse-Lite principle.

Device Chains as Physical Destinations

A useful way to think about a track's device chain is:

```
Track
  │
  ├── Device 1
  ├── Device 2
  ├── Device 3
  ├── Device 4
  └── Device 5
```

BANK navigates those destinations.

CONTROL lets you choose one directly.

Once there, CHANNEL navigates parameter pages.

OPTION lets you choose a page directly.

So the X-Touch gives the device chain both:

```
sequential navigation
```

and:

```
direct navigation
```

This is more efficient than thinking of Device Mode as a single linear sequence of pages.

A Practical Device Exercise

Create or open a track containing several devices.

For example:

```
EQ+
Compressor
Saturator
Delay
Reverb
```

Select the track and press:

```
DEVICE
```

1. Use BANK

Press:

```
BANK >
```

several times.

Watch the selected device change.

Then use:

```
BANK <
```

to move back.

2. Use CHANNEL

Select a device with several pages.

Press:

```
CHANNEL >
```

to move through parameter pages.

Then use:

```
CHANNEL <
```

to move back.

3. Use CONTROL

Hold CONTROL.

Observe the device names across the V-Pots.

Press the V-Pot for a device several positions away.

4. Use OPTION

Hold OPTION.

Observe the parameter-page choices.

Press one directly.

The purpose of the exercise is to make this relationship instinctive:

```
BANK
   → Devices

CHANNEL
   → Pages

CONTROL
   → Direct Device

OPTION
   → Direct Page
```

A Practical Pinning Exercise

Select a device you want to keep under X-Touch control.

Press:

```
OPTION + DEVICE
```

Now change focus elsewhere in Bitwig.

Observe whether the cursor device remains pinned according to the DrivenByMoss behaviour.

Then unpin when appropriate.

The purpose is to understand pinning as a way of keeping controller focus stable.

EQ Mode

The X-Touch also has an EQ button.

DrivenByMoss provides a specialised EQ edit mode built around Bitwig's EQ+ device.

This is conceptually related to Device Mode because it exposes device parameters on the V-Pots.

But EQ Mode gives a faster route to a very common mixing task.

Entering EQ Mode

Press:

```
EQ
```

DrivenByMoss targets the EQ+ device for the selected track.

If the selected track does not already contain an EQ+ in the required context, DrivenByMoss can insert one automatically.

Conceptually:

```
SELECT Track
     ↓
EQ
     ↓
EQ+ available
     ↓
V-Pots control EQ parameters
```

This is a powerful workflow shortcut.

EQ Mode Can Modify the Project

The automatic insertion behaviour deserves attention.

Entering EQ Mode is not always a purely navigational operation.

If DrivenByMoss adds an EQ+ device, the project has changed.

So:

```
EQ
```

can mean either:

```
Control existing EQ+
```

or:

```
Insert EQ+
and then control it
```

depending on the selected track.

That makes EQ Mode slightly more consequential than a simple view change.

Why Automatic EQ+ Insertion Is Useful

EQ is one of the most common mixing tasks.

Without the specialised mode:

```
SELECT Track
     ↓
Open Browser
     ↓
Find EQ+
     ↓
Insert
     ↓
DEVICE
     ↓
Select EQ+
```

With EQ Mode:

```
SELECT Track
     ↓
EQ
     ↓
Work
```

That is a significant reduction in interaction.

It is exactly the kind of workflow compression a control surface is good at.

EQ Mode Uses Familiar Principles

Once EQ Mode is active, the interaction remains familiar:

```
V-Pots
   → Parameters

CHANNEL
   → Parameter Pages

Feedback
   → Scribble Strips / LED Rings
```

So EQ Mode does not require learning an entirely new physical language.

It is a specialised application of the same device-control model.

Device Mode and EQ Mode Compared

Device Mode asks:

Which device on this track do I want to control?

EQ Mode asks:

Take me directly to the track's equalizer.

So:

```
DEVICE
   → general device navigation
```

while:

```
EQ
   → specialised EQ workflow
```

Both use the same underlying V-Pot and page-navigation ideas.

Don't Confuse Device Selection with Track Selection

When using Device Mode, there are now two levels of focus:

```
Selected Track
      │
      ▼
Selected Device
```

SELECT changes the track.

BANK or CONTROL changes the device.

This distinction matters.

If the wrong device appears, ask:

```
Is the correct track selected?
```

```
Is the correct device selected?
```

Those are separate questions.

A Complete Device Workflow

Suppose you want to adjust the feedback of a delay on the Vocal track.

A hardware-oriented workflow might be:

```
SELECT Vocal
      ↓
DEVICE
      ↓
Hold CONTROL
      ↓
Press Delay V-Pot
      ↓
Hold OPTION
      ↓
Choose Parameter Page
      ↓
Turn Feedback V-Pot
```

If the parameter is on the current page already, some of those steps disappear.

The workflow scales according to how directly you can reach the required control.

A Faster Workflow with Familiarity

At first, you may think:

```
Which device?

Which page?

Which knob?
```

Later:

```
Vocal
  ↓
DEVICE
  ↓
Delay
  ↓
Feedback
```

And eventually your hands may perform the sequence almost automatically.

This is the recurring Project XTC progression:

```
Remember
   ↓
Recognise
   ↓
Repeat
   ↓
Muscle Memory
```

Device Mode as a Moving Window

The eight V-Pots are another moving window.

Across devices:

```
BANK
   → choose which device
```

Within the device:

```
CHANNEL
   → choose which parameter page
```

Across the page:

```
V-Pots 1–8
   → choose which parameter
```

So the complete model is:

```
Track
  │
  ▼
Device
  │
  ▼
Parameter Page
  │
```

```
  ▼
Parameter
```

The X-Touch can navigate each level physically.

The Important Idea

Device Mode turns the X-Touch into a contextual device editor.

The verified normal mapping is:

```
DEVICE
   → Enter Device Mode
```

Inside Device Mode:

```
BANK <
BANK >
   → Previous / Next Device
```

```
CHANNEL <
CHANNEL >
   → Previous / Next Parameter Page
```

```
V-Pots
   → Current Device Parameters
```

For direct selection:

```
Hold CONTROL
   → Show Devices
   → Press V-Pot to Select Device
```

```
Hold OPTION
   → Show Parameter Pages
   → Press V-Pot to Select Page
```

And:

```
OPTION + DEVICE
   → Pin Cursor Device
```

EQ Mode provides a specialised route:

```
EQ
   → Control EQ+
   → Insert EQ+ automatically if required
```

So the most useful mental model is:

```
SELECT Track
     ↓
DEVICE
     ↓
Choose Device
     ↓
Choose Page
     ↓
Adjust Parameter
```

Once that sequence becomes familiar, the X-Touch can reach deeply into a Bitwig track without requiring every parameter change to begin with the mouse.

Coming Next

Device Mode gives us direct access to devices that already exist.

But sometimes the thing we want is not in the project yet.

DrivenByMoss also gives the X-Touch a hardware route into Bitwig's Browser:

- opening it;
- navigating categories and results;
- inserting devices before or after the current device;
- replacing a device;
- confirming or cancelling the selection.

Next:

Browser Mode.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 13 - Browser Mode

Browser Mode

Device Mode lets us control devices that are already present.

Browser Mode answers a different question:

What if the device, preset or item I want is not there yet?

Bitwig's Browser is where new material enters the project.

Normally, that means reaching for the mouse.

DrivenByMoss gives the X-Touch a hardware route into the Browser.

The surface can be used to:

- open the Browser;
- navigate Browser tabs and columns;
- scroll results;
- insert a device before the current device;
- insert a device after the current device;
- replace the current device;
- confirm the selection;
- cancel the operation.

So Browser Mode is not merely another edit mode.

It is the point where the X-Touch can help bring something new into the project.

Entering Browser Mode

DrivenByMoss maps the X-Touch's USER button to the Browser.

Press:

```
USER
   |
   ▼
Open Browser
```

This starts a normal Browser operation.

The X-Touch controls then acquire Browser-specific meanings.

USER Is Not Just "User"

The button is physically labelled:

```
USER
```

because that is part of the Mackie Control layout.

DrivenByMoss gives it a useful Bitwig-specific role:

```
USER
   → Browser
```

This is another example of a principle we have already encountered:

The printed MCU label does not necessarily describe the literal Bitwig function.

The DrivenByMoss mapping is what matters.

Three Ways to Start Browsing

DrivenByMoss documents three Browser-entry commands:

```
USER
   → Start Browser

SHIFT + USER
   → Insert new device before
     the current device

OPTION + USER
   → Insert new device after
     the current device
```

So the modifier determines the intended insertion point.

Normal Browser Entry

Pressing:

```
USER
```

opens the Browser in its normal context.

Think:

I want to browse.

Once inside, the Browser-specific controls determine what happens next.

SHIFT + USER — Insert Before

Hold SHIFT and press USER:

```
SHIFT + USER
        │
        ▼
Insert New Device
Before Current Device
```

Suppose a device chain is:

```
EQ
   ↓
Compressor
   ↓
Delay
```

and Compressor is the current device.

Using:

```
SHIFT + USER
```

starts the Browser with the intention of inserting something before Compressor.

Conceptually:

```
Before

EQ
   ↓
[Compressor]
   ↓
Delay
```

then:

```
SHIFT + USER
```

and after choosing a new device:

```
EQ
   ↓
New Device
   ↓
[Compressor]
   ↓
Delay
```

OPTION + USER — Insert After

OPTION gives the complementary operation:

```
OPTION + USER
       │
       ▼
Insert New Device
After Current Device
```

Using the same chain:

```
EQ
   ↓
[Compressor]
   ↓
Delay
```

OPTION + USER begins a Browser operation that inserts after the current device.

After choosing something:

```
EQ
   ↓
[Compressor]
   ↓
New Device
   ↓
Delay
```

So the pair is easy to understand:

```
SHIFT + USER
   → before

OPTION + USER
   → after
```

Why Insertion Position Matters

A device chain is ordered.

Changing the order can change the sound.

For example:

```
Distortion
   ↓
Delay
```

does not necessarily sound the same as:

```
Delay
   ↓
Distortion
```

So Browser insertion is not simply:

Add something.

It is:

Add something at the correct place in the chain.

DrivenByMoss lets the insertion intention be established before browsing begins.

The Browser Changes the Surface

Once Browser Mode is active, controls that previously manipulated tracks or parameters now navigate the Browser.

Conceptually:

```
Normal Context
```

```
        │
        ▼
Mixer / Device Controls
```

then:

```
USER
  ↓
Browser Context
        │
        ▼
Browser Navigation Controls
```

The hardware has not changed.

The job has.

Browser Tabs

Bitwig's Browser can contain several tabs or categories of browsing context.

DrivenByMoss maps the vertical arrow buttons to them:

```
↑
   → Previous Browser Tab

↓
   → Next Browser Tab
```

So the up/down arrows move between Browser tabs rather than behaving as ordinary keyboard arrows while this context is active.

Browser Tabs as a Higher Level

A useful way to think about Browser navigation is:

```
Browser
   │
   ├── Tab
   │     │
   │     └── Columns / Filters
   │             │
   │             └── Results
   │
```

The vertical arrows operate at the Tab level.

The V-Pots and Jog Wheel work further down inside the Browser.

V-Pots Navigate Browser Columns

The eight V-Pots become Browser controls.

DrivenByMoss uses them to navigate the Browser columns.

Conceptually:

```
Browser Column 1
        │
        ▼
```

```
V-Pot 1

Browser Column 2
      │
      ▼
V-Pot 2

Browser Column 3
      │
      ▼
V-Pot 3

...
```

Turning a V-Pot moves through the choices available in that column.

Turning Means Choosing from a List

Outside Browser Mode, a V-Pot often changes a continuous value:

```
0% ─────────────── 100%
```

In Browser Mode, it moves through discrete choices:

```
Bass
Keys
Lead
Pad
Percussion
FX
```

So:

```
Turn V-Pot
      │
      ▼
Move through Browser choices
```

The encoder works equally well for both because it has no fixed physical endpoint.

Pressing a V-Pot

Pressing a V-Pot enters or confirms the current filter or result at that Browser level.

Conceptually:

```
Turn
   → choose
```

then:

```
Press
   → enter / confirm
```

This makes the V-Pot a two-part Browser control:

```
Turn
```

```
   → navigate

Press
   → select
```

That is a very natural extension of the rotary-and-push design.

The Jog Wheel Scrolls Results

DrivenByMoss also maps the Jog Wheel to Browser result navigation.

In Browser Mode:

```
Jog Wheel
   → Scroll Browser Results
```

This is particularly useful once the broad filters have narrowed the Browser to a list of candidate items.

Think:

```
V-Pots
   → narrow the search

Jog Wheel
   → move through results
```

The controls divide the Browser into different navigation jobs.

Browser Filtering and Result Selection

Conceptually, the workflow can be:

```
Choose Browser Tab
       ↓
Use V-Pots to choose filters
       ↓
Use Jog Wheel to scroll results
       ↓
Choose Result
       ↓
ENTER
```

This allows a large part of the Browser interaction to remain on the X-Touch.

LEFT — Insert Before

Once Browser Mode is active, the left arrow has a device-insertion meaning.

DrivenByMoss documents:

```
←
   → Insert Before Current Device
```

This allows the insertion intention to be changed from within the Browser.

Conceptually:

```
Current Device
```

```
      |
      ▼
←
      |
      ▼
Insert Before
```

RIGHT — Insert After

The right arrow provides the complementary operation:

```
→
   → Insert After Current Device
```

So inside Browser Mode:

```
←
   → before
```

```
→
   → after
```

The spatial relationship makes this particularly easy to remember.

ZOOM — Replace the Current Device

DrivenByMoss assigns another Browser function to the ZOOM button:

```
ZOOM
   → Replace Current Device
```

This gives us a third insertion strategy.

We can:

```
Insert Before
```

```
Insert After
```

```
Replace
```

without leaving the Browser workflow.

Three Placement Choices

The Browser therefore offers three device-placement intentions:

```
←
   → Before
```

```
ZOOM
   → Replace
```

```
→
   → After
```

Conceptually:

```
        Current Device
Before      Replace      After
```

```
  |             |             |
  ▼             ▼             ▼
  ←           ZOOM            →
```

That is a remarkably compact physical model.

Why Replace Is Different

Insertion preserves the current device and adds another one.

Replacement removes the current device from that position and substitutes the chosen Browser result.

So:

```
Insert
   → add
```

while:

```
Replace
   → substitute
```

That is a consequential difference.

Use ZOOM deliberately when browsing.

ENTER — Confirm and Close

Once you have chosen the desired Browser result:

```
ENTER
   → Confirm Selection
     and Close Browser
```

This is one of the clearest uses of ENTER on the X-Touch.

The operation means exactly what its label suggests:

Yes — use this.

CANCEL — Discard

If you decide not to use the Browser selection:

```
CANCEL
   → Discard Browser Operation
```

Think:

No — leave things as they were.

So Browser Mode gives ENTER and CANCEL a very natural pair:

```
ENTER
   → accept

CANCEL
   → reject
```

ENTER and CANCEL Finally Have Their Obvious Jobs

Earlier chapters deliberately clarified that ENTER and CANCEL are not used for hierarchical Group navigation.

That workflow uses SELECT.

Browser Mode is where ENTER and CANCEL become particularly intuitive:

```
Browser Result
      │
      ├── ENTER
      │      ↓
      │     Keep It
      │
      └── CANCEL
             ↓
            Discard
```

This separation of roles is worth learning.

A Basic Browser Workflow

Suppose you want to add a delay after the currently selected device.

A hardware-oriented workflow might be:

```
OPTION + USER
       ↓
Browser Opens
       ↓
Choose Browser Tab
       ↓
Use V-Pots
       ↓
Narrow to Delay
       ↓
Use Jog Wheel
       ↓
Choose Result
       ↓
ENTER
       ↓
Delay Inserted After
Current Device
```

The mouse need not be involved.

A Replace Workflow

Suppose a track already contains one delay but you want to audition another.

Start Browser Mode.

Then choose:

```
ZOOM
```

```
   → Replace
```

Navigate the Browser.

Select another delay.

Press:

```
ENTER
```

The current device is replaced by the selected result.

If you change your mind before confirming:

```
CANCEL
```

abandons the operation.

Auditioning Possibilities

Browser Mode can be especially useful when you know broadly what you want but not the exact item.

For example:

I want a delay, but I am not sure which one.

The process becomes:

```
Open Browser
     ↓
Filter to Delay
     ↓
Scroll Results
     ↓
Try Candidate
     ↓
Choose
```

The controller handles the navigation while your attention stays on the musical result.

Browser Mode Is About Decisions

Device Mode is mostly about:

```
adjusting something that already exists
```

Browser Mode is about:

```
choosing something new
```

That difference is important.

In Device Mode:

```
Turn
   → change value
```

In Browser Mode:

```
Turn
   → move through possibilities
```

The X-Touch changes from an editor into a selector.

The V-Pots Become a Menu

A useful way to think about the Browser is that the V-Pot row becomes a physical menu system.

Normally:

```
V-Pot
   → parameter
```

In Browser Mode:

```
V-Pot
   → Browser choice
```

Turn:

```
Next / Previous Choice
```

Press:

```
Choose
```

This is another example of the V-Pots acting as general-purpose contextual controls.

Browse by Category, Not by Screen Position

A mouse-driven Browser workflow often feels spatial:

```
Where is the category?
```

```
Where is the result?
```

```
Where is the scroll bar?
```

The X-Touch workflow is more conceptual:

```
Which category?
```

```
Which filter?
```

```
Which result?
```

That can become faster once the Browser structure is familiar.

Browser Mode and the Scribble Strips

The scribble strips become especially important in Browser Mode because the V-Pots are no longer controlling familiar mixer parameters.

They need to tell you:

```
which Browser field
```

and:

```
which current choice
```

each V-Pot represents.

So:

Read before you turn.

This principle is at least as important in Browser Mode as in Device Mode.

Do Not Browse Blindly

If you forget which Browser column a V-Pot currently represents, do not simply turn controls until something useful happens.

Instead:

```
Look
  ↓
Read Labels
  ↓
Understand Current Filter
  ↓
Turn
```

Browser navigation is inherently about making choices.

Good feedback makes those choices deliberate.

Browser Mode Can Change the Project

Browser operations are consequential.

Depending on the chosen operation, you may:

```
Insert Device Before
```

```
Insert Device After
```

or:

```
Replace Current Device
```

These are project edits, not merely navigation.

So before confirming with ENTER, ask:

Is this the item I want, and is it going to the right place?

That is a useful Browser habit.

CANCEL Is Your Safety Net

Because Browser Mode can change the device chain, CANCEL is valuable.

If you become unsure:

```
CANCEL
```

Discard the operation.

Then begin again.

There is no prize for forcing an uncertain Browser choice through to completion.

The controller gives you an explicit way out.

Browser Mode and Device Chains

Suppose the current track contains:

```
EQ
   ↓
Compressor
   ↓
Delay
```

with Compressor selected.

The Browser can now support three different intentions.

Before

```
SHIFT + USER
```

or choose the left-arrow insertion behaviour.

Result:

```
EQ
   ↓
New Device
   ↓
Compressor
   ↓
Delay
```

After

```
OPTION + USER
```

or choose the right-arrow insertion behaviour.

Result:

```
EQ
   ↓
Compressor
```

```
   ↓
New Device
   ↓
Delay
```

Replace

```
ZOOM
```

Result:

```
EQ
   ↓
New Device
   ↓
Delay
```

This gives the Browser a clear structural relationship with the device chain.

Starting Before or After Can Save a Step

If you already know the desired insertion position, it is efficient to begin with:

```
SHIFT + USER
```

or:

```
OPTION + USER
```

rather than opening the Browser first and changing the insertion mode afterwards.

This follows a useful workflow principle:

Establish intention as early as possible.

The Browser then opens already aimed at the correct kind of operation.

The Arrow Keys Let You Change Your Mind

But perhaps you opened the Browser normally and then realise:

Actually, this should go before the compressor.

Use:

```
←
```

Or:

It should go after it.

Use:

```
→
```

The Browser workflow is flexible.

You do not necessarily need to cancel and start again merely because the insertion point changes.

Replace Is Particularly Useful for Comparison

Suppose you are choosing between several compressors.

The current track contains:

```
EQ
   ↓
Compressor A
   ↓
Saturator
```

Using Browser Replace, you can substitute:

```
Compressor B
```

at the same position.

This makes comparison more straightforward because the surrounding chain remains structurally similar.

The important idea is:

```
same place
   +
different device
```

rather than continually adding new devices to the chain.

Browser Mode and the Mouse-Lite Workflow

The Browser is one of the more ambitious parts of a Mouse-Lite workflow.

For a completely unfamiliar Browser search, the graphical interface may still be faster.

For a familiar task such as:

Add a delay after the current device

the X-Touch route can be very direct:

```
OPTION + USER
       ↓
Filter
       ↓
Choose
       ↓
ENTER
```

The point is not to prove that every Browser operation can be performed without the mouse.

The point is that routine Browser tasks no longer have to begin with it.

When the Mouse Is Better

Bitwig's graphical Browser may still be preferable when:

- exploring unfamiliar categories;
- reading long names;
- comparing lots of metadata;
- dragging items to precise locations;
- searching visually through a large set of results.

That is fine.

Project XTC's Mouse-Lite principle remains:

Use the interface best suited to the task.

Hardware browsing becomes valuable when it reduces friction, not when it merely proves that hardware browsing is possible.

Browser Mode as a Performance Tool

Browser Mode can also support experimentation.

Suppose playback is running and you decide the selected track needs another effect.

You can:

```
Open Browser
     ↓
Choose Effect
     ↓
Insert
     ↓
Return to Listening
```

The creative cycle remains:

```
Hear
  ↓
Choose
  ↓
Listen
```

rather than:

```
Hear
  ↓
Leave musical focus
  ↓
Navigate interface
  ↓
Return
```

This is where hardware Browser access can feel particularly useful.

A Practical Browser Exercise

Create a track containing at least one device.

Select that device.

1. Press USER

Observe the Browser open.

2. Use ↑ and ↓

Move between Browser tabs.

3. Turn the V-Pots

Observe the available filters or columns change.

4. Press a V-Pot

Enter or select the corresponding Browser choice.

5. Turn the Jog Wheel

Move through the result list.

6. Press CANCEL

Discard the operation.

Repeat the exercise until the Browser navigation itself feels comfortable.

A Practical Insertion Exercise

Now try three placement operations.

Start with a simple chain:

```
Device A
   ↓
Device B
```

Select Device B.

Insert Before

Use:

```
SHIFT + USER
```

choose a new device, then:

```
ENTER
```

Observe where it appears.

Undo if required.

Insert After

Use:

```
OPTION + USER
```

choose another device and confirm.

Replace

Open Browser Mode and use:

```
ZOOM
```

choose a replacement and confirm.

The aim is to make these three intentions distinct:

```
before
```

```
after
```

```
replace
```

A Useful Mental Model

Browser Mode can be summarised as:

```
                    Browser
                       │
           ┌───────────┼───────────┐
           │           │           │
         Before     Replace      After
           │           │           │
           ▼           ▼           ▼
      SHIFT+USER      ZOOM     OPTION+USER
```

Then inside the Browser:

```
↑ / ↓
   → Browser Tabs

V-Pots
   → Browser Columns / Choices

Jog Wheel
   → Results

←
   → Insert Before

→
   → Insert After

ZOOM
   → Replace
ENTER
```

```
   → Confirm

CANCEL
   → Discard
```

The Browser therefore becomes another temporary control environment.

The Important Idea

Browser Mode gives the X-Touch a hardware route for choosing and inserting new material.

The verified entry commands are:

```
USER
   → Start Browser

SHIFT + USER
   → Insert New Device Before
     Current Device

OPTION + USER
   → Insert New Device After
     Current Device
```

Inside the Browser:

```
↑ / ↓
   → Previous / Next Browser Tab

V-Pots
   → Navigate Browser Columns

Jog Wheel
   → Scroll Results

←
   → Insert Before

→
   → Insert After

ZOOM
   → Replace Current Device

ENTER
   → Confirm and Close

CANCEL
   → Discard
```

The central workflow is:

```
Choose Placement
      ↓
Browse
      ↓
Choose Item
      ↓
```

`Confirm`

Device Mode answers:

How do I control what is already here?

Browser Mode answers:

How do I bring in something new?

Together, the two modes make the X-Touch far more than a mixer.

They give it a route into the structure and contents of the Bitwig project itself.

Coming Next

We have now completed the main Part II chapters:

- V-Pots;
- Motor Faders;
- Transport;
- Device Mode;
- Browser Mode.

The next part goes deeper into the less obvious DrivenByMoss functionality.

We begin by reorganising the mixer around particular tasks rather than individual channel strips.

Next:

Mixer Edit Modes.

Chapter 14 - Mixer Edit Modes

Mixer Edit Modes

By now, we have become used to an important characteristic of the X-Touch:

The physical controls stay in the same place, but what they control can change.

The eight channel strips normally give us a familiar view of the mixer.

We have faders for track levels, V-Pots for parameters, and buttons for operations such as SELECT, MUTE, SOLO and ARM.

DrivenByMoss can take this idea further.

Instead of thinking only in terms of eight complete channel strips, we can temporarily ask the X-Touch to concentrate on one particular aspect of the mixer.

That is the idea behind the mixer edit modes.

Looking Across the Mixer

Imagine eight tracks:

```
Track 1   Track 2   Track 3   Track 4   Track 5   Track 6   Track 7   Track 8
```

There are several different ways we might want to look across them.

We could ask:

```
What are their volumes?
```

or:

```
What are their panorama positions?
```

or:

```
How much of each track is being sent to Send 1?
```

The tracks have not changed.

What changes is the dimension of the mixer that we are looking at.

Conceptually:

```
                 Tracks 1–8
                      │
       ┌──────────────┼──────────────┐
       │              │              │
       ▼              ▼              ▼
    Volume        Panorama         Sends
```

This is a useful way to understand the mixer edit modes.

Track Edit Mode

Press TRACK to enter Track Edit Mode.

Rather than treating each V-Pot simply as the normal control for its channel strip, Track Edit Mode exposes parameters associated with the selected track.

This gives the V-Pots another role:

```
Selected Track
      │
      ▼
Track Edit Mode
      │
      ▼
V-Pots → Track Parameters
```

The precise parameters available can depend on the current DrivenByMoss configuration.

The important point is the change in perspective.

We are no longer primarily looking across eight tracks.

We are looking more deeply at one selected track.

Volume Edit Mode

Press TRACK again and DrivenByMoss switches to Volume Edit Mode.

Now the V-Pots provide another way of controlling the volumes of the eight tracks in the current bank.

Conceptually:

```
V-Pot 1  → Track 1 Volume
V-Pot 2  → Track 2 Volume
V-Pot 3  → Track 3 Volume
...
V-Pot 8  → Track 8 Volume
```

Of course, the X-Touch already has eight motor faders.

So why would we want volume on the V-Pots as well?

One answer is FLIP, which we met earlier.

The ability to exchange the roles of faders and rotary controls becomes much more useful when the controller can present parameters in different ways.

The important lesson is not that one control is the "correct" way to adjust volume.

It is that DrivenByMoss allows the surface to be reorganised around the task.

Panorama Edit Mode

Press PAN to enter Panorama Edit Mode.

The eight V-Pots now control the panorama positions of the eight tracks in the current bank:

```
V-Pot 1  → Track 1 Pan
V-Pot 2  → Track 2 Pan
V-Pot 3  → Track 3 Pan
...
V-Pot 8  → Track 8 Pan
```

This is perhaps the clearest example of an edit mode.

Instead of thinking:

```
Channel 1
   ├── Volume
   ├── Pan
   ├── Mute
   └── Solo
```

we temporarily rotate our view through ninety degrees and think:

```
Pan
   ├── Track 1
   ├── Track 2
   ├── Track 3
   ├── ...
   └── Track 8
```

The information is the same.

The view onto it has changed.

Fine Adjustment with SHIFT

As we saw in Chapter 8, SHIFT frequently provides finer adjustment of continuous parameters.

That pattern applies here too.

Hold SHIFT while turning a V-Pot when you need more precise control.

Conceptually:

```
Turn V-Pot
```

```
     ↓
Normal adjustment

SHIFT + Turn V-Pot
     ↓
Finer adjustment
```

This is a good example of why we introduced modifiers before exploring the advanced modes.

The modifier itself is no longer something new to learn.

We simply apply an existing idea to a new task.

Send Edit Modes

The SEND control opens another particularly useful family of mixer views.

Press SEND and the V-Pots control a Send across the eight tracks.

For example, in the first Send mode:

```
             SEND 1

Track 1  ──────────────► Send 1
Track 2  ──────────────► Send 1
Track 3  ──────────────► Send 1
Track 4  ──────────────► Send 1
Track 5  ──────────────► Send 1
Track 6  ──────────────► Send 1
Track 7  ──────────────► Send 1
Track 8  ──────────────► Send 1
```

The eight V-Pots now answer one question:

How much of each track is being sent to this destination?

This is an especially natural way to work with effects sends.

If Send 1 feeds a reverb, for example, you can adjust the amount of reverb for eight tracks without changing the basic mixer bank.

Moving Through the Sends

A project may contain more than one Send.

Repeated presses of SEND move through the available Send modes.

Conceptually:

```
SEND
  ↓
Send 1
  ↓
SEND
  ↓
```

```
Send 2
  ↓
SEND
  ↓
Send 3
  ↓
...
```

DrivenByMoss supports Send modes 1–8.

Hold SHIFT while pressing SEND to move through the Sends in the opposite direction.

So:

```
SEND
   → next Send

SHIFT + SEND
   → previous Send
```

This is another modifier pattern we have already encountered:

SHIFT can provide the related or reverse operation.

Jumping Directly to a Send

Cycling is useful when moving between neighbouring Sends.

But if you already know which Send you want, DrivenByMoss provides a quicker route.

Hold SEND and use the channel SELECT buttons to choose the Send directly.

Conceptually:

```
SEND + SELECT 1  → Send 1
SEND + SELECT 2  → Send 2
SEND + SELECT 3  → Send 3
...
SEND + SELECT 8  → Send 8
```

This is an important example of a control acquiring a new meaning from context.

Normally:

```
SELECT 3
```

means:

Select Track 3.

But while SEND is being used as the context:

```
SEND + SELECT 3
```

means:

Select Send 3.

The physical SELECT button has not changed.

The context has.

Switching a Send On or Off

Send level and Send state are two different things.

Turning a V-Pot adjusts the amount being sent.

DrivenByMoss also allows the Send itself to be switched on or off.

In Send Mode:

```
OPTION + V-Pot press
```

toggles the corresponding Send.

So one rotary control can provide both:

```
Turn
   ↓
Send level

OPTION + Press
   ↓
Send on/off
```

This is exactly the kind of compact interaction for which modifiers are useful.

The common operation remains immediately available.

The related secondary operation is one modifier away.

Send Mode as a Mixing Tool

Send Mode is worth thinking about as more than a shortcut.

Suppose Send 1 contains a reverb and Send 2 contains a delay.

You can move between two complete views of your mix:

```
Send 1 — Reverb
────────────────────
Track 1 amount
Track 2 amount
Track 3 amount
...
Track 8 amount
```

then:

```
Send 2 — Delay
────────────────────
Track 1 amount
```

```
Track 2 amount
Track 3 amount
...
Track 8 amount
```

Rather than visiting each track and adjusting one Send at a time, you can work across the mix according to the effect you are shaping.

For dub-style mixing in particular, this way of thinking can be extremely useful.

ARM, MUTE and SOLO

The channel-strip buttons continue to provide immediate control over the tracks.

ARM

Press ARM on a channel strip to arm that track for recording.

The bank of ARM buttons allows record state to be managed directly from the surface.

MUTE

Press MUTE to mute the corresponding track.

SOLO

Press SOLO to solo the corresponding track.

These are familiar mixer operations, but DrivenByMoss also provides modified versions for useful bank-wide or related operations.

Clearing Mutes

When several tracks have been muted, clearing them individually can be tedious.

DrivenByMoss provides a global operation for clearing active mutes.

This is particularly useful after using mute creatively during playback.

Rather than searching the surface for every illuminated MUTE button, the controller can return the mixer to an unmuted state in one operation.

The exact global-control combination will be included in the Quick Reference once the final command set has been verified against the current DrivenByMoss version.

Clearing Solos

The same problem occurs with Solo.

During mixing it is easy to accumulate a state in which one or more tracks are soloed.

DrivenByMoss provides a global operation for clearing active solos.

Again, the important workflow idea is:

```
Experiment
    ↓
Mute / Solo tracks
    ↓
Return the mixer to a known state
```

The controller is not merely a way to activate states.

It also provides efficient ways to recover from them.

Monitoring

SHIFT-modified channel buttons provide access to monitoring-related functions.

These are useful when recording and will become more meaningful when we look at advanced recording workflows later in the guide.

For now, the important distinction is that ARM, MUTE and SOLO buttons are not necessarily limited to the words printed on their caps.

Like the rest of the X-Touch, they participate in the modifier system.

FLIP

FLIP changes the relationship between rotary and fader control.

This becomes especially useful in the mixer edit modes.

A parameter that would normally be controlled from a V-Pot can be placed onto the motor faders.

Why might we want that?

Because a fader gives us:

- a long physical travel;
- touch sensitivity;
- motorised feedback;
- a very different physical feel from a rotary encoder.

For some adjustments, the V-Pot is ideal.

For others, the fader is more expressive.

FLIP lets the controller adapt.

Conceptually:

```
Before FLIP

V-Pots   → parameter
Faders   → volume


After FLIP

V-Pots   → volume
Faders   → parameter
```

The exact result depends on the current mode, but the principle remains the same:

FLIP exchanges control roles.

Mixing by Dimension

The most useful way to think about this chapter is not as a collection of modes.

Think of it as several different ways to slice through the same mixer.

By channel

```
Track 1
   ├── Volume
   ├── Pan
   ├── Sends
   ├── Mute
   └── Solo
```

By parameter

```
Pan
   ├── Track 1
   ├── Track 2
   ├── Track 3
   └── ...
```

By Send

```
Reverb Send
   ├── Track 1
   ├── Track 2
   ├── Track 3
   └── ...
```

None of these is more correct than another.

They are different views of the same project.

Choosing the Useful View

A good control surface should reduce the distance between an intention and an action.

If your intention is:

Turn Track 4 down.

the motor fader is probably already exactly what you want.

If your intention is:

Spread these eight tracks across the stereo field.

Panorama Mode gives you eight related controls together.

If your intention is:

Decide which tracks should feed the delay.

Send Mode gives you a complete view of that Send across the bank.

The important skill is therefore not memorising modes.

It is recognising which view best matches the job you are doing.

The Important Idea

Mixer Edit Modes let you reorganise the X-Touch around a particular mixing task.

Instead of always seeing:

```
eight tracks
×
many different controls
```

you can temporarily see:

```
one kind of control
×
eight tracks
```

That change of perspective is what makes these modes useful.

The X-Touch is still an eight-channel control surface.

But DrivenByMoss lets those eight channels become eight simultaneous views of:

- volume;
- panorama;
- Sends;
- track parameters;

depending on what you need at that moment.

Once this way of thinking becomes natural, the controller begins to feel less like a fixed bank of knobs and faders and more like a surface that reorganises itself around the mix.

Coming Next

Mixer Edit Modes help us change what we are controlling.

The next chapter deals with another question:

Where are we in the project?

Markers give us named positions in the timeline, and DrivenByMoss gives the X-Touch several ways to create, display and navigate them.

In the next chapter we will look at Markers and Advanced Navigation.

Chapter 15 - Markers and Advanced Navigation

Markers and Advanced Navigation

Transport controls let us move through a project.

We can play, stop, rewind, fast-forward and use the Jog Wheel to position the cursor precisely.

But as a project grows, there is another way to think about navigation.

Instead of asking:

How far should I move?

we can ask:

Where do I want to go?

That is where markers become useful.

A marker gives a position in the project a meaningful identity.

Instead of remembering that the chorus begins somewhere around bar 33, we can create a marker called:

`Chorus`

and navigate directly in terms of the structure of the music.

DrivenByMoss extends this idea onto the X-Touch.

From Position to Structure

Without markers, navigation is principally about position:

```
1       9       17      25      33      41
|-------|-------|-------|-------|-------|
```

With markers, the same timeline can acquire musical meaning:

```
Intro       Verse       Chorus       Breakdown
  |           |            |             |
  ▼           ▼            ▼             ▼
--●-----------●------------●-------------●------
```

Now the project is not merely a sequence of bars.

It has landmarks.

And once those landmarks exist, the X-Touch can use them for navigation.

The MARKER Button

The MARKER button is the centre of marker operations.

Pressing MARKER enters Marker Mode.

This changes the context of the controller so that markers can be accessed from the surface.

As with the other modes we have encountered:

The hardware has not changed. Its current meaning has.

Marker Mode turns the X-Touch from a device that moves through time into one that can navigate the structure of the project.

Creating a Marker

Hold OPTION and press MARKER:

```
OPTION + MARKER
```

to create a marker at the current position.

Conceptually:

```
Current Position
        │
        │ OPTION + MARKER
        ▼
    New Marker
```

This is particularly useful because creating the marker does not require breaking away from the control surface and reaching for the mouse.

Imagine playback reaching the beginning of an important section.

You stop at the required position and press:

```
OPTION + MARKER
```

That position is now represented by a marker in the project.

The operation that originally looked like an obscure modifier combination now makes sense as part of a workflow:

```
Navigate
   ↓
Find an important position
   ↓
```

```
OPTION + MARKER
   ↓
Create a landmark
```

Showing and Hiding Markers

Hold SHIFT and press MARKER:

```
SHIFT + MARKER
```

to show or hide the markers.

This provides another example of the modifier vocabulary introduced in Chapter 8.

MARKER performs the primary marker operation.

SHIFT + MARKER accesses a related secondary operation.

You do not need to remember this as an isolated shortcut.

Think:

```
MARKER
   → work with markers

SHIFT + MARKER
   → change their visibility
```

Previous and Next Marker

Once markers exist, REWIND and FORWARD gain useful alternative functions.

Normally these controls move through the timeline.

Hold OPTION, however, and they navigate between markers:

```
OPTION + REWIND
   → previous marker

OPTION + FORWARD
   → next marker
```

This gives us two distinct forms of navigation:

```
REWIND / FORWARD
        │
        ├── normally
        │      move through time
        │
        └── OPTION
               move through structure
```

That distinction is worth remembering.

Without OPTION, you are navigating distance.

With OPTION, you are navigating landmarks.

A Different Way to Move Through a Song

Suppose a project contains:

```
Intro
Verse 1
Chorus 1
Verse 2
Chorus 2
Breakdown
Final Chorus
Outro
```

With markers placed at those positions, moving through the project no longer requires repeatedly turning the Jog Wheel or holding FORWARD.

Instead:

```
OPTION + FORWARD
```

can step through the musical sections.

Likewise:

```
OPTION + REWIND
```

moves back through them.

So navigation begins to resemble:

```
Verse 1
   |
   ▼
Chorus 1
   |
   ▼
Verse 2
   |
   ▼
Chorus 2
```

rather than:

```
bar 17
   |
   ▼
bar 33
   |
   ▼
bar 49
```

Both views are useful.

Markers simply add another level of meaning.

Playing from a Marker

In Marker Mode, the V-Pots give us another way to interact with markers.

Press the V-Pot associated with a marker to start playback from that marker.

Conceptually:

```
Marker
   │
   │ press associated V-Pot
   ▼
Playback starts here
```

This turns the X-Touch into something closer to a set of structural navigation controls.

Instead of moving to a location and then pressing PLAY, the marker itself becomes the starting point.

Why Marker Mode Matters

Marker Mode may initially seem like a fairly specialised feature.

It becomes much more useful when we consider how often navigation interrupts a workflow.

Imagine repeatedly working on a transition between a verse and chorus.

With ordinary transport navigation, the cycle might be:

```
Stop
  ↓
Find the position
  ↓
Reposition
  ↓
Play
```

With a marker already placed at the start of the section:

```
Select marker
      ↓
Play from marker
```

The mechanical part of the operation becomes smaller.

That matters because every navigation operation is something happening between musical decisions.

Markers as a Project Map

There is a broader principle here.

A well-marked project effectively contains a map of itself.

For example:

```
SONG
 │
 ├── Intro
 │
 ├── Verse 1
 │
 ├── Chorus 1
 │
 ├── Verse 2
 │
 ├── Chorus 2
 │
 ├── Breakdown
 │
 ├── Final Chorus
 │
 └── Outro
```

The X-Touch can then navigate that map.

This is fundamentally different from using the controller merely as a remote transport.

The transport controls know about time.

Markers know about places.

Together they provide both.

Markers During Arrangement

Markers are particularly useful while arranging.

Suppose you are deciding whether the breakdown is too long.

You might repeatedly compare:

```
Chorus 2
    ↓
Breakdown
    ↓
Final Chorus
```

Marker navigation lets you move between those structural points quickly.

You can concentrate on questions such as:

- Does the transition work?
- Does the next section arrive soon enough?
- Does the energy drop too far?
- Does the final chorus have enough impact?

The X-Touch handles the navigation while your attention remains on the arrangement.

Markers During Mixing

Markers are equally useful during mixing.

A mix rarely needs attention everywhere at once.

You may want to revisit:

```
Vocal entrance
Bass drop
Busy chorus
Quiet breakdown
Final hit
```

Markers can turn these into repeatable destinations.

For example:

```
OPTION + FORWARD
      ↓
Busy Chorus
      ↓
listen
      ↓
OPTION + FORWARD
      ↓
Quiet Breakdown
      ↓
listen
```

This is much faster than repeatedly finding those positions visually.

It also encourages listening rather than screen-watching.

Markers During Performance

Markers can also provide a useful structural framework when Bitwig is being used more performatively.

The important point is not that markers replace clips, scenes or Launcher workflows.

They do not.

Markers describe positions in the Arranger timeline.

But where the Arranger itself forms part of a performance, marker navigation gives the X-Touch another way to interact with that structure.

As always, the appropriate tool depends on the job.

Marker Navigation and the Transport Controls

We first encountered REWIND and FORWARD as transport controls.

Now we can see that they have two related roles.

Ordinary transport

```
REWIND
FORWARD
```

move through the timeline.

Structural navigation

```
OPTION + REWIND
OPTION + FORWARD
```

move through markers.

This is why modifier commands are best learned in context.

If Chapter 8 had simply presented:

```
OPTION + FORWARD = next marker
```

as one line in a large table, it would have been another fact to memorise.

Here, its purpose is obvious.

Marker Mode and the Mental Model

Markers also fit neatly into the mental model developed throughout this guide.

The same physical controls can acquire meaning from context.

For example:

```
V-Pot
  +
Marker Mode
  =
Marker interaction
```

And:

```
FORWARD
   +
OPTION
   =
Next Marker
```

So our increasingly complete model is:

```
Physical Control
       +
Current Mode
       +
Modifier
       =
Current Function
```

The surface is not a collection of controls with permanently fixed meanings.

It is a system of contextual controls.

A Practical Marker Workflow

A simple workflow might look like this.

1. Find the beginning of a section

Use the normal transport controls or Jog Wheel.

```
Navigate
   ↓
Beginning of Chorus
```

2. Create the marker

```
OPTION + MARKER
```

3. Repeat for other important sections

For example:

```
Intro
Verse
Chorus
Breakdown
Outro
```

4. Navigate structurally

Use:

```
OPTION + REWIND
OPTION + FORWARD
```

to move between those landmarks.

5. Enter Marker Mode when appropriate

Use MARKER to expose marker-oriented control from the X-Touch.

6. Start playback from the required marker

Press the corresponding V-Pot.

The result is a project that can increasingly be navigated by musical structure rather than screen position.

You Don't Need a Marker Everywhere

Markers are useful because they simplify navigation.

Too many markers can have the opposite effect.

A project containing:

```
Intro
Verse
Chorus
Breakdown
Outro
```

provides obvious destinations.

A project containing a marker every few bars may simply replace one navigation problem with another.

Use markers for positions you expect to revisit.

Good candidates include:

- major song sections;
- important transitions;
- difficult edits;
- mix-check locations;
- unusual events;
- positions needed repeatedly during a session.

The aim is not to document every metre of the timeline.

It is to create useful landmarks.

Less Looking, More Listening

There is another benefit to marker navigation that fits the larger aim of Project XTC.

If you know that the next press of:

```
OPTION + FORWARD
```

will take you to the next important section, you do not need to locate that section visually.

Your eyes do not need to find the mouse pointer.

The pointer does not need to find the timeline.

The timeline does not need to be zoomed to the right scale.

You simply move to the next landmark.

This is a small example of the mouse-lite principle we will return to later:

The best control-surface operation is often the one that lets your attention remain on the music.

The Important Idea

Transport navigation answers:

How do I move through time?

Marker navigation answers:

How do I move through the structure of my project?

DrivenByMoss gives the X-Touch both.

The essential marker commands are:

```
MARKER
   → Marker Mode

OPTION + MARKER
   → Create marker

SHIFT + MARKER
   → Show/hide markers

OPTION + REWIND
   → Previous marker

OPTION + FORWARD
   → Next marker
```

And within Marker Mode, the V-Pots allow markers to become direct playback destinations.

Once markers are part of your normal project workflow, navigation stops being purely a matter of bars, beats and cursor positions.

The project acquires places you can go.

Coming Next

So far we have used the motor faders to control parameters and respond to changes from Bitwig.

But their touch sensitivity becomes particularly powerful when Bitwig is recording those movements.

That brings us to one of the areas where a motorised control surface can feel dramatically different from using a mouse.

Next:

Automation.

Chapter 16 - Automation

Automation

The motor faders on the X-Touch do something rather special.

Move a fader and Bitwig responds.

Change the same parameter in Bitwig and the physical fader moves.

We saw this two-way relationship earlier:

```
You move the fader
         │
         ▼
      Bitwig

      Bitwig
         │
         ▼
The fader moves
```

Automation adds another dimension.

Bitwig can remember the movement.

Instead of a fader representing one fixed value, it can become part of a performance that changes over time.

From Position to Movement

Suppose a track begins quietly and gradually becomes louder.

Without automation, we might set the fader to one compromise level.

With automation, we can perform the change:

```
quiet                           loud
  │                               │
  ▼                               ▼
  ────────/────────────────────────
         /
        /
```

Bitwig records the changing value.

When the project plays again, that movement can be reproduced.

And because the X-Touch has motorised faders, the physical fader can move with it.

That is one of the moments when a control surface stops feeling like a remote control and starts feeling like part of the DAW.

Automation Is About Behaviour

The X-Touch provides five buttons in its AUTOMATION section:

```
READ/OFF   WRITE   TRIM   TOUCH   LATCH
```

These labels come from the Mackie Control world.

DrivenByMoss maps them onto the automation modes that Bitwig actually provides.

The verified mapping is:

```
READ/OFF
   → Disable Arranger automation recording

WRITE
   → Enable Arranger automation recording
     in Write mode

TRIM
   → Enable Read mode
     because Bitwig has no Trim mode

TOUCH
   → Enable Arranger automation recording
     in Touch mode

LATCH
   → Enable Arranger automation recording
     in Latch mode
```

There is also:

```
OPTION + READ/OFF
   → Reset automation overrides
```

The important lesson is already familiar:

The words printed on the X-Touch tell us where the controls came from. DrivenByMoss determines what they do in Bitwig.

Three Useful Questions

The automation modes become easier to understand if we ask three questions:

```
What happens before I touch the control?

What happens while I move it?
What happens when I release it?
```

For a touch-sensitive motor fader, that gives us a simple model:

```
Touch
  ↓
Move
  ↓
Release
  ↓
What happens now?
```

The answer depends on the automation mode.

READ/OFF

Pressing READ/OFF disables Arranger automation recording.

This puts us back into the normal situation where existing automation can be read without our movements being written as a new automation pass.

Conceptually:

```
Recorded automation
        │
        ▼
      Bitwig
        │
        ▼
   Motor fader
```

The physical fader follows the changing automated value.

This is useful feedback.

You can see and feel the automation happening rather than merely watching a line move on the screen.

Automation Playback

Imagine that a track contains this volume automation:

```
Volume

high          ──────
             /
            /
low   ──────
      ────────────────────► time
```

During playback, the X-Touch fader follows it:

```
Bitwig automation
       │
       ▼
Motor position
       │
       ▼
```

```
Fader moves
```

Do not fight the fader while simply reading automation.

Its movement is information.

The controller is showing you what the project is doing.

WRITE

Pressing WRITE enables Arranger automation recording in Write mode.

Write is the most direct of the automation-writing modes.

When Write is active, the current parameter values are written into the automation while the automation pass is running.

Conceptually:

```
WRITE active
     │
     ▼
Automation recording
     │
     ▼
Current values written
     │
     ▼
Move control
     │
     ▼
New values written
```

Write is powerful precisely because it is direct.

It also deserves care.

If you leave Write active while playing through a section, you can replace automation that you intended to keep.

Think of WRITE as:

Write the current control state into the automation.

That makes it useful when deliberately creating or replacing an automation passage.

TOUCH

Pressing TOUCH enables Arranger automation recording in Touch mode.

TOUCH makes particular sense with the X-Touch because the faders are touch-sensitive.

The important event is not merely moving the fader.

It is touching it.

Conceptually:

```
Existing automation plays
          │
          ▼
     Touch fader
          │
          ▼
You take control
          │
          ▼
      Move fader
          │
          ▼
New movement is written
          │
          ▼
    Release fader
          │
          ▼
Existing automation resumes
```

This is one of the most natural ways to make corrections to an existing automation pass.

You can listen to the automated mix, reach for the fader when something needs changing, make the adjustment, and let go.

Bitwig then resumes control.

Touch as an Override

The relationship can be thought of as a temporary handover:

```
Bitwig
  │
  │ automation playing
  ▼
Fader
  │
  │ you touch it
  ▼
 YOU
  │
  │ you release it
  ▼
Bitwig
```

This is where touch sensitivity becomes much more than a hardware specification.

The fader knows when you have deliberately put your hand on it.

That lets Bitwig distinguish between:

The motor is moving the fader

and:

The user is moving the fader.

That distinction is fundamental to Touch automation.

LATCH

Pressing LATCH enables Arranger automation recording in Latch mode.

LATCH begins similarly to Touch.

Existing automation can play until you take control of the parameter.

The important difference appears when you release it.

Conceptually:

```
Existing automation
        │
        ▼
   Touch fader
        │
        ▼
   Take control
        │
        ▼
    Move fader
        │
        ▼
  Release fader
        │
        ▼
New value remains latched
```

Instead of immediately returning to the previously recorded automation, the new value continues to be written.

That makes Latch useful when you want to establish a new level and keep it there.

TOUCH and LATCH Compared

These two modes are easier to understand side by side.

TOUCH

```
automation
    │
    ▼
  touch
    │
    ▼
your movement
    │
    ▼
 release
    │
    ▼
automation resumes
```

LATCH

```
automation
    │
    ▼
  touch
    │
    ▼
your movement
    │
    ▼
 release
    │
    ▼
new value continues
```

The key difference is therefore not what happens while you are holding the fader.

It is what happens after you let go.

Choosing Between TOUCH and LATCH

Suppose a vocal is slightly too loud for one phrase.

TOUCH is a natural choice:

```
normal automation
       ↓
phrase arrives
       ↓
touch fader
       ↓
pull it down
       ↓
phrase ends
       ↓
release
       ↓
previous automation resumes
```

Now suppose a synth needs to become quieter from the second chorus onwards.

LATCH may make more sense:

```
second chorus
     ↓
touch fader
     ↓
lower level
     ↓
release
     ↓
new level continues
```

The mode follows the musical intention.

What About TRIM?

The X-Touch has a button labelled:

```
TRIM
```

On some automation systems, Trim is a distinct mode for making relative changes to existing automation.

Bitwig does not provide a Trim automation mode.

DrivenByMoss therefore maps the X-Touch's TRIM button to:

```
TRIM
   ↓
Read mode
```

This is important because we should not infer functionality from the word printed on the hardware.

Pressing TRIM does not give Bitwig a console-style Trim automation mode that Bitwig itself does not possess.

The button exists because it is part of the Mackie Control layout.

DrivenByMoss gives it the closest useful Bitwig behaviour.

The Labels Belong to the MCU

TRIM gives us a particularly clear example of a principle that applies throughout the X-Touch.

The hardware carries labels such as:

```
READ/OFF
WRITE
TRIM
TOUCH
LATCH
```

because those controls belong to the Mackie Control design.

But our actual system is:

```
X-Touch
   │
   ▼
Mackie Control messages
   │
   ▼
DrivenByMoss
   │
   ▼
Bitwig
```

So:

Trust the current DrivenByMoss mapping, not assumptions based solely on the button legend.

We have encountered the same principle elsewhere with controls such as USER, DROP and the function buttons.

Resetting Automation Overrides

DrivenByMoss provides a particularly useful modified command:

```
OPTION + READ/OFF
```

This resets automation overrides.

Conceptually:

```
Automation
    │
    ▼
Manual override
    │
    ▼
OPTION + READ/OFF
    │
    ▼
Reset override
```

This gives you a quick way to clear overridden automation states and return parameters to normal automation control.

Arranger Automation

There is one more important word in the mappings above:

Arranger.

The automation buttons normally control Arranger automation recording.

That distinction becomes important because Bitwig also has its Launcher environment.

As we saw in Chapter 19, Arranger and Launcher recording states are related but separate concepts.

DrivenByMoss also provides configuration that can change the priority between Arranger and Clip automation behaviour.

We will return to that in Chapter 21.

For now, the normal automation-button model is:

```
Automation buttons
       │
       ▼
Arranger automation
```

Performing Automation

Automation does not have to be drawn.

It can be performed.

That distinction is especially important with a control surface.

Suppose you want a delay return to rise dramatically into the end of a phrase.

With a mouse, you might create automation points and draw a curve.

With the X-Touch, you can approach the same problem musically:

```
Play
  ↓
Listen
  ↓
Touch control
  ↓
Ride the level
  ↓
Release
```

The automation becomes a recorded gesture.

That is much closer to the way levels were traditionally ridden on a mixing console.

The First Pass Does Not Have to Be Perfect

One advantage of automation modes such as Touch is that automation can be refined.

A workflow might be:

```
First pass
```

```
   |
   ▼
Capture the broad movement
   |
   ▼
Listen again
   |
   ▼
Touch only where needed
   |
   ▼
Correct the problem area
```

This is often more natural than trying to perform an entire complicated automation move perfectly in one pass.

Automation can be treated like any other performance:

record it, listen, and improve it.

Automation and the Motor Fader

The motor fader gives us a particularly clear feedback loop:

```
            You
             |
             ▼
         Motor Fader
             |
             ▼
           Bitwig
             |
             ▼
         Automation
             |
             ▼
           Bitwig
             |
             ▼
         Motor Fader
```

Information travels both ways.

When you perform a move, the X-Touch sends control information to Bitwig.

When Bitwig plays the automation back, it sends the resulting state back to the X-Touch.

The fader therefore becomes both:

- an input device;
- an output display.

That is one of the defining advantages of a motorised control surface.

Don't Grab a Moving Fader Casually

A moving motor fader is telling you that Bitwig is controlling that parameter.

Touching it may have meaning depending on the current automation mode.

So before grabbing a moving fader, know which automation mode is active.

With automation recording disabled, you may simply be observing playback.

In TOUCH, touching the fader can deliberately hand control to you.

In WRITE, parameter states may be written continuously.

The same physical gesture can therefore have very different consequences.

The current mode matters.

Automation Is Not Just Volume

Faders make volume automation particularly obvious, but automation in Bitwig is much broader.

Parameters throughout the project can change over time.

The X-Touch may expose those parameters through:

- faders;
- V-Pots;
- Device Mode;
- Mixer Edit Modes;
- FLIP.

So the same general automation ideas apply beyond channel volume.

Conceptually:

```
Physical control
      │
      ▼
Bitwig parameter
      │
      ▼
Automation
```

If the parameter is automatable and the current X-Touch context gives you control over it, the surface can become part of the automation workflow.

FLIP and Automation

FLIP becomes particularly interesting here.

Suppose a parameter is normally assigned to a V-Pot.

FLIP can place the relevant control onto a fader in supported contexts.

That gives the parameter:

• longer physical travel;
• touch sensitivity;
• motorised position feedback.

So FLIP is not merely a convenience for mixing.

It can change the physical way in which an automation performance is made.

For expressive automation, that can be significant.

Watching Automation Come Back

There is something worth doing at least once simply to understand the system.

Record a deliberate fader movement.

Then:

1. stop;
2. return to a point before the movement;
3. disable automation recording;
4. press PLAY;
5. take your hand away.

Watch the fader reproduce what you performed.

The movement you made has become part of the project.

That simple demonstration makes the feedback loop tangible:

```
You moved it
     ↓
Bitwig remembered it
     ↓
Bitwig plays it
     ↓
The X-Touch moves it
```

A Practical Automation Workflow

A straightforward automation pass might look like this.

1. Choose the parameter

For example, track volume.

2. Choose the automation mode

For an existing automated mix that needs correction, TOUCH may be appropriate.

3. Start playback

Listen rather than watching the screen.

4. Touch the fader

Take control when the musical moment arrives.

5. Perform the movement

Ride the level by ear.

6. Release

In TOUCH, Bitwig returns to the existing automation behaviour.

7. Play the section again

Disable automation recording if necessary and listen to the result while the motor fader reproduces the automation.

8. Correct it if necessary

Automation is editable and repeatable.

You are not committing to a one-take performance.

Automation and Mouse-Lite Working

Automation provides another example of why a control surface can change the relationship with a DAW.

With a mouse, automation often encourages this:

```
Look
  ↓
Point
  ↓
Click
  ↓
Draw
  ↓
Adjust
```

With a touch-sensitive fader:

```
Listen
  ↓
Touch
  ↓
```

```
Move
  ↓
Release
```

Neither approach is universally better.

Drawing automation is invaluable when exact editing is required.

But for a musical gesture, the physical approach can be far more immediate.

The aim of Project XTC is not to prohibit the mouse.

It is to make reaching for it a choice rather than a reflex.

Automation as Performance

This leads to perhaps the most useful way to think about automation with the X-Touch.

Do not think only:

I am programming a parameter change.

Think:

I am performing a parameter change and asking Bitwig to remember it.

That shift in perspective matters.

A fade becomes something you play.

A Send ride becomes something you play.

A filter movement becomes something you play.

And because the X-Touch can reproduce the resulting movement, the surface remains connected to that performance afterwards.

The Important Idea

The automation buttons determine what happens when control passes between Bitwig and you.

The verified normal mappings are:

```
READ/OFF
   → Disable Arranger automation recording

WRITE
   → Write mode

TRIM
   → Read mode
     (Bitwig has no Trim mode)

TOUCH
```

```
   → Touch mode

LATCH
   → Latch mode

OPTION + READ/OFF
   → Reset automation overrides
```

For Touch and Latch, the particularly useful question is:

```
Before I touch:
Who has control?

While I move:
What is being written?

When I release:
What happens next?
```

And the motor fader makes that relationship physical.

You move it.

Bitwig remembers.

Then Bitwig moves it back.

Coming Next

So far, most of our navigation has treated the eight visible channels as a relatively flat bank.

But Bitwig projects can contain structures within structures:

- Groups;
- instruments;
- layers;
- drum pads.

DrivenByMoss allows the X-Touch to navigate those structures too.

Next we will look at:

Groups, Layers and Drum Pads.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 17 - Groups, Layers and Drum Pads

Groups, Layers and Drum Pads

So far, we have often treated the eight channel strips as a window onto a simple row of tracks.

Something like:

```
Track 1   Track 2   Track 3   Track 4
Track 5   Track 6   Track 7   Track 8
```

Real Bitwig projects are rarely that simple.

A track may be a Group containing other tracks.

An instrument may contain Layers.

A Drum Machine may contain many Drum Pads.

So the project can have structure within structure:

```
Project
   │
   ├── Drums
   │     ├── Kick
   │     ├── Snare
   │     └── Hats
   │
   ├── Bass
   │
   └── Synth
         │
         ├── Layer 1
         └── Layer 2
```

DrivenByMoss allows the X-Touch to move through these structures.

The eight channel strips do not merely move sideways through the project.

They can also move deeper into it.

A Correction to an Easy Assumption

It would be natural to imagine hierarchical navigation working like this:

```
SELECT
   ↓
ENTER
```

```
   ↓
work inside
   ↓
CANCEL
   ↓
go back
```

That is not how DrivenByMoss navigates Groups and Layers.

The actual mechanism is built primarily around the channel SELECT buttons.

For hierarchical Group navigation:

```
SELECT Group
     ↓
Group becomes selected
     ↓
Press the same SELECT again
     ↓
Enter Group
     ↓
Long-press any SELECT
     ↓
Leave Group
```

For Layers and Drum Pads:

```
SELECT Track
     ↓
Track becomes selected
     ↓
Press the same SELECT again
     ↓
Enter Layers / Drum Pads
     ↓
Long-press any SELECT
     ↓
Leave Layers / Drum Pads
```

This repeated-SELECT behaviour is central to understanding the chapter.

Why ENTER and CANCEL Are Not Used Here

The X-Touch does have ENTER and CANCEL buttons.

They simply have different jobs.

When the Browser is active:

```
ENTER
   → confirm Browser selection

CANCEL
   → discard Browser selection
```

Outside the Browser they behave like the computer keyboard's Enter and Escape keys.

They are therefore useful controls.

They are just not the mechanism for entering and leaving Groups or Layers.

For the structures in this chapter, think:

```
SELECT
   → choose

SELECT again
   → go inside

Long-press SELECT
   → come back out
```

Groups

A Bitwig Group is a track that contains other tracks.

For example:

```
Drums
   │
   ├── Kick
   ├── Snare
   ├── Hats
   ├── Toms
   └── Percussion
```

At the top level of the project, the X-Touch may initially show:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

The individual drum tracks are not necessarily part of this top-level view.

They live inside the Drums Group.

Hierarchical Track Navigation

DrivenByMoss provides a Track navigation preference.

When it is set to:

```
Hierarchical
```

the controller presents the project according to its Group structure.

At the top level, that might look like:

```
┌───────┬──────┬─────────┬──────┬────────┬─────┐
│ Drums │ Bass │ Guitars │ Keys │ Vocals │ FX  │
└───────┴──────┴─────────┴──────┴────────┴─────┘
```

The contents of the Drums Group do not have to consume channel strips until you actually enter that Group.

This allows a large project to remain manageable on an eight-channel surface.

Entering a Group

Suppose the Drums Group is on Channel 1.

First press its SELECT button:

```
Drums
  │
  ▼
SELECT
```

The Drums Group becomes the selected track.

Now press the same SELECT button again:

```
Drums selected
      │
      ▼
SELECT again
      │
      ▼
Enter Drums Group
```

The eight channel strips now represent tracks inside the Group.

For example:

```
Kick   Snare   Hats   Toms   Perc   Room
```

The physical hardware has not changed.

The level of the project represented by it has changed.

Leaving a Group

To leave the current Group:

```
Long-press any SELECT button
```

Conceptually:

```
Kick   Snare   Hats   Toms   Perc
              │
              ▼
     long-press SELECT
              │
              ▼
Drums   Bass   Guitars   Keys   Vocals
```

This takes the controller back up to the parent level.

So the essential hierarchical movement is:

```
SELECT
   ↓
select object

SELECT again
   ↓
```

```
enter object

long-press SELECT
   ↓
leave object
```

SELECT Has Acquired a Second Dimension

Earlier in Project XTC, SELECT primarily meant:

This one.

That remains true.

But in a hierarchical project, repeated SELECT can also mean:

Show me inside this one.

So SELECT now has two related roles:

```
First press
   → focus

Second press
   → descend, where appropriate
```

and:

```
Long press
   → ascend
```

This is a compact navigation system built into buttons that already have an obvious relationship with the channel strips.

Think of It as Zooming Into the Mixer

Another useful mental model is magnification.

At first:

```
Drums   Bass   Guitars   Keys   Vocals
```

Then:

```
SELECT Drums
SELECT Drums again
```

and the controller effectively zooms in:

```
Kick   Snare   Hats   Toms   Percussion
```

Long-press SELECT and it zooms back out:

```
Drums   Bass   Guitars   Keys   Vocals
```

Nothing has moved physically.

The X-Touch has changed the scale at which you are viewing the project.

Nested Groups

Groups may themselves contain Groups.

Conceptually:

```
Project
   │
   └── Drums
         │
         ├── Acoustic Kit
         │      ├── Kick
         │      ├── Snare
         │      └── Overheads
         │
         └── Percussion
                ├── Shaker
                └── Tambourine
```

The same principle can be applied repeatedly.

At the top level:

```
Drums
```

Enter Drums:

```
Acoustic Kit   Percussion
```

Enter Acoustic Kit:

```
Kick   Snare   Overheads
```

Then long-press SELECT to move back towards the parent level.

The important point is not how many levels exist.

It is that the navigation rule remains understandable.

The Eight Strips Are a Window

This gives us a more powerful version of the idea introduced in Chapter 4.

Originally:

```
Large project
────────────────────────────────────────

       ┌────────────────────┐
       │ 8 visible tracks   │
       └────────────────────┘
```

Banking moved that window sideways.

Hierarchical navigation adds another direction:

```
             Project
```

```
              │
              ▼
          ┌──────────┐
          │  Group   │
          └──────────┘
              │
              ▼
          Child Tracks
```

So the X-Touch can now move:

```
← sideways through banks →
```

```
and
```

```
↓ deeper into structure
↑ back towards the parent
```

This is how eight physical channel strips can remain useful in a much larger project.

Flat Track Navigation

Hierarchical navigation is not the only option.

DrivenByMoss also provides:

```
Track navigation: Flat
```

In Flat mode, all tracks are presented in one flat track bank rather than using the Group structure as the controller's navigation hierarchy.

That changes what happens when an already-selected Group is selected again.

Repeated SELECT in Flat Mode

With flat navigation:

```
SELECT an already-selected Group
```

does not enter the Group as a new controller level.

Instead it toggles the Group's expanded state.

Conceptually:

```
Drums ▶
```

becomes:

```
Drums ▼
   Kick
   Snare
   Hats
```

and vice versa.

This is an important distinction.

The same physical gesture:

```
SELECT again
```

has different Group behaviour depending on the Track navigation preference.

Flat or Hierarchical?

Neither mode is universally correct.

They represent two different ways of thinking about a project.

Flat

Think:

Show me the tracks as one long mixer.

Hierarchical

Think:

Show me the project structure, and let me enter the part I need.

For a small project, Flat may be extremely convenient.

For a large project containing many Groups, Hierarchical navigation can make eight channel strips feel much less restrictive.

The choice is made in the DrivenByMoss settings, which we discuss in Chapter 21.

Opening and Closing Groups Without Entering Them

DrivenByMoss provides another useful Group command:

```
CONTROL + SELECT
```

If the selected channel is a Group, this opens or closes the Group folder.

So we should distinguish between two ideas:

```
SELECT again
   → hierarchical navigation into the Group
     when Hierarchical navigation is configured
```

and:

```
CONTROL + SELECT
   → open / close the Group folder
```

They are related, but they are not the same operation.

Navigation Versus Presentation

This distinction is easier to understand if we think in terms of:

```
Where am I?
```

versus:

```
How is the Group displayed?
```

Hierarchical SELECT navigation changes the controller's current level.

CONTROL + SELECT changes the Group's open/closed state.

That gives us two different kinds of Group control without requiring a separate bank of dedicated Group buttons.

Layers

Bitwig instruments can contain Layers.

For example:

```
Instrument
   │
   ├── Layer 1
   ├── Layer 2
   ├── Layer 3
   └── Layer 4
```

DrivenByMoss allows the X-Touch channel strips to represent those Layers.

The mechanism is deliberately similar to Group navigation.

Entering Layers Mode

First select the track containing the instrument.

If it is not already selected:

```
SELECT Track
```

Once that track is selected, press its SELECT button again.

If the track contains an instrument with Layers or Drum Pads at the top level, DrivenByMoss enters Layers mode.

Conceptually:

```
Synth Track
    │
    ▼
SELECT
    │
    ▼
Track selected
    │
    ▼
```

```
SELECT again
    │
    ▼
Layer 1   Layer 2   Layer 3   Layer 4
```

The channel strips now represent the instrument's internal elements rather than normal project tracks.

Leaving Layers Mode

The exit mechanism is the same principle used for hierarchical Groups:

```
Long-press any SELECT button
```

So:

```
Layer 1   Layer 2   Layer 3   Layer 4
                   │
                   ▼
          long-press SELECT
                   │
                   ▼
              Synth Track
```

This consistency is useful.

Once the repeated-SELECT / long-press-SELECT model becomes familiar, it can be applied to more than one kind of hierarchy.

Layer Mixing

Once Layers mode is active, the Layers can be edited using familiar mixer concepts.

DrivenByMoss supports Layer control for:

```
Volume

Pan

Sends

Mute

Solo
```

So a layered instrument can effectively become a small mixer inside the larger project mixer.

For example:

```
Layer A   Layer B   Layer C   Layer D
   │         │         │         │
 Volume    Volume    Volume    Volume
```

You can balance the Layers without needing to treat the instrument as one indivisible sound.

Familiar Modes at a Different Level

The important point is that many of the controls you already know continue to make sense.

The context has changed.

Instead of:

```
Track 1   Track 2   Track 3   Track 4
```

you may now have:

```
Layer 1   Layer 2   Layer 3   Layer 4
```

But concepts such as:

```
Volume
Pan
Send
Mute
Solo
```

remain familiar.

This is a recurring DrivenByMoss design idea:

Learn a small number of interaction patterns, then reuse them at different levels of the project.

Drum Pads

Drum Pads follow the same general model.

Suppose a Bitwig Drum Machine contains:

```
Kick
Snare
Closed Hat
Open Hat
Clap
Rim
Shaker
Tambourine
```

The X-Touch can expose these through its channel strips when the containing track is entered in Layers/Drum Pad mode.

Entering Drum Pad Mode

Select the track containing the Drum Machine.

Then press SELECT again:

```
Drum Track
    │
    ▼
SELECT
    │
    ▼
Track selected
    │
    ▼
SELECT again
    │
    ▼
Kick   Snare   CHat   OHat   Clap   Rim   Shaker   Tamb
```

The eight physical channel strips are now a window onto the Drum Machine's pads.

A Drum Machine Becomes a Mixer

This is where the idea becomes particularly useful.

Instead of treating the Drum Machine as one track:

```
Drums
  │
  ▼
one volume
```

you can work with its internal sounds:

```
Kick    Snare    Hats    Clap    Perc
 │        │        │       │       │
 ▼        ▼        ▼       ▼       ▼
level    level    level   level   level
```

That gives the X-Touch access to a level of detail that would otherwise require returning to Bitwig's graphical device interface.

Drum Pad Mute and Solo

Mute and Solo become especially useful here.

Suppose you want to hear only the kick and snare.

You can use the familiar SOLO buttons at the Drum Pad level.

Or perhaps one percussion sound is getting in the way.

Use MUTE on that pad.

The physical controls have not changed.

Their targets have.

This is another example of why the current context matters so much on the X-Touch.

Drum Pad Sends

Sends can also be useful at the Layer or Drum Pad level.

Imagine a Drum Machine where:

```
Kick
   → mostly dry

Snare
   → some reverb

Clap
   → more reverb

Percussion
   → delay
```

Rather than processing the entire Drum Machine identically, individual elements can participate differently in the mix.

This can be particularly useful in electronic music and dub-style workflows.

The Same Surface at Different Scales

We can now see several possible meanings for one physical channel strip.

At one moment it may represent:

```
Track
```

At another:

```
Group child
```

At another:

```
Instrument Layer
```

At another:

```
Drum Pad
```

Yet the controls remain recognisable:

```
V-Pot

ARM

SOLO

MUTE

SELECT
Fader
```

The X-Touch is not changing physically.

DrivenByMoss is changing the context represented by the hardware.

Context Is Everything

This is why the scribble strips and displays matter.

If Channel 3 represents:

```
Bass
```

you need to know that.

If you enter a Group and Channel 3 now represents:

```
Hi-Hat
```

you need to know that too.

If you then enter a layered instrument and Channel 3 represents:

```
Noise Layer
```

the same physical controls have acquired yet another meaning.

The surface is contextual.

The feedback tells you what the current context is.

A Complete Hierarchical Example

Imagine this project:

```
Project
│
├── Drums
│   ├── Acoustic Kit
│   └── Drum Machine
│
├── Bass
│
├── Synths
│   ├── Pad
│   └── Lead
│
└── Vocals
```

Assume Track navigation is set to Hierarchical.

At the top level, the X-Touch might show:

```
Drums   Bass   Synths   Vocals
```

You want the snare inside the Drum Machine.

Step 1 — Select Drums

```
SELECT Drums
```

Drums becomes the selected Group.

Step 2 — Enter Drums

Press the same SELECT again:

```
SELECT Drums again
```

The surface now shows:

```
Acoustic Kit   Drum Machine
```

Step 3 — Select Drum Machine

```
SELECT Drum Machine
```

The Drum Machine track becomes selected.

Step 4 — Enter its Drum Pads

Press the same SELECT again:

```
SELECT Drum Machine again
```

Now the surface might show:

```
Kick   Snare   CHat   OHat   Clap   Rim   Perc   Tamb
```

Step 5 — Adjust the Snare

The Snare is now simply one of the visible channel-strip targets.

Use the appropriate edit mode to adjust:

```
Volume
```

```
Pan
```

```
Send
```

```
Mute
```

```
Solo
```

Step 6 — Leave Drum Pad Mode

Long-press any SELECT button.

You return from the Drum Pads to the containing track context.

Step 7 — Leave the Drums Group

Long-press SELECT again as appropriate to return to the parent Group level.

Eventually:

```
Drums   Bass   Synths   Vocals
```

is visible again.

We have moved:

```
Project
   ↓
Drums Group
   ↓
Drum Machine
   ↓
Drum Pads
   ↓
Snare
```

and then back out again.

All without needing ENTER or CANCEL for the hierarchy.

SELECT as Navigation

We can now refine the mental model of SELECT that began much earlier in this guide.

At its simplest:

```
SELECT
   → this one
```

But in a hierarchical context:

```
SELECT
   → focus this object

SELECT again
   → enter its internal level,
     where supported

long-press SELECT
   → leave the current internal level
```

And with a modifier:

```
CONTROL + SELECT
   → open / close a Group folder
```

That is a remarkable amount of navigation built around one consistent row of buttons.

Other Useful SELECT Modifiers

DrivenByMoss also provides several other operations on the channel SELECT buttons.

These are worth recognising even though they are not all specifically about hierarchy.

```
SHIFT + SELECT
   → Multi-select tracks,
     where supported by the DAW

OPTION + SELECT
   → Stop the playing clip
     on that track

CONTROL + SELECT
   → Open / close Group folder

ALT + SELECT
   → Set New Clip Length

SEND + SELECT
   → Select Send 1–8
```

So SELECT is not merely a row of eight identical track-selection switches.

It is one of the most contextually useful parts of the surface.

Hierarchy and Banking Work Together

Entering a Group does not mean that banking ceases to matter.

A Group may contain more than eight child tracks.

For example:

```
Drums
│
├── Kick
├── Snare Top
├── Snare Bottom
├── Hi-Hat
├── Tom 1
├── Tom 2
├── Overheads
├── Room
├── Shaker
├── Tambourine
└── Claps
```

The first bank might expose:

```
Kick   SnTop   SnBot   Hat   Tom1   Tom2   OH   Room
```

Then BANK can move the eight-channel window further through the Group.

So we now have two complementary navigation systems:

```
Hierarchy
   → choose the level

Banking
   → choose the portion of that level
```

This is how the X-Touch scales.

A Useful Mental Picture

Imagine the Bitwig project as a building.

Groups are rooms.

Tracks, Layers and Pads are things inside those rooms.

The X-Touch gives you a window.

BANK moves the window sideways:

```
←          →
```

Hierarchical SELECT navigation moves you through the structure:

```
↓ enter
```

```
↑ leave
```

The eight physical strips remain the same size.

But the space they can explore becomes much larger.

Why Hierarchical Navigation Matters

Without hierarchy, a controller with eight channel strips can seem limited.

A project may contain:

```
40 tracks
```

or:

```
80 tracks
```

or more.

But a large project is often organised into meaningful structures:

```
Drums
```

```
Bass
```

```
Guitars
```

```
Synths
```

```
Vocals
FX
```

Hierarchical navigation lets the controller work with those structures rather than treating the project as one enormous flat list.

At the top level, the project can remain comprehensible.

When detail is needed, you descend into it.

Then you come back out.

The Controller Follows Your Attention

This leads to a useful way of thinking about the X-Touch.

Your attention may move like this:

```
Whole Project
     ↓
Drums
     ↓
Drum Machine
     ↓
Snare
```

The X-Touch can follow that attention.

Then:

```
Snare
  ↑
Drum Machine
  ↑
Drums
  ↑
Whole Project
```

The controller moves back out with you.

That is more useful than thinking merely in terms of button combinations.

The buttons are implementing a change in attention.

Hierarchical Versus Flat Is a Workflow Choice

There is no requirement to use hierarchical navigation.

If you prefer:

```
one long mixer
```

choose Flat.

If you prefer:

```
top-level structure
       ↓
drill into detail
       ↓
```

```
return
```

choose Hierarchical.

This is a configuration choice, not a test of whether you are using the X-Touch correctly.

Chapter 21 discusses the DrivenByMoss settings in more detail.

A Mouse-Lite Benefit

Groups, Layers and Drum Pads provide a good example of the Mouse-Lite idea.

Without hardware navigation, a common sequence might be:

```
look at screen
     ↓
find Group
     ↓
open Group
     ↓
find track
     ↓
open device
     ↓
find Drum Machine
     ↓
find pad
     ↓
adjust control
```

With a familiar X-Touch workflow:

```
SELECT
   ↓
SELECT again
   ↓
SELECT
   ↓
SELECT again
   ↓
adjust
```

That does not mean the graphical route is wrong.

Sometimes it will be clearer or faster.

But the physical route can allow your hands and ears to remain engaged with the mix.

Don't Descend Further Than You Need

Hierarchy is useful, but there is no prize for navigating to the deepest possible level.

If the problem is:

The entire drum Group is too loud

adjust the Group.

Do not enter it.

If the problem is:

The snare is too loud

then descend far enough to reach the snare.

A useful rule is:

Work at the highest level that solves the problem.

This keeps the workflow efficient.

The Important Idea

The X-Touch's eight channel strips are not tied permanently to eight top-level tracks.

They can represent different levels of the Bitwig project.

With hierarchical Track navigation:

```
SELECT
   → select

SELECT again
   → enter Group

long-press SELECT
   → leave Group
```

For a selected track containing top-level Layers or Drum Pads:

```
SELECT again
   → enter Layers / Drum Pads

long-press SELECT
   → leave Layers / Drum Pads
```

With Flat Track navigation:

```
SELECT an already-selected Group
   → toggle its expanded state
```

And independently:

```
CONTROL + SELECT
   → open / close Group folder
```

Once inside Layers or Drum Pads, familiar mixer concepts continue to apply:

```
Volume
Pan
Sends
```

```
Mute
Solo
```

So the controller can move from:

```
Project
```

to:

```
Group
```

to:

```
Track
```

to:

```
Layer / Drum Pad
```

and back again.

The physical surface stays the same.

Its meaning changes with your attention.

Coming Next

Hierarchy lets us move downward into the details of a project.

But the X-Touch can also move in the opposite direction — upwards to the level of the project itself.

Touching the Master fader enters a special DrivenByMoss context where the V-Pots no longer represent ordinary track controls.

They can control the Master channel, the audio engine and even project navigation.

Next:

Master Mode and Project Control.

Chapter 18 - Master Mode and Project Control

Master Mode and Project Control

Most of the X-Touch is concerned with the tracks that make up a project.

We select them.

We bank through them.

We mix them.

We control their devices and parameters.

But Bitwig also has a level above the individual tracks.

The Master track belongs to the project as a whole.

DrivenByMoss gives the X-Touch a specialised Master Edit Mode for working at that level.

The Master Fader Is More Than a Fader

The obvious job of the Master fader is to control the project's overall output level.

That is already useful.

But with DrivenByMoss, touching the Master fader also changes the controller's context:

```
Touch Master Fader
        │
        ▼
Select Master Track
        │
        ▼
Enter Master Edit Mode
```

So the Master fader performs two related jobs:

- it controls the Master level;
- touching it establishes the Master context.

This is another example of a familiar Project XTC principle:

An action can both control a value and establish focus.

From Track Focus to Project Focus

Earlier chapters concentrated on questions such as:

Which track am I controlling?

Master Mode asks a different question:

What if the thing I want to control belongs to the whole project?

Conceptually:

```
Track Context
     │
     ▼
Track-specific controls


Master Context
     │
     ▼
Master and project-level controls
```

The physical V-Pots are the same.

What they represent has changed.

Entering Master Edit Mode

Touch the Master fader.

DrivenByMoss selects the Master track and enters Master Edit Mode.

The V-Pots then acquire these assignments:

```
Master Edit Mode

V-Pot 1      → Master Volume

V-Pot 2      → Master Panorama

V-Pots 3–5   → Press to toggle
                the project's audio engine

V-Pot 7      → Press for Previous Project

V-Pot 8      → Press for Next Project
```

V-Pot 6 has no Master Mode function documented in the current DrivenByMoss MCU reference.

The important point is not merely the list.

The important point is that the X-Touch has moved from:

```
one track in the project
```

to:

```
the project-level Master context
```

V-Pot 1 — Master Volume

V-Pot 1 controls the Master volume.

The Master fader already provides a large physical control for the same parameter, so this may initially seem redundant.

But redundancy is not necessarily wasteful on a control surface.

It means the current mode remains internally consistent: the V-Pots themselves expose the parameters belonging to the Master context.

Turn V-Pot 1:

```
V-Pot 1
   |
   ▼
Master Volume
```

Press V-Pot 1:

```
V-Pot 1 Press
      |
      ▼
Reset Master Volume
```

The press behaviour follows the general V-Pot principle we met earlier:

Pressing a parameter knob resets it to its default value.

V-Pot 2 — Master Panorama

V-Pot 2 controls the Master panorama.

Turn it:

```
V-Pot 2
   |
   ▼
Master Panorama
```

Press it:

```
V-Pot 2 Press
      |
      ▼
Reset Master Panorama
```

So the first two V-Pots form a straightforward pair:

```
V-Pot 1   Master Volume
```

```
V-Pot 2   Master Panorama
```

Both can be pressed to reset their parameter.

V-Pots 3–5 — Audio Engine Control

V-Pots 3, 4 and 5 have a different kind of assignment.

They are not continuous parameter controls.

Pressing any of them toggles the audio engine on or off for the current project.

Conceptually:

```
Press V-Pot 3, 4 or 5
          │
          ▼
Toggle Project Audio Engine
```

This is an important distinction.

With V-Pots 1 and 2:

```
TURN
   → change a value
```

With V-Pots 3–5:

```
PRESS
   → perform an action
```

The same physical row therefore contains both parameter controls and project-level commands.

Why Three Knobs?

The current DrivenByMoss documentation assigns the same audio-engine toggle behaviour to V-Pots 3, 4 and 5.

Project XTC should describe that behaviour exactly as documented rather than inventing separate meanings for the three controls.

So the useful fact to remember is simply:

```
V-Pots 3–5
   │
   ▼
Press
   │
   ▼
Toggle Audio Engine
```

There is no need to assign a different conceptual role to each one.

What Does Toggling the Audio Engine Mean?

This is a project-level action.

It affects Bitwig's audio engine for the current project rather than a single track, device or parameter.

That makes it qualitatively different from most of the controls we have used so far.

The hierarchy is roughly:

```
Parameter
   ↓
Device
   ↓
Track
   ↓
Group
   ↓
Project
   ↓
Audio Engine
```

Master Edit Mode is one of the places where the X-Touch reaches that upper level.

Use Project-Level Actions Deliberately

Because the audio-engine control affects the project as a whole, it deserves more care than changing one track parameter.

A good habit is:

Check that you are in Master Mode before pressing project-level controls.

The displays and current context should confirm what the V-Pots represent.

As throughout this guide:

Observe before you adjust.

V-Pot 7 — Previous Project

Press V-Pot 7 to switch to the previous project.

Conceptually:

```
Current Project
      │
      │ press V-Pot 7
      ▼
Previous Project
```

This is a surprisingly powerful operation to have on the control surface.

Project navigation itself becomes part of the hardware workflow.

V-Pot 8 — Next Project

Press V-Pot 8 to switch to the next project.

So V-Pots 7 and 8 form a natural pair:

```
V-Pot 7
   ← Previous Project


V-Pot 8
   Next Project →
```

The spatial relationship is easy to remember.

Left means previous.

Right means next.

Moving Between Projects from the Surface

Imagine working through several related Bitwig projects.

Perhaps they are:

- different songs in a set;
- alternative versions;
- works in progress;
- test projects.

Instead of returning to project-selection controls with the mouse, Master Mode can provide direct navigation.

The workflow becomes:

```
Finish with Project A
        │
        ▼
Touch Master Fader
        │
        ▼
Master Edit Mode
        │
        ▼
Press V-Pot 8
        │
        ▼
Project B
```

This is not a glamorous function.

But it can make the overall workflow feel much more continuous.

The Master Fader and Metronome Volume

The Master fader also participates in another useful command:

```
SHIFT + Master Fader
```

changes the Metronome volume.

This is not part of Master Edit Mode itself.

It is a modifier operation on the Master fader.

So the Master area now illustrates three different kinds of context:

```
Master Fader
     │
     ├── move normally
     │      → Master Volume
     │
     ├── touch
     │      → Select Master
     │        and enter Master Edit Mode
     │
     └── SHIFT + move
            → Metronome Volume
```

One physical fader participates in several related workflows.

Master Mode and Context

Suppose you were just working in Device Mode.

The V-Pots represented device parameters.

Then you touch the Master fader.

Now those same V-Pots represent Master and project-level functions.

Conceptually:

```
Device Mode
   │
   ▼
V-Pots = Device Parameters

Touch Master Fader

   │
   ▼

Master Edit Mode
   │
   ▼
V-Pots = Master / Project Controls
```

Nothing about the hardware has changed.

The current focus has changed.

This is precisely the mental model Project XTC has been building throughout the guide.

The Master Track Is a Different Scale

There is a useful way to think about this.

Ordinary track control is local:

```
Track
   │
   ├── Volume
   ├── Panorama
   ├── Sends
   └── Devices
```

Master control is global:

```
Project
   │
   ├── Master Volume
   ├── Master Panorama
   ├── Audio Engine
   └── Project Navigation
```

The X-Touch lets you change scale.

You can work on:

```
one parameter
```

then:

```
one track
```

then:

```
one Group
```

then:

```
the project as a whole
```

without changing control surfaces.

Moving Up and Down the Project

Chapter 17 showed how the controller can descend:

```
Project
   ↓
Group
   ↓
Track
   ↓
Layer / Drum Pad
```

Master Mode reminds us that the controller can also move in the other direction.

From a detailed track or device context:

```
Parameter
   ↑
Device
   ↑
Track
   ↑
Project
```

Touching the Master fader gives us a direct route to that project-level view.

The Hierarchy Keeps Expanding

At this point, our conceptual model looks something like:

```
Project
   │
   ├── Master
   │
   ├── Groups
   │     └── Tracks
   │
   ├── Tracks
   │     └── Devices
   │           └── Parameters
   │
   └── Instruments
         └── Layers / Drum Pads
```

The X-Touch does not expose all of this at once.

It exposes the level that is currently useful.

That is what keeps the surface manageable.

Project Control Is Still Feedback-Driven

Master Mode is another context where displays matter.

When the V-Pots no longer represent ordinary mixer or device parameters, the feedback tells you what they now mean.

That is especially important when some V-Pots are controlling values while others perform project-level actions.

Do not assume that a V-Pot still does what it did moments ago.

Read the controller.

Check the context.

Then act.

The same rule continues to pay off:

Observe before you adjust.

A Practical Master Workflow

A simple Master Mode workflow might look like this.

1. Touch the Master fader

This selects the Master track and enters Master Edit Mode.

2. Check the displays

Confirm that the controller is showing the Master/project context.

3. Adjust Master level if required

Use the Master fader or V-Pot 1.

4. Adjust Master panorama if required

Use V-Pot 2.

5. Toggle the audio engine only deliberately

Press V-Pot 3, 4 or 5 when you actually intend to change the project's audio-engine state.

6. Navigate projects if required

Use:

```
V-Pot 7
   → Previous Project

V-Pot 8
   → Next Project
```

7. Return to ordinary work

Select the required track or mode and continue.

The important thing is the shift in scale.

You temporarily step out of the track-level view, work at the project level, then return.

Project Navigation and Mouse-Lite Working

Project switching is another good example of Mouse-Lite philosophy.

The goal is not:

Never use Bitwig's project interface.

The goal is:

If you already know you want the previous or next project, why should that necessarily require the mouse?

The X-Touch gives you a direct physical route.

That is the recurring theme of this guide:

use the physical surface when it shortens the path between intention and action.

One Surface, Several Scales

By this point, the same X-Touch may have represented:

```
Project Tracks
```

```
Group Contents
```

```
Instrument Layers
```

```
Drum Pads
```

```
Device Parameters
```

```
Mixer Dimensions
```

```
Master / Project Controls
```

The hardware has not become larger.

The conceptual surface has.

That is the power of context.

The Important Idea

Master Mode changes the X-Touch's point of view from:

one part of the project

to:

the project-level Master context.

Touching the Master fader enters that mode.

The verified Master Edit Mode assignments are:

```
V-Pot 1
   → Master Volume
```

```
   → Press to reset

V-Pot 2
   → Master Panorama
   → Press to reset

V-Pots 3–5
   → Press to toggle
     the project's audio engine

V-Pot 7
   → Previous Project

V-Pot 8
   → Next Project
```

And independently:

```
SHIFT + Master Fader
   → Metronome Volume
```

So our mental model expands once more:

```
Physical Control
      +
Current Focus
      +
Current Mode
      +
Modifier, if any
      =
Current Function
```

At track level, the focus is a track.

At Device level, it is a device.

Inside a Group, it may be a child track.

In Master Edit Mode, it is the Master and project context.

The same surface moves between all of them.

Coming Next

So far, we have looked at how to navigate, mix, automate and control increasingly complex structures.

The next chapter turns to another fundamental workflow:

recording.

DrivenByMoss gives the X-Touch explicit operations for:

- normal recording;
- Launcher overdub;
- Arranger overdub;

- clip creation;
- New Clip Length.

Next:

Advanced Recording and Overdub.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 19 - Advanced Recording and Overdub

Advanced Recording and Overdub

Recording sounds simple.

Press RECORD.

Play something.

Press STOP.

And for straightforward Arranger recording, that may be all you need.

But Bitwig has more than one recording environment.

It has the linear Arranger.

It has the performance-oriented Clip Launcher.

And it has overdub, where new material is added without replacing what is already there.

DrivenByMoss gives the X-Touch direct access to these different recording workflows.

The important thing is knowing which kind of recording you are asking Bitwig to perform.

The Basic RECORD Button

The normal RECORD button performs the most obvious function:

```
RECORD
   │
   ▼
Start / Stop Recording
```

This is the standard recording command.

For a conventional linear recording workflow:

```
Arm Track
    ↓
Position Play Cursor
    ↓
```

```
RECORD
    ↓
Perform
    ↓
STOP
```

there is nothing mysterious about it.

But RECORD also participates in two important modifier combinations.

Three RECORD Operations

DrivenByMoss documents these three RECORD commands:

```
RECORD
   → Start / Stop recording


SHIFT + RECORD
   → Toggle Launcher overdub


OPTION + RECORD
   → Create a new clip on the selected
     track and slot,
     start playback,
     and enable overdub
```

These are not three variations of the same operation.

They represent three different recording intentions.

RECORD — Record Normally

Think:

Record.

```
RECORD
   ↓
Normal recording
```

This is the direct transport operation.

It is the appropriate starting point when your intention is simply to record in the normal Bitwig workflow.

SHIFT + RECORD — Launcher Overdub

Hold SHIFT and press RECORD:

```
SHIFT + RECORD
       │
       ▼
Toggle Launcher Overdub
```

This does not merely start another ordinary recording pass.

It changes the Launcher overdub state.

Think:

When working with Launcher clips, allow new material to be added to what is already there.

OPTION + RECORD — Create, Play and Overdub

This is the more specialised command:

```
OPTION + RECORD
       │
       ▼
Create new clip
on selected track and slot
       │
       ▼
Start playback
       │
       ▼
Enable overdub
```

That combination is particularly interesting because one physical command establishes an entire clip-recording state.

It does not merely toggle an existing setting.

It creates something and gets it ready for performance.

Arranger and Launcher

To understand these commands, we need to distinguish Bitwig's two principal working environments.

Conceptually:

```
Bitwig
  │
  ├── Arranger
  │      │
  │      └── linear timeline
  │
  └── Launcher
         │
         └── clips / scenes
```

Both can contain musical material.

Both can participate in recording.

But they encourage different ways of working.

Arranger Thinking

The Arranger is fundamentally linear.

Think:

```
START ──────────────────────────────► END
```

A song develops through time.

Recording naturally fits a model such as:

```
Bar 1
  ↓
Bar 2
  ↓
Bar 3
  ↓
Bar 4
  ↓
...
```

This is familiar tape-machine-style thinking.

Launcher Thinking

The Launcher is more modular.

Think:

```
Clip A    Clip B    Clip C

Clip D    Clip E    Clip F

Clip G    Clip H    Clip I
```

A clip can repeat.

Another clip can be launched.

Material can be built incrementally.

That makes overdub particularly important.

What Is Overdub?

Suppose a MIDI clip already contains:

```
Kick
Kick
Kick
Kick
```

You want to add a hi-hat pattern without losing the kicks.

Ordinary replacement recording would be the wrong idea.

Overdub means:

```
Existing material
      +
New performance
      =
Combined clip
```

So:

```
Pass 1
Kick     Kick     Kick     Kick

Pass 2
  Hat Hat   Hat Hat   Hat Hat   Hat Hat

Result
Kick+Hat  Kick+Hat  Kick+Hat  Kick+Hat
```

The second pass adds to the first.

Overdub Is Layering in Time

A useful mental model is:

```
First pass
    ↓
Listen
    ↓
Second pass
    ↓
Add
    ↓
Third pass
    ↓
Add
```

The clip grows through repeated performances.

This is especially useful for:

- drum programming;
- percussion;
- layered MIDI parts;
- loop-based composition;
- performance-oriented workflows.

The OVR Button

The X-Touch has an OVR button.

In DrivenByMoss:

```
OVR
   → Toggle Arranger Overdub
```

and:

```
SHIFT + OVR
   → Toggle Launcher Overdub
```

This gives us a very useful pair:

```
OVR
   → Arranger overdub

SHIFT + OVR
   → Launcher overdub
```

The modifier changes the target environment.

RECORD and OVR Are Related but Different

It is worth separating two concepts.

RECORD answers:

Should recording run?

OVR answers:

When recording, should new material be added to existing material?

Conceptually:

```
RECORD
   |
   ▼
Recording state


OVR
   |
   ▼
Overdub behaviour
```

They interact, but they are not interchangeable.

A Useful Recording Map

The verified normal mapping can be summarised as:

```
RECORD
   → Start / Stop recording

SHIFT + RECORD
   → Toggle Launcher overdub

OPTION + RECORD
   → Create new clip
     + start playback
     + enable overdub

OVR
   → Toggle Arranger overdub
SHIFT + OVR
```

```
→ Toggle Launcher overdub
```

Notice something interesting:

```
SHIFT + RECORD
```

and:

```
SHIFT + OVR
```

both give access to Launcher overdub.

That is not a documentation mistake.

DrivenByMoss provides more than one physical route to the same state.

Why Provide Two Routes?

The two routes make sense in different contexts.

If your hand is already around the transport controls:

```
SHIFT + RECORD
```

may feel natural.

If you are thinking specifically in terms of overdub state:

```
SHIFT + OVR
```

may be easier to remember.

A control surface does not always need one and only one route to every function.

Sometimes redundancy makes a workflow easier.

OPTION + RECORD Is Different

OPTION + RECORD deserves special attention because it does more than toggle Launcher overdub.

It performs a sequence:

```
Selected Track
      +
Selected Slot
      │
      ▼
OPTION + RECORD
      │
      ▼
Create Clip
      │
      ▼
Start Playback
      │
      ▼
```

```
Enable Overdub
```

This is a workflow command.

One button combination prepares the selected Launcher location for loop-oriented recording.

Selected Track and Selected Slot Matter

OPTION + RECORD operates on:

```
Selected Track
      +
Selected Slot
```

So before invoking it, establish the destination.

The command is powerful because it acts immediately.

That means focus matters.

A useful habit is:

```
Select destination
      ↓
Check destination
      ↓
OPTION + RECORD
```

As always:

Establish focus before performing an action that creates something.

New Clip Length

When creating clips from the controller, another setting becomes important:

```
New Clip Length
```

DrivenByMoss allows the default length for newly created clips to be configured.

This means the recording workflow can be prepared before the performance begins.

For example:

```
New Clip Length
      │
      ▼
4 bars
      │
      ▼
Create new clip
      │
      ▼
Four-bar working structure
```

The exact choice depends on the music.

Setting New Clip Length from the Surface

DrivenByMoss also lets the X-Touch choose a new clip length using the track SELECT buttons with a modifier.

The eight choices are:

```
SHIFT + SELECT 1   → 16 bars
```

```
SHIFT + SELECT 2   → 8 bars
```

```
SHIFT + SELECT 3   → 4 bars
```

```
SHIFT + SELECT 4   → 2 bars
```

```
SHIFT + SELECT 5   → 1 bar
```

```
SHIFT + SELECT 6   → 2 beats
```

```
SHIFT + SELECT 7   → 1 beat
```

```
SHIFT + SELECT 8   → 32 bars
```

So New Clip Length does not necessarily require a trip to the DrivenByMoss preferences.

It can be changed directly from the surface.

The Order Is Worth Learning

The sequence is not simply shortest-to-longest or longest-to-shortest.

It is:

```
SELECT 1     16 bars
SELECT 2      8 bars
SELECT 3      4 bars
SELECT 4      2 bars
SELECT 5      1 bar
SELECT 6      2 beats
SELECT 7      1 beat
SELECT 8     32 bars
```

That final 32-bar assignment means it is worth learning the mapping rather than assuming the buttons form a perfectly linear progression.

This is exactly the sort of detail that will belong in the Quick Reference later.

ALT + SELECT and New Clip Length

DrivenByMoss's common edit-mode functions also document:

```
ALT + Track SELECT
   → Set the length of a new clip
```

The important conceptual point is that the channel SELECT row can participate in clip-length selection as well as track selection.

This is another example of modifiers turning familiar controls into a temporary command surface.

The exact controller feedback should always be observed when using these modified functions.

A Four-Bar Loop Workflow

Suppose you want to build a four-bar MIDI percussion loop.

A controller-oriented workflow could be:

1. Select four bars

Use the appropriate New Clip Length selection:

```
SHIFT + SELECT 3
```

2. Select the destination track and slot

Establish where the new clip should be created.

3. Press OPTION + RECORD

DrivenByMoss creates the clip, starts playback and enables overdub.

4. Play the first layer

For example:

```
Kick
```

5. Let the clip repeat

Listen to what you recorded.

6. Add another layer

For example:

```
Snare
```

7. Add another

Perhaps:

```
Hi-Hat
```

The clip grows while playback continues.

Building Rather Than Recording

This workflow changes the meaning of the word recording.

Instead of:

```
Record complete performance
          ↓
Stop
```

we have:

```
Create loop
    ↓
Add idea
    ↓
Listen
    ↓
Add idea
    ↓
Listen
    ↓
Add idea
```

That is closer to building than traditional recording.

The X-Touch can participate in both approaches.

Arranger Overdub

Now consider the Arranger.

Suppose a MIDI passage already exists and you want to add notes without replacing the existing performance.

Enable:

```
OVR
```

DrivenByMoss toggles Arranger overdub.

Conceptually:

```
Existing Arranger material
          +
New recorded material
          │
          ▼
Combined result
```

This is the linear-timeline counterpart to Launcher overdub.

Launcher Overdub

For Launcher clips:

```
SHIFT + OVR
```

toggles Launcher overdub.

Or:

```
SHIFT + RECORD
```

can toggle the same Launcher overdub state.

So the conceptual distinction remains simple:

```
OVR
   → Arranger

SHIFT + OVR
   → Launcher
```

even though Launcher overdub also has the SHIFT + RECORD route.

A Note About Audio and MIDI

Overdub is particularly easy to understand with MIDI because new notes can be added to an existing clip.

For example:

```
Pass 1   Kick

Pass 2   Snare

Pass 3   Hats
```

produces one increasingly complete MIDI performance.

Do not assume that every audio-recording situation behaves identically.

Bitwig's recording behaviour depends on the type of track, clip and recording context.

Project XTC's concern here is the X-Touch command mapping:

```
which recording state
or overdub state
does the controller request?
```

The detailed editing consequences remain Bitwig behaviour.

Recording Is a State Machine

It can help to think of the recording controls as changing states.

For example:

```
Stopped
   │
   │ RECORD
   ▼
Recording
   │
   │ RECORD / STOP
   ▼
Stopped
```

Then overdub adds another dimension:

```
Recording
   │
   ├── Arranger overdub OFF
   │
   └── Arranger overdub ON
```

And Launcher operation has its own overdub state:

```
Launcher
   │
   ├── Overdub OFF
   │
   └── Overdub ON
```

Thinking in terms of states helps explain why several buttons can appear to affect “recording” while actually controlling different aspects of it.

Check the State Before Performing

Recording commands are consequential.

A mistaken Pan value is easy to undo.

Recording into the wrong clip or changing the wrong overdub state can be more disruptive.

So before beginning:

```
Check Track
     ↓
Check Slot
     ↓
Check Clip Length
     ↓
Check Overdub State
     ↓
Perform
```

The more fluent you become, the faster this check becomes.

Eventually it is no more cumbersome than glancing at a mixer before moving a fader.

Arm Is Still Important

The channel-strip ARM buttons remain the obvious way to establish recording targets.

Press the ARM button for the appropriate track:

```
ARM
 │
 ▼
Track armed
```

DrivenByMoss also provides:

```
SHIFT + ARM
```

to toggle the record-arm state across all tracks in the active Bitwig bank page.

That can be powerful, so use it deliberately.

One Track Versus the Bank

The distinction is:

```
ARM
   → specific track
```

versus:

```
SHIFT + ARM
   → toggle record-arm state
     across tracks in active bank page
```

Again, the modifier changes the scale of the command.

This is a pattern we have seen throughout the X-Touch.

Monitoring from the Surface

DrivenByMoss also provides channel-strip monitoring commands:

```
SHIFT + MUTE
   → Toggle monitor

SHIFT + SOLO
   → Toggle auto monitor
```

These functions belong naturally to a recording workflow.

Before recording, you may need to determine how the armed track is monitored.

The X-Touch can therefore participate not only in:

```
recording
```

but also:

```
arming
monitoring
overdubbing
```

Recording Without Constant Screen Attention

A hardware-oriented recording workflow might become:

```
SELECT track
     ↓
ARM
     ↓
```

```
Set monitoring if needed
     ↓
Choose recording / overdub state
     ↓
RECORD
     ↓
Perform
     ↓
STOP
```

For Launcher loop building:

```
Choose clip length
     ↓
Select track / slot
     ↓
OPTION + RECORD
     ↓
Perform
     ↓
Overdub
     ↓
Listen
```

The computer screen remains available.

It simply does not have to mediate every step.

Recording and Musical Attention

This matters because recording is one of the moments when you least want to be thinking about software.

You may be:

- playing a keyboard;
- holding a guitar;
- singing;
- programming drums;
- manipulating another controller.

Every unnecessary mouse operation interrupts that activity.

A control surface can move routine recording operations into physical memory.

Eventually:

```
I want another pass
```

becomes:

```
hand moves
button pressed
continue playing
```

without requiring a menu search.

Launcher-Oriented Users

There is an important DrivenByMoss preference that affects the recording controls:

```
Flip arranger and clip record / automation
```

When enabled, the normal and SHIFT functions of the Record and Automation buttons are flipped between Arranger-oriented and Clip-oriented behaviour.

This exists for users who spend more time in the Clip Launcher than the Arranger.

Conceptually:

```
Default configuration

Normal
   → Arranger-oriented function

SHIFT
   → Launcher-oriented function
```

can become:

```
Flipped configuration

Normal
   → Launcher-oriented function

SHIFT
   → Arranger-oriented function
```

We will discuss this preference properly in Chapter 21.

Why the Flip Preference Matters

Without knowing about this preference, two people can press the same physical buttons and report apparently contradictory behaviour.

One says:

RECORD does this.

Another says:

No, SHIFT + RECORD does that.

Both controllers may be behaving correctly.

Their DrivenByMoss configuration may simply differ.

This reinforces an important documentation rule:

When behaviour can be configured, describe the default and identify the preference that can change it.

Recording and Automation Are Parallel Ideas

Recording musical notes and recording automation are conceptually related.

In both cases:

```
You perform something
        ↓
Bitwig captures it
        ↓
Bitwig reproduces it later
```

For notes:

```
play notes
   ↓
record MIDI
```

For automation:

```
move fader
   ↓
record automation
```

This is one reason the X-Touch can feel so natural in both roles.

It turns software recording into a physical act.

Overdub as a Performance Tool

Overdub is not merely a technical recording option.

It can become part of a performance.

Imagine building a rhythmic clip:

```
Loop 1
   → kick

Loop 2
   → add snare

Loop 3
   → add hats

Loop 4
   → add percussion
```

The arrangement develops through physical performance rather than through drawing notes.

The X-Touch's role is not to create the musical material.

Its role is to make the recording state accessible while the musical material is being created.

A Practical Launcher Workflow

A useful practice exercise is:

1. Create an empty MIDI track

Load an instrument.

2. Select a four-bar New Clip Length

Use:

```
SHIFT + SELECT 3
```

3. Select an empty Launcher slot

Make sure the intended track and slot are selected.

4. Press OPTION + RECORD

The clip is created, playback starts and overdub is enabled.

5. Play a simple pattern

Do not try to make it complicated.

6. Let it loop

Listen.

7. Add another part

Use the active overdub state to layer another performance.

8. Toggle Launcher overdub if required

Use:

```
SHIFT + RECORD
```

or:

```
SHIFT + OVR
```

9. Listen again

The aim of the exercise is not the music.

It is learning the physical relationship between:

```
Clip
   +
Recording
   +
Overdub
   +
X-Touch
```

A Practical Arranger Workflow

For comparison:

1. Select and arm the track

Use the channel-strip controls.

2. Position the play cursor

Use the transport controls or Jog Wheel.

3. Decide whether Arranger overdub is required

If yes:

```
OVR
```

4. Press RECORD

Begin recording.

5. Perform

Concentrate on the music.

6. Press STOP

Finish the pass.

7. Listen

Decide whether another pass is required.

This is much closer to a traditional linear recording workflow.

The same hardware supports both models.

Do Not Memorise Everything at Once

There are several commands in this chapter.

You do not need to learn all of them simultaneously.

Start with:

```
RECORD
   → Record

OVR
   → Arranger overdub
```

Then add:

```
SHIFT + RECORD
   → Launcher overdub
```

Then:

```
OPTION + RECORD
   → New clip + play + overdub
```

Once those are familiar, the rest becomes easier to place around them.

This is the same learning strategy we have used throughout Project XTC:

Build a mental model first. Add commands to it gradually.

The Important Idea

The X-Touch does not have one generic concept called “recording”.

DrivenByMoss exposes several related states and operations.

The verified normal mappings are:

```
RECORD
   → Start / Stop recording

SHIFT + RECORD
   → Toggle Launcher overdub

OPTION + RECORD
   → Create a new clip on the
     selected track and slot,
     start playback,
     enable overdub

OVR
   → Toggle Arranger overdub

SHIFT + OVR
   → Toggle Launcher overdub
```

And the channel strips contribute:

```
ARM
   → Arm specific track

SHIFT + ARM
   → Toggle record-arm state
     across the active bank

SHIFT + MUTE
   → Toggle monitor

SHIFT + SOLO
   → Toggle auto monitor
```

The important question is therefore not simply:

How do I record?

It is:

What am I recording, where am I recording it, and do I want to replace or add to what is already there?

Once that intention is clear, the X-Touch command becomes much easier to choose.

Coming Next

We now have most of the pieces.

We can:

- navigate tracks and banks;
- control devices;
- browse;
- mix;
- use modifiers;
- work with markers;
- perform automation;
- enter Groups, Layers and Drum Pads;
- control the Master context;
- record and overdub.

The next question is:

What does all of this feel like when we stop thinking about individual features and actually use the X-Touch to make music?

Next:

Towards a Mouse-Free — or Mouse-Lite — Workflow.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 20 - Towards a Mouse-Free — or Mouse-Lite — Workflow

Towards a Mouse-Free — or Mouse-Lite — Workflow

We now know a great many things the X-Touch can do.

We can:

- select tracks;
- move through banks;
- control volume, panorama and Sends;
- navigate devices and parameter pages;
- browse for devices and presets;
- work with markers;
- control automation;
- enter Groups, Layers and Drum Pads;
- record and overdub;
- navigate the project.

But knowing a collection of commands is not the same thing as having a workflow.

The real question is:

Can the X-Touch disappear into the process of making music?

That is what this chapter is about.

Mouse-Free Is Not the Goal

The phrase mouse-free sounds attractive.

It suggests complete command of the DAW from hardware.

But taken literally, it can become a trap.

You may find yourself performing six button presses merely to avoid one easy mouse click.

That is not efficiency.

That is ideology.

So Project XTC prefers another phrase:

Mouse-Lite.

The aim is not to eliminate the mouse.

The aim is to stop reaching for it automatically.

The Mouse Is Still Excellent at Some Things

A mouse or trackpad is extremely good for:

- detailed editing;
- drawing;
- dragging;
- arranging clips visually;
- inspecting complex interfaces;
- operations you perform rarely;
- tasks where the screen itself contains the useful information.

The X-Touch is extremely good for:

- repeated mixer operations;
- simultaneous controls;
- tactile adjustments;
- transport;
- navigation;
- performance;
- automation;
- operations that benefit from muscle memory.

So the useful question is not:

Can I do this without the mouse?

It is:

Which tool gives me the shortest and clearest route to what I want?

Intention Before Control

Throughout this guide, we have gradually moved away from thinking in terms of buttons.

Instead of:

Which button does what?

we can increasingly think:

What am I trying to do?

For example:

```
Intention
   │
   ▼
"The vocal needs more delay"
   │
   ▼
Find vocal
   │
   ▼
Select Send
   │
   ▼
Turn V-Pot
```

The hardware becomes a route from intention to result.

That is the real goal.

A Working Vocabulary

By now, a relatively small set of concepts explains a large proportion of the controller.

```
SELECT
   → establish focus

BANK / CHANNEL
   → move the window

MODE
   → change what the controls represent

MODIFIER
   → temporarily extend a control

V-POT
   → adjust the current parameter

FADER
   → make a continuous physical adjustment

DISPLAY
   → tell us what the controls currently mean
```

And hierarchical navigation adds:

```
SELECT again
   → enter, where appropriate

Long-press SELECT
   → leave
```

This vocabulary matters more than memorising hundreds of isolated commands.

Observe Before You Adjust

The X-Touch is contextual.

A V-Pot may control:

```
Pan
```

then:

```
Send level
```

then:

```
Device parameter
```

then:

```
Drum Pad level
```

then:

```
Master panorama
```

The physical control has not changed.

Its meaning has.

So before turning something:

Look at the feedback.

This simple habit prevents a large proportion of control-surface mistakes.

A Three-Step Habit

A useful general workflow is:

```
1. Establish focus
```

```
2. Establish mode
```

```
3. Adjust
```

For example:

```
Select Bass
    ↓
SEND
    ↓
Turn V-Pot
```

Or:

```
Select Synth
    ↓
DEVICE
    ↓
Turn parameter V-Pot
```

Or:

```
Select Drums
    ↓
SELECT again
    ↓
Work inside Group
```

Once that sequence becomes habitual, the X-Touch feels far less complicated.

Navigation Without Losing the Plot

Large projects create a particular problem.

It is easy to lose track of:

Where am I?

The X-Touch may currently represent:

- top-level tracks;
- tracks inside a Group;
- Layers;
- Drum Pads;
- Device parameters;
- Sends;
- Master controls.

So Mouse-Lite working depends on having a reliable mental route back.

Hierarchical Navigation

With hierarchical Track navigation, the verified Group workflow is:

```
SELECT Group
      ↓
Group selected
      ↓
SELECT same Group again
      ↓
Enter Group
      ↓
Long-press any SELECT
      ↓
Leave Group
```

Layers and Drum Pads follow the same broad idea:

```
SELECT containing track
       ↓
SELECT same track again
       ↓
Enter Layers / Drum Pads
       ↓
Long-press any SELECT
       ↓
Leave
```

This gives us a compact navigation vocabulary:

```
SELECT
   → focus

SELECT again
   → descend

Long SELECT
   → ascend
```

Don't Use ENTER and CANCEL for Hierarchy

ENTER and CANCEL are useful X-Touch controls, but they are not the Group-navigation pair.

Their particularly important role is in Browser Mode:

```
ENTER
   → confirm Browser selection

CANCEL
   → discard Browser selection
```

So when navigating project hierarchy, keep the mental model centred on SELECT.

When browsing, ENTER and CANCEL regain their obvious confirm/cancel roles.

This separation makes both workflows easier to remember.

Work at the Highest Useful Level

Suppose the drums are too loud.

There are several possible responses.

You could enter the Drums Group, adjust the kick, snare, hats and percussion individually, then rebalance everything.

Or:

```
Drums Group
     │
     ▼
Lower Group fader
```

If that solves the problem, stop there.

A useful rule is:

Work at the highest level that solves the musical problem.

Only descend into detail when the problem itself is detailed.

Banks Are Not an Obstacle

An eight-channel controller cannot show a forty-track project simultaneously.

That does not mean it cannot control one efficiently.

Think of the X-Touch as a window:

```
Project
────────────────────────────────────────────

        ┌────────────────┐
        │ eight channels │
        └────────────────┘
```

BANK moves the window by eight.

CHANNEL moves it by one.

Hierarchy changes the level at which the window is looking.

Together:

```
BANK
   → move widely

CHANNEL
   → move precisely

SELECT again
   → move deeper

Long SELECT
   → move outward
```

That is a surprisingly capable navigation system.

Keep Related Tracks Together

The controller becomes easier to use when the Bitwig project itself has a sensible structure.

For example:

```
Drums
Bass
Guitars
Keys
Vocals
FX
```

is easier to navigate than a project whose tracks are scattered arbitrarily.

Hardware workflow and project organisation reinforce one another.

A well-organised Bitwig project makes the X-Touch feel more intelligent.

A chaotic project makes the controller work harder.

Use Groups as Navigation Landmarks

Groups are not only useful for audio routing.

They can become landmarks.

For example:

```
Project
   │
   ├── Rhythm
   ├── Instruments
   ├── Vocals
   └── FX
```

With hierarchical navigation, those top-level Groups form a compact map of the project.

Enter only the area you currently need.

This can be much easier than continually banking through a long flat list of tracks.

Device Work Without Hunting

Suppose you want to change a synthesizer parameter.

A screen-first workflow might be:

```
find track
   ↓
click track
   ↓
find device
   ↓
click device
   ↓
find parameter
   ↓
drag parameter
```

With a familiar X-Touch workflow:

```
SELECT track
    ↓
DEVICE
    ↓
choose device / page
    ↓
turn V-Pot
```

In Device Mode:

```
BANK ← / →
   → previous / next device

CHANNEL ← / →
   → previous / next parameter page
```

And the modifiers give us faster direct selection:

```
Hold CONTROL
   → display devices
   → press V-Pot to choose one

Hold OPTION
   → display parameter pages
   → press V-Pot to choose one
```

The aim is not to avoid looking at the device forever.

It is to make routine parameter work available physically.

Pin What Matters

DrivenByMoss provides:

```
OPTION + TRACK
   → Pin cursor track

OPTION + DEVICE
   → Pin cursor device
```

Pinning can be useful when you want the controller's focus to remain attached to something while other activity happens in Bitwig.

This is another Mouse-Lite principle:

If the thing you care about keeps moving away, pin it rather than repeatedly finding it again.

Mixing by Ear

The X-Touch becomes particularly valuable when the screen is no longer the centre of attention.

Try balancing several tracks while deliberately looking away from Bitwig.

Use the faders.

Listen.

Then look back.

This exercise reveals something important.

With a mouse, mixing can easily become:

```
look
  ↓
move
  ↓
look
  ↓
move
```

With physical faders:

```
listen
   ↓
move
   ↓
listen
```

That is a different relationship with the music.

More Than One Control at Once

A mouse pointer controls one thing at a time.

Your hands do not have that limitation.

For example:

```
Left hand
   → Vocal fader

Right hand
   → Delay return fader
```

or:

```
Finger 1
   → Kick

Finger 2
   → Bass
```

This is one of the strongest arguments for a physical mixing surface.

The advantage is not merely that a fader is nicer than a mouse.

It is that several physical controls can be part of one gesture.

FLIP When the Fader Is the Better Instrument

Sometimes a parameter normally lives on a V-Pot but would be easier to perform with a fader.

That is where FLIP becomes useful.

Conceptually:

```
Parameter on V-Pot
       │
       ▼
      FLIP
       │
       ▼
Parameter on Fader
```

The fader gives you:

- longer travel;
- touch sensitivity;

• motorised feedback;
• a different physical gesture.

For automation or expressive Send rides, this can be much more natural.

Automation as Performance

With a mouse, automation often means drawing.

With the X-Touch it can mean performing.

```
Listen
  ↓
Touch fader
  ↓
Move
  ↓
Release
  ↓
Listen back
```

In TOUCH mode, the automation can return to the existing behaviour after you release the fader.

In LATCH, the new value continues.

So instead of thinking:

I need to edit the automation curve.

you can sometimes think:

I need to play this movement.

Recording Without Breaking the Performance

Recording is another area where physical controls can preserve musical attention.

A normal recording workflow might be:

```
SELECT track
     ↓
ARM
     ↓
RECORD
     ↓
perform
     ↓
STOP
```

For Launcher-oriented loop building:

```
Select track / slot
       ↓
Choose New Clip Length
```

```
       ↓
OPTION + RECORD
       ↓
New clip created
       ↓
Playback starts
       ↓
Overdub enabled
       ↓
Perform
```

The point is not that the X-Touch magically creates the performance.

It removes some of the software-management interruption surrounding it.

Markers as Musical Landmarks

Markers are another powerful Mouse-Lite tool.

Instead of dragging the play cursor around a long arrangement, use meaningful locations.

For example:

```
Intro

Verse

Chorus

Breakdown

Outro
```

DrivenByMoss provides:

```
OPTION + MARKER
   → Insert marker at current play position

OPTION + REWIND
   → Previous marker

OPTION + FORWARD
   → Next marker
```

And Marker Mode allows the V-Pots to start playback from marker positions.

Markers turn a long timeline into a set of named musical destinations.

A Practical Mixing Session

Let's put several ideas together.

Imagine a project containing:

```
Drums
Bass
Skank Guitar
Organ
```

```
Percussion
Lead
Vocal
FX
```

You press PLAY.

The mix begins.

The Bass Is Too Loud

Your hand goes directly to the Bass fader.

```
Bass too loud
    ↓
Bass fader
    ↓
lower
```

No menu.

No cursor.

No visual search.

The Vocal Needs More Delay

Select the Vocal track.

Enter the appropriate Send context.

Turn the V-Pot.

```
SELECT Vocal
     ↓
SEND
     ↓
choose delay Send
     ↓
turn
```

Listen to the result.

The Delay Needs a Performance Ride

Perhaps a simple static Send level is not enough.

You want the final word of the phrase to explode into delay.

Use FLIP if appropriate to place the Send on a fader.

Then perform the movement.

```
phrase plays
    ↓
raise Send
```

```
    ↓
word hits delay
    ↓
pull Send back
```

That is not merely editing.

It is mixing as performance.

A Dub-Oriented Example

This becomes especially interesting in dub.

A traditional dub mix treats the mixer almost like an instrument.

Channels disappear.

Echoes appear.

Reverb blooms.

Elements are dropped and returned.

The X-Touch is well suited to this because the physical controls encourage exactly that kind of interaction.

Start with a Stable Mix

Imagine:

```
1   Drums
2   Bass
3   Skank
4   Organ
5   Percussion
6   Vocal
7   FX Return 1
8   FX Return 2
```

First establish a sensible balance.

The point is to have a stable place from which to perform.

Drop the Vocal

At the end of a phrase:

```
Vocal
  ↓
MUTE
```

The space suddenly opens.

Throw the Last Word into Delay

Before muting or dropping the vocal, increase its delay Send.

```
Vocal Send
    ↑
last word
    ↓
delay repeats
```

Then bring the Send back.

The echo continues while the dry vocal disappears.

That is a classic dub gesture.

Ride the Return

The effect return itself can be treated dynamically.

```
Delay Return
     │
     ├── raise
     ├── lower
     ├── mute
     └── return
```

The effect becomes part of the performance rather than a static processor.

Work with Several Controls

One hand might ride the vocal Send.

The other might ride the effect return.

```
Left hand
   → Vocal Send

Right hand
   → Delay Return
```

That interaction is extremely awkward with one mouse pointer.

With physical controls it is natural.

Groups Make Performance Mixes Manageable

Suppose the project is larger than eight tracks.

Create meaningful Groups:

```
Drums
Bass
Instruments
Vocals
FX
```

At the top level, those Groups become performance landmarks.

If the whole drum section needs changing:

```
Drums Group fader
```

may be enough.

If the snare needs changing:

```
SELECT Drums
     ↓
SELECT Drums again
     ↓
find Snare
     ↓
adjust
```

Then long-press SELECT to leave the Group.

The hierarchy follows the level of musical attention.

Drum Machines Can Become Performance Mixers

A Drum Machine can itself become a physical submixer.

Select the containing track.

Press SELECT again to enter its Drum Pads.

Now the channel strips may represent:

```
Kick
Snare
Hat
Clap
Perc
Rim
Shaker
Tamb
```

You can now:

- mute individual elements;
- solo them;
- change levels;
- change panorama;
- adjust Sends.

This can turn a single Bitwig track into a surprisingly tactile performance environment.

Don't Forget the Displays

In a fast performance workflow, the danger is assuming.

You think Channel 4 is still the Organ.

But perhaps you entered a Group.

Or changed bank.

Or switched mode.

Now Channel 4 means something else.

So develop a glance habit:

```
Intention
   ↓
quick glance
   ↓
physical action
```

Not:

```
physical action
   ↓
"Oh."
```

The displays are there to keep the contextual surface understandable.

Use the Screen for Confirmation, Not Permission

A mature Mouse-Lite workflow often changes the role of the computer display.

Initially:

```
Look at screen
     ↓
decide
     ↓
act
```

Later:

```
hear / decide
     ↓
act physically
     ↓
glance if confirmation is useful
```

The screen remains important.

It simply stops being the starting point for every operation.

Learn the Surface in Layers

Do not try to become Mouse-Lite by memorising the entire manual.

Build the workflow gradually.

A useful progression is:

```
Transport
    ↓
Volume
```

```
    ↓
Pan
    ↓
Mute / Solo
    ↓
Banks
    ↓
Sends
    ↓
Devices
    ↓
Modifiers
    ↓
Markers
    ↓
Hierarchy
    ↓
Automation
    ↓
Recording
```

At each stage, use the physical control until it becomes automatic.

Then add another layer.

Muscle Memory Is the Real Upgrade

The X-Touch becomes faster not because its buttons change.

It becomes faster because you stop thinking about them.

At first:

```
I want Send 2
      ↓
Which button?
      ↓
Which mode?
      ↓
Which knob?
```

Later:

```
I want more delay
      ↓
hand moves
```

That is the transition we are aiming for.

Do Not Optimise Rare Tasks

Suppose you perform an operation once every six months.

You discover that it can technically be achieved through:

```
modifier
   +
mode
   +
```

```
three button presses
   +
V-Pot
```

but you cannot remember the sequence.

Use the mouse.

There is no failure in that.

Hardware shortcuts earn their value through repetition.

A useful rule is:

The more often you perform something, the more valuable a physical workflow becomes.

Use Customisation to Remove Repeated Friction

Later, Chapter 22 will look at assignable controls.

The principle is simple.

Do not customise because a button is available.

Customise because you repeatedly encounter this:

```
intention
   ↓
awkward route
   ↓
same awkward route
   ↓
same awkward route
```

Then a custom control can turn it into:

```
intention
   ↓
button
```

Customisation should simplify an established workflow, not replace the process of learning one.

A Mouse-Lite Test

Try working for ten minutes with one rule:

Do not reach for the mouse immediately.

When you want to do something, ask:

```
Do I know the X-Touch route?
       │
       ├── yes → use it
       │
       └── no
            │
```

```
             ▼
Would learning it be useful?
       │
       ├── yes → find out
       │
       └── no  → use mouse
```

This avoids both extremes.

You are neither mouse-dependent nor hardware-dogmatic.

The Mouse-Lite Loop

Over time, the process becomes:

```
Intention
    ↓
Do I know a direct physical route?
    │
    ├── Yes
    │     ↓
    │   X-Touch
    │
    └── No
          ↓
   Is this operation frequent?
          │
          ├── Yes → learn / customise
          │
          └── No  → mouse
```

That is a sustainable workflow.

A Controller Should Reduce Thought

This may sound paradoxical after a guide containing twenty chapters.

But the purpose of learning the controller is eventually to think about it less.

At first:

```
hardware
   ↓
thinking
   ↓
music
```

Eventually:

```
intention
   ↓
movement
   ↓
music
```

The hardware disappears from conscious attention.

That is when it becomes useful.

What Should Stay on the Screen?

Even in a strongly hardware-oriented workflow, the screen remains excellent for information that is fundamentally visual.

For example:

- waveform editing;
- piano-roll editing;
- arrangement structure;
- automation detail;
- plugin interfaces;
- complex routing;
- naming;
- file management.

The X-Touch does not need to replace these things.

It complements them.

What Should Move to the Surface?

Operations that benefit particularly from hardware include:

- transport;
- volume;
- panorama;
- Sends;
- Mute and Solo;
- track selection;
- bank navigation;
- device parameters;
- automation performance;
- marker navigation;
- recording;
- repeated performance actions.

These are operations where physical location and muscle memory can outperform pointing and clicking.

The Screen and Surface Are Partners

The most productive model is therefore not:

```
X-Touch
   VS
Mouse
```

It is:

```
            You
           /   \
```

```
        /      \
   X-Touch    Screen
        \      /
         \    /
          Bitwig
```

Each interface does the work it is best suited to.

The skill lies in moving naturally between them.

A Complete Example

Imagine you are working on a song.

You want to:

1. move to the chorus;
2. lower the bass slightly;
3. add delay to the vocal;
4. adjust a synth parameter;
5. ride the vocal level;
6. record another percussion layer.

A Mouse-Lite workflow might be:

```
Marker navigation
      ↓
Chorus
      ↓
Bass fader
      ↓
SELECT Vocal
      ↓
SEND
      ↓
adjust delay
      ↓
SELECT Synth
      ↓
DEVICE
      ↓
adjust parameter
      ↓
SELECT Vocal
      ↓
TOUCH automation
      ↓
perform fader ride
      ↓
SELECT percussion track / slot
      ↓
OPTION + RECORD
      ↓
perform overdub
```

The mouse may not be needed.

But if you then decide to redraw one MIDI note:

```
use the mouse
```

That is exactly what Mouse-Lite means.

Fluency Beats Purity

The best X-Touch workflow is not the one with the fewest mouse clicks.

It is the one with the least unnecessary friction.

Sometimes that means:

```
fader
```

Sometimes:

```
V-Pot
```

Sometimes:

```
button
```

Sometimes:

```
mouse
```

The important thing is that the choice becomes deliberate rather than habitual.

The Important Idea

A control surface becomes powerful when you stop thinking of it as a collection of features.

Think instead in terms of musical intentions:

```
"That is too loud."
```

```
"That needs more delay."
```

```
"Take me to the chorus."
```

```
"Open the synth."
```

```
"Let me ride this level."
```

```
"Record another layer."
```

```
"Show me the snare."
```

Then let the X-Touch provide the physical route.

The recurring model is:

```
Intention
    │
    ▼
Focus
    │
```

```
    ▼
Context / Mode
    │
    ▼
Physical Action
    │
    ▼
Musical Result
```

And when another interface is genuinely better:

```
use it
```

Mouse-Lite is not a restriction.

It is the freedom to choose the right interface without automatically defaulting to the mouse.

Coming Next

The workflow is becoming increasingly physical.

But much of that fluency depends on how DrivenByMoss itself is configured.

Choices such as:

- Flat or Hierarchical track navigation;
- New Clip Length;
- fader-touch behaviour;
- startup mode;
- Arranger versus Launcher priority;
- knob sensitivity;

can change how the controller behaves.

So before customising individual buttons, we should make sure the underlying system is configured deliberately.

Next:

Configuring DrivenByMoss for the X-Touch.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 21 - Configuring DrivenByMoss for the X-Touch

Configuring DrivenByMoss for the X-Touch

For most of this guide, we have concentrated on using the X-Touch.

Press a button.

Turn a V-Pot.

Move a fader.

Select a mode.

But all of those operations take place inside an environment created by DrivenByMoss configuration.

The complete system is:

```
X-Touch
   │
   ▼
Mackie Control Protocol
   │
   ▼
DrivenByMoss
   │
   ▼
Bitwig Studio
```

DrivenByMoss does more than translate button presses.

Its preferences determine:

- how the hardware is represented;
- what appears in the track banks;
- how Groups are navigated;
- what happens when a fader is touched;
- which mode appears at startup;
- how quickly the encoders respond;
- whether Arranger or Launcher functions have priority;
- how Browser information is presented;
- what programmable buttons and footswitches do.

So configuration is not separate from workflow.

It helps define the workflow.

Configuration Should Reduce Decisions

A good configuration removes unnecessary decisions from the musical moment.

For example, if you normally create four-bar clips, setting an appropriate New Clip Length means the recording workflow can become:

```
Create
   ↓
Record
```

rather than:

```
Create
   ↓
Choose Length
   ↓
Confirm
   ↓
Record
```

Likewise, if you prefer hierarchical project navigation, configuring that once means you do not need to rethink the controller's navigation model every session.

A useful principle is:

Configure beforehand so that you have less to configure while making music.

The Preferences Are Global

DrivenByMoss stores these preferences globally rather than separately for each project.

That is important.

Changing a controller preference affects the way the X-Touch behaves generally.

It is not merely modifying the current Bitwig project.

So think of the settings as defining:

```
My X-Touch Environment
```

rather than:

```
This Song's X-Touch Settings
```

That makes configuration worth doing deliberately.

Start with the X-Touch Profile

DrivenByMoss provides hardware profiles for supported controllers.

For the Behringer X-Touch, choose the appropriate X-Touch profile.

The profile configures the relevant MCU hardware options for the controller.

One slightly unusual detail is worth knowing:

The Profile menu does not remain showing the selected profile afterwards.

That is intentional.

The profile is a convenient way of setting the hardware options.

It is not itself a persistent mode indicator.

So do not assume that the setup has failed merely because the menu no longer visibly says X-Touch.

X-Touch Hardware Requirements

The DrivenByMoss documentation specifies three important requirements for full X-Touch support.

Firmware

Use the current X-Touch firmware documented by DrivenByMoss:

```
1.22
```

This is particularly important for display-colour support.

Operating Mode

The X-Touch must be set to:

```
MC
```

for Mackie Control operation.

Conceptually:

```
X-Touch
   |
   ▼
MC Mode
   |
   ▼
DrivenByMoss MCU Extension
```

If the hardware is in another protocol mode, the behaviour described throughout Project XTC should not be expected to match.

Display Colours

The X-Touch display-colour option should be enabled.

Selecting the X-Touch hardware profile is the easiest way to configure the appropriate display settings.

Get the Foundation Right First

Before adjusting advanced preferences, make sure the basic control surface works.

A useful order is:

```
X-Touch in MC Mode
       ↓
DrivenByMoss Controller Active
       ↓
MIDI Communication Working
       ↓
Transport Working
       ↓
Faders Working
       ↓
Feedback Working
       ↓
Advanced Preferences
```

Do not troubleshoot specialised Layer Mode or Browser behaviour while the fundamental controller connection is uncertain.

Two-Way Communication Matters

The X-Touch is not merely sending commands to Bitwig.

Bitwig and DrivenByMoss also send information back.

For example:

```
Move Fader
    ↓
Bitwig Changes
```

but also:

```
Bitwig Changes
      ↓
Motor Fader Moves
```

Likewise, the return path supplies:

- scribble-strip information;
- assignment displays;
- LEDs;
- V-Pot rings;
- VU information;

• motor positions.

So a working X-Touch needs a conversation:

```
X-Touch
   ⇄
DrivenByMoss
   ⇄
Bitwig
```

not a one-way stream of commands.

A Simple Connection Test

Before changing preferences, try a few basic operations.

Fader to Bitwig

Move a channel fader.

Does the corresponding Bitwig track volume change?

Bitwig to Fader

Move the same track volume in Bitwig.

Does the physical fader follow?

Track Selection

Press SELECT.

Does the expected track become selected?

Transport

Press PLAY and STOP.

Does Bitwig respond?

Display Feedback

Do the scribble strips contain meaningful track or parameter information?

If these basics work, the foundation is sound.

Hardware Setup

DrivenByMoss exposes several Hardware Setup options.

The X-Touch profile should normally configure these appropriately, but understanding what they mean is useful.

Main Display

This tells DrivenByMoss that the controller has a main MCU-style display.

For hardware consisting of eight separate display sections rather than one uninterrupted display, DrivenByMoss also provides a seven-character display option.

The purpose is simple:

```
Bitwig State
     ↓
DrivenByMoss
     ↓
Correct Display Format
     ↓
Readable X-Touch Feedback
```

Segment Display

The X-Touch has a segment display capable of showing transport-related information.

DrivenByMoss can use it for:

```
Play Position
```

and supplementary information such as:

```
Tempo
```

or:

```
Ticks
```

The Segment Display preferences control what information is presented there.

Assignment Display

The assignment display shows the current mode.

For a highly contextual surface, this is particularly valuable.

The controller may currently be in:

```
Panorama Mode
```

```
Send Mode
```

```
Device Mode
```

```
Browser Mode
```

```
Marker Mode
```

and the assignment display helps make that context visible.

Motor Faders

DrivenByMoss includes a hardware option indicating that the controller has motor faders.

For the X-Touch, of course, this should be enabled.

Without correct motor-fader handling, one of the controller's most important feedback mechanisms would be lost.

Display Colours

The X-Touch supports coloured scribble-strip backlighting.

DrivenByMoss has an explicit:

```
Display Colors
```

option for the Behringer X-Touch and X-Touch Extender.

Again, choosing the X-Touch hardware profile is the simplest way to establish the appropriate setup.

VU Meters

DrivenByMoss can send VU information to compatible MCU hardware.

The settings include:

```
VU Meters
```

and:

```
Always Send VU Meters
```

The latter exists for controllers that stop showing a VU value if they do not receive repeated updates when the level is unchanged.

This is mainly a hardware-compatibility setting.

For normal use, the aim is straightforward:

```
Audio Level
    ↓
DrivenByMoss
    ↓
Physical Meter Feedback
```

Use Faders Like Editing Knobs

DrivenByMoss has a preference named:

```
Use faders like editing knobs
```

This causes the faders to execute the functions normally assigned to the V-Pots.

The documentation specifically notes that this can be useful for recording automation.

This is closely related to the X-Touch's FLIP behaviour.

Conceptually:

```
Knob Assignment
      ↓
Fader
```

The attraction is obvious for parameters that benefit from:

- long physical travel;
- touch sensitivity;
- motorised feedback.

Track Navigation

One of the most important workflow preferences is:

```
Track Navigation
```

DrivenByMoss provides two approaches:

```
Flat
```

and:

```
Hierarchical
```

These produce meaningfully different SELECT-button behaviour.

Flat Track Navigation

In Flat mode, tracks are shown together in a flat track bank.

Conceptually:

```
Track 1
Track 2
Track 3
Track 4
Track 5
...
```

Groups do not become separate controller-navigation levels in the same way.

If an already selected Group is selected again:

```
SELECT Group again
        │
        ▼
Toggle Expanded State
```

This can be convenient if you prefer thinking of the project as one long mixer.

Hierarchical Track Navigation

In Hierarchical mode, the controller follows the project structure.

For example:

```
Project
    │
    ├── Drums
    │
    ├── Bass
    │
    ├── Guitars
    │
    ├── Keys
    │
    └── Vocals
```

Select a Group:

```
SELECT Drums
```

then select it again:

```
SELECT Drums again
       │
       ▼
Enter Group
```

The X-Touch can now show:

```
Kick   Snare   Hats   Toms   Percussion
```

To leave:

```
Long-press any SELECT
```

This is the navigation model described in Chapters 6 and 17.

Choosing Flat or Hierarchical

Neither choice is universally better.

Ask how you think about a project.

If the natural model is:

Give me one long mixer.

Flat may suit you.

If the natural model is:

Show me the broad project structure and let me drill into detail.

Hierarchical may be preferable.

For large, well-organised projects, hierarchical navigation can make eight channel strips feel considerably less restrictive.

Include FX and Master Tracks in the Track Bank

DrivenByMoss provides:

```
Include FX and master tracks in track bank
```

When enabled, these tracks are included in the normal track bank.

This can be useful on controllers that do not have dedicated access to those tracks.

On an X-Touch, the decision is more about workflow.

You may prefer:

```
ordinary audio / instrument tracks
```

as the main bank,

or you may want:

```
FX and Master
```

included in the same navigation sequence.

The setting changes what the eight-channel window contains.

Pin FX Tracks to the Last Device

For multi-controller systems, DrivenByMoss can:

```
Pin FX tracks to last device
```

This creates a bank of up to eight FX tracks on the right-most controller.

The instrument/audio track bank is reduced accordingly.

For a single X-Touch this option is less central.

With an Extender setup it can be extremely useful.

Conceptually:

```
Main Track Surface
        +
Dedicated FX Surface
```

We discuss expansion in Chapter 22.

Exclude Deactivated Items

The Workflow preferences contain:

```
Exclude deactivated items
```

When enabled, deactivated tracks and similar items are not shown on the controller.

This can make the banks cleaner.

For example:

```
Active
Active
Deactivated
Active
Deactivated
Active
```

can effectively become:

```
Active
Active
Active
Active
```

on the controller.

But there is a trade-off.

If the item is excluded, you also lose the ability to activate it from the surface.

So the choice is:

```
Cleaner Navigation
```

versus:

```
Access to Deactivated Items
```

Choose according to the way you work.

Startup Mode

DrivenByMoss lets you choose:

```
Startup Mode
```

This determines which parameter mode becomes active when the controller starts.

That sounds like a minor convenience.

In daily use it can be significant.

Suppose most sessions begin with ordinary mixing.

A useful startup context might be one that immediately gives you the controls you normally expect.

If most sessions begin with another task, choose accordingly.

The principle is:

Start where you usually work.

New Clip Length

DrivenByMoss provides a configurable:

```
New Clip Length
```

This determines the length of clips created by the relevant New/clip-creation workflows.

For example, if you regularly build four-bar loops:

```
New Clip Length
        │
        ▼
4 Bars
```

then commands such as the clip-based recording workflows begin with a useful default.

This connects directly with Chapter 19.

Configure for the Common Case

Suppose your new clips are roughly:

```
70%   4 bars
```

```
20%   8 bars
```

```
10%   something else
```

A four-bar default makes sense.

It does not have to cover every possibility.

A good default is the one that removes a decision most often.

Fader-Touch Behaviour

The current DrivenByMoss preferences expose two particularly useful fader-touch options:

```
Select Channel on Fader Touch
```

and:

```
Activate Volume mode on Fader Touch
```

These deserve careful explanation because they affect the physical feel of the controller.

Select Channel on Fader Touch

When enabled:

```
Touch Fader
     │
     ▼
Select Corresponding Channel
```

This can make mixing very fluid.

You hear something on the Vocal.

Reach for the Vocal fader.

The act of touching it establishes the Vocal as the selected track.

The workflow becomes:

```
Touch
  ↓
Focus
  ↓
Adjust
```

rather than:

```
SELECT
   ↓
Touch
   ↓
Adjust
```

Why You Might Disable Fader Selection

Touch-to-select is not universally desirable.

You may want to:

```
keep Synth selected
```

while simultaneously:

```
adjust Vocal volume
```

If touching the Vocal fader automatically changes the selection, that may interrupt another controller context.

So DrivenByMoss makes the behaviour configurable.

This corrects an important assumption:

Fader touch selecting the track should be treated as configurable behaviour, not an unavoidable property of the X-Touch mapping.

Activate Volume Mode on Fader Touch

DrivenByMoss can also temporarily activate Volume Mode when a fader is touched.

Conceptually:

```
Current Mode
```

```
     |
     ▼
Touch Fader
     |
     ▼
Volume Mode
     |
     ▼
Release Fader
     |
     ▼
Previous Context
```

This can be useful if you want fader interaction to return temporarily to an obvious volume-oriented view.

Again, it is a workflow preference.

Some users will find it reassuring.

Others may prefer the controller context to remain unchanged.

Knob Sensitivity

DrivenByMoss provides separate settings for:

```
Knob Sensitivity Default
```

and:

```
Knob Sensitivity Slow
```

Negative values slow the encoder response.

Positive values speed it up.

This lets you tune the relationship between:

```
Physical Turn
```

and:

```
Parameter Change
```

Why Sensitivity Matters

Suppose a small V-Pot movement causes a very large parameter change.

The encoder may feel too fast.

Reduce the sensitivity.

Conversely, if reaching the desired value requires excessive turning, increase it.

The useful question is:

Does the physical movement feel proportional to the musical adjustment?

The correct value is the one that feels predictable.

Default and Slow Sensitivity

The two sensitivity settings correspond to the normal and slower/fine-adjustment behaviour.

This means you can separately tune:

```
Normal Adjustment
```

and:

```
Fine Adjustment
```

Rather than assuming that the factory relationship is ideal for everyone, DrivenByMoss lets the physical response be adapted.

Encoder Knob Slow Down

The preferences also contain:

```
Encoder Knob Slow Down
```

This applies to the main encoder.

Use a higher value if the encoder changes values too quickly.

Again, the purpose is predictability.

A control surface should not feel twitchy.

It should feel like the software is following the physical gesture.

Transport Behaviour

DrivenByMoss exposes configurable behaviour for:

```
Behaviour on Stop
```

and:

```
Behaviour on Pause
```

The first controls what happens when playback is stopped with STOP.

The second controls what happens when playback is stopped using PLAY.

This matters because the Transport chapter showed that STOP and PLAY are not necessarily identical ways of ending playback.

The preferences let that distinction be customised.

Flip Arranger and Clip Record / Automation

This is one of the most important workflow preferences in the entire MCU configuration.

DrivenByMoss calls it:

```
Flip arranger and clip record / automation
```

Normally, the mapping gives the unmodified controls an Arranger-oriented role and SHIFT provides the related Launcher/Clip operation.

Conceptually:

```
Normal
   → Arranger-oriented function

SHIFT
   → Clip / Launcher-oriented function
```

With the preference enabled:

```
Normal
   → Clip / Launcher-oriented function

SHIFT
   → Arranger-oriented function
```

The relationship is reversed.

Why Flip the Recording Functions?

Imagine two users.

One mainly works in the Arranger.

The other builds music primarily in the Clip Launcher.

For the first user:

```
Arranger function
   → easiest gesture
```

makes sense.

For the second:

```
Launcher function
   → easiest gesture
```

makes more sense.

DrivenByMoss therefore lets the normal-versus-SHIFT priority follow the workflow.

Why This Preference Matters to Documentation

This setting can make two correctly configured X-Touch systems appear to behave differently.

One user reports:

The normal button does the Arranger operation.

Another reports:

Mine does the Launcher operation.

Both can be correct.

The difference may simply be:

```
Flip arranger and clip record / automation
```

So throughout Project XTC we describe the normal mapping unless a configuration-dependent behaviour is explicitly stated.

Browser Preferences

DrivenByMoss also lets you hide Browser filter columns that you do not use.

This may sound cosmetic.

On the X-Touch it can be quite useful.

Browser Mode has limited physical display space.

If irrelevant filter columns are hidden:

```
Fewer Columns
     ↓
Less Noise
     ↓
Relevant Choices Easier to Find
```

This is another example of configuration reducing decisions during the actual workflow.

Configure Browser Mode Around Your Search Habits

If you consistently ignore a Browser filter category, there is little benefit in forcing it to occupy controller attention.

A good Browser setup can therefore make the hardware workflow feel substantially more direct.

The principle remains:

Remove recurring friction, not theoretical possibilities.

Display Track Names

DrivenByMoss can configure the first display row to show track names rather than mode labels.

This is another trade-off.

Track names answer:

What channels am I looking at?

Mode labels answer:

What controller context am I in?

Both forms of information are useful.

Choose the display arrangement that makes the surface easiest for you to read.

Use Vertical Zoom to Change Modes

DrivenByMoss also provides:

```
Use vertical zoom to change modes
```

When enabled, the up/down arrows in Zoom Mode can select parameter modes.

This gives the arrow keys another possible controller-level role.

It is a good example of a preference that may be useful to one workflow and unnecessary complication to another.

Do not enable features simply because they exist.

Enable them because they solve a real problem.

The Segment Display

If the hardware has a segment display, DrivenByMoss lets you choose whether the play position is represented as:

```
Time
```

or:

```
Beats / Measures
```

It can also choose whether the final digits display:

```
Tempo
```

or:

```
Ticks
```

For music production, measures and tempo may feel natural.

For another workflow, absolute time may be more useful.

Again, configuration determines what information reaches you most efficiently.

Configure for Mixing

If your X-Touch is used primarily as a mixer, priorities may include:

• clear track names;
• predictable banking;
• sensible V-Pot response;
• immediate volume control;
• fader-touch selection if useful;
• Sends;
• reliable VU feedback.

A good mixing configuration should feel like:

```
Find Channel
     ↓
Touch / Select
     ↓
Adjust
```

with very little controller-management overhead.

Configure for Large Projects

For large projects, priorities may change.

Hierarchical navigation can become especially useful.

A top-level view such as:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

gives the project a manageable structure.

Then:

```
SELECT Drums
      ↓
SELECT again
      ↓
Kick   Snare   Hats   Toms   Percussion
```

reveals the detail only when it is required.

For this kind of workflow, the Track Navigation preference has a much greater effect than a subtle knob-sensitivity adjustment.

Configure for Launcher Work

A Launcher-oriented setup may prioritise:

```
New Clip Length
```

and:

```
Flip arranger and clip record / automation
```

The desired workflow may be:

```
Select Destination
      ↓
```

```
Create
      ↓
Record
      ↓
Overdub
      ↓
Listen
```

with Launcher functions available on the simplest physical gestures.

Configuration should support the musical model you actually use.

Configure for Device Work

A device-heavy workflow may care particularly about:

- display readability;
- knob sensitivity;
- fine adjustment;
- fader-as-knob behaviour;
- startup mode;
- Browser filter visibility.

The goal is:

```
Track
  ↓
Device
  ↓
Page
  ↓
Parameter
```

with as little unnecessary physical navigation as possible.

Configure for Performance Mixing

Our Chapter 20 dub example suggests another set of priorities.

A performance-oriented mixer benefits from:

- predictable fader response;
- useful fader-touch behaviour;
- clear channel feedback;
- fast Send access;
- sensible V-Pot sensitivity;
- stable track banking;
- immediate effect control.

When mixing performatively, predictability matters enormously.

You do not want to wonder:

What will happen if I touch this?

The configuration should make the answer obvious.

Change One Thing at a Time

When experimenting with preferences, avoid changing many settings simultaneously.

Use:

```
Change One Setting
       ↓
Test
       ↓
Understand Result
       ↓
Keep or Revert
```

rather than:

```
Change Six Settings
       ↓
Controller Feels Different
       ↓
Which Setting Did It?
```

This is particularly important with a contextual controller.

One preference may affect several different workflows.

Test Configuration Musically

A preference can look sensible in a settings panel but still feel awkward in practice.

If you change:

```
Track Navigation
```

navigate a real project.

If you change:

```
Knob Sensitivity
```

adjust real parameters.

If you change:

```
Fader Touch
```

mix something.

If you change:

```
New Clip Length
```

create some clips.

The real question is not:

Does this setting technically work?

It is:

Does this make the workflow better?

Keep a Known-Good Setup

Once the controller behaves reliably, it is useful to know what that setup consists of.

Record important information such as:

```
Bitwig Version

DrivenByMoss Version

X-Touch Firmware

X-Touch Mode

Track Navigation

Fader-Touch Preferences

New Clip Length

Important Sensitivity Values

Custom Assignments
```

If something changes after an update, you now have a known reference.

Version Matters

DrivenByMoss continues to develop.

Bitwig continues to develop.

The mapping may therefore change over time.

Project XTC should state the versions against which its behaviour has been verified.

That does not mean the guide instantly becomes obsolete after an update.

It means the reader knows:

This is the configuration against which these instructions were tested.

A newer version can then be compared against something concrete.

Perform a Sanity Check After Updates

After updating Bitwig or DrivenByMoss, test a few fundamentals.

For example:

```
Transport       ✓

Faders          ✓

SELECT          ✓

BANK / CHANNEL  ✓

Displays        ✓

V-Pots          ✓

Device Mode     ✓

Sends           ✓
```

Then test any specialised workflows you depend on.

This takes little time and can prevent a great deal of confusion.

Troubleshooting by Layer

Remember the system:

```
X-Touch Hardware
       │
       ▼
MC Mode
       │
       ▼
MIDI Communication
       │
       ▼
DrivenByMoss
       │
       ▼
Bitwig
```

If something fails, work from the bottom upwards.

Is the X-Touch in MC Mode?

If not, fix that first.

Does Basic Transport Work?

If PLAY and STOP do not work, the problem is probably more fundamental than Browser Mode or Layers.

Do Faders Send and Receive?

If moving a fader changes Bitwig but the motor never moves in response, investigate the return communication.

Do Displays Update?

If control works but assignments are unclear or stale, investigate the feedback path and hardware/display setup.

Does Only One Advanced Feature Fail?

Now inspect:

- the relevant mode;
- the relevant preference;
- the selected track/device;
- the current project structure.

Troubleshooting becomes much easier when approached in layers.

Don't Forget the Obvious Things

Sophisticated control-surface problems can have wonderfully uninteresting causes.

Check:

```
Power

MC Mode

Controller Extension Active

Correct Communication

Expected Track Selected

Expected Mode Selected

Expected Device Exists

Expected Project Object Exists
```

Do not begin by assuming an obscure MCU protocol failure.

A Sensible Starting Configuration

Project XTC should not prescribe one universal configuration.

But for someone following the workflows in this guide, a sensible starting point is:

```
X-Touch
   → MC Mode

Hardware
   → X-Touch Profile

Firmware
   → Current supported X-Touch firmware

Track Navigation
   → Choose Flat or Hierarchical deliberately

New Clip Length
   → Set to common working length

Fader Touch
   → Configure deliberately rather than accidentally

Knob Sensitivity
   → Adjust only if physical response feels wrong

Startup Mode
   → Choose normal starting workflow

Advanced Preferences
   → Leave alone until they solve a real problem
```

Then use the controller.

Let actual workflow friction tell you which setting deserves attention.

Configuration Should Eventually Disappear

This may sound odd in a chapter about configuration.

But the goal of good configuration is to stop thinking about configuration.

Eventually:

```
Turn On X-Touch
      ↓
Open Bitwig
      ↓
Work
```

The controller should not require a ritual of preference checking at the start of every session.

The setup has succeeded when it becomes invisible.

The Important Idea

DrivenByMoss configuration determines the environment in which all the controls described in Project XTC operate.

Important preferences include:

```
Hardware Profile
```

```
Display Setup

Track Navigation

FX / Master Track Banking

Startup Mode

New Clip Length

Fader-Touch Behaviour

Knob Sensitivity

Transport Behaviour

Arranger / Launcher Priority

Browser Filtering
```

The aim is not to create the most sophisticated possible setup.

It is to create the most predictable one.

A good configuration reduces repeated decisions.

It makes the physical behaviour match the way you think about the project.

And then it gets out of the way.

The ideal result is:

```
Intention
    ↓
Physical Action
    ↓
Musical Result
```

with the configuration quietly supporting that connection in the background.

Coming Next

DrivenByMoss configuration determines how the standard X-Touch workflow behaves.

But the system can go further.

Function buttons and footswitches can be assigned.

Bitwig Actions can be attached to physical controls.

Clip Based Looper provides a specialised performance workflow.

And additional MCU-compatible surfaces can expand the number of physical channel strips.

Next:

Customisation and Expansion.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 22 - Customisation and Expansion

Customisation and Expansion

We began Project XTC with a fixed piece of hardware.

Eight channel strips.

Eight V-Pots.

Motor faders.

Transport controls.

Mode buttons.

A Jog Wheel.

It would be easy to assume that this defines the limits of the system.

But DrivenByMoss provides another layer:

some parts of the X-Touch can be adapted to the way you work.

And if one X-Touch is not enough, the Mackie MCU implementation can also support additional control surfaces.

So this final chapter is about two related ideas:

```
Customisation
     │
     ▼
Make the existing surface suit you


Expansion
     │
     ▼
Make the physical surface larger
```

Neither is required.

A single X-Touch with its normal mappings is already a very capable controller.

But once the basic workflow is familiar, these options allow the system to become more personal.

Customise After You Understand

There is a temptation to customise a new controller immediately.

Change the function buttons.

Reassign the footswitches.

Modify preferences.

Create the perfect setup before making any music.

That is usually backwards.

A better progression is:

```
Use the standard workflow
        ↓
Notice repeated friction
        ↓
Identify the missing action
        ↓
Customise deliberately
```

In other words:

Solve problems you actually have.

Do not solve theoretical problems merely because a configuration option exists.

The Value of an Unused Button

A programmable button is valuable because it can shorten a repeated workflow.

Suppose you perform an operation twenty times during a session.

If that operation normally requires:

```
open menu
   ↓
find command
   ↓
click
```

but can instead become:

```
press button
```

that assignment may be worth making.

The best custom controls often feel surprisingly mundane.

They remove tiny interruptions.

And tiny interruptions repeated many times become significant.

Function Buttons

The X-Touch provides a row of function buttons labelled:

```
F1  F2  F3  F4  F5  F6  F7  F8
```

DrivenByMoss allows function-button behaviour to be assigned through its settings.

This makes the F-buttons a useful area for personal workflow commands.

Instead of asking:

What are F1–F8 supposed to do?

a better question is:

What would I benefit from having one button away?

Choose Functions by Frequency

A useful assignment is normally something that is:

- performed frequently;
- slightly awkward to reach otherwise;
- unambiguous;
- safe to trigger accidentally;
- easy to remember.

Suppose you repeatedly use a particular Bitwig operation.

Giving it an F-button may turn:

```
intention
   ↓
search
   ↓
mouse
   ↓
command
```

into:

```
intention
   ↓
F-button
```

That is exactly the kind of compression a control surface is good at.

Don't Fill Every Button

Eight labelled buttons can create a strange psychological pressure:

I must find eight things to assign.

You don't.

An unused button is not wasted.

A button assigned to something you never remember is more useless than an intentionally empty one.

Start with one or two functions that genuinely improve the workflow.

Then let experience tell you whether more are needed.

A Documentation Detail to Verify

There is currently a small discrepancy in the DrivenByMoss MCU documentation.

The general Functions section describes:

```
F1–F8
```

as assignable function buttons.

The Preferences section specifically lists configurable assignments for:

```
F1–F5
```

Project XTC should therefore not claim that all eight buttons are independently assignable in the current configuration interface until this has been verified against the exact DrivenByMoss version covered by the final guide.

This is a useful example of an important documentation principle:

When two sources inside the same reference disagree, test rather than guess.

The Quick Reference should contain only the verified behaviour.

Actions

One of the assignable function types in DrivenByMoss is an Action.

This allows an available Bitwig action to be selected and triggered from the controller.

Conceptually:

```
Physical Button
      │
      ▼
DrivenByMoss
      │
      ▼
Selected Bitwig Action
```

This opens the door to a much more personal controller layout.

Actions Turn Workflow into Hardware

Suppose Bitwig provides an action that you use constantly.

Without customisation:

```
remember shortcut
       or
find command
```

With an assigned controller button:

```
press
```

The software command has acquired a physical location.

And once an action has a physical location, muscle memory can develop around it.

This is one of the subtle advantages of programmable hardware.

Pick Actions You Can Explain in One Sentence

A good custom assignment should normally have an obvious purpose.

For example:

This button does X.

If you need a paragraph to remember why you assigned it, the mapping may be too clever.

Simple mappings survive.

Complicated mappings get forgotten.

A useful test is:

If I return to this project after a month, will I still know what this button does?

If not, either simplify the assignment or document it.

Label Your Customisations

The X-Touch's printed F-button labels obviously cannot change.

If you create important custom assignments, keep a record.

For example:

```
F1   → [your chosen action]
F2   → [your chosen action]
F3   → unused
F4   → [your chosen action]
```

This record could live:

- in your Project XTC notes;
- beside the controller;
- in a small text file;
- in the project documentation.

The important thing is that a personal mapping should not become a personal mystery.

Footswitches

DrivenByMoss exposes two MCU footswitch functions:

```
Footswitch 1 → USER A
```

```
Footswitch 2 → USER B
```

Their functions can be selected in the DrivenByMoss settings.

This adds something fundamentally different from another button on the control surface.

A footswitch can be operated while both hands are busy.

Why Feet Matter

Consider a recording situation.

Your left hand may be on an instrument.

Your right hand may also be on the instrument.

Reaching for the X-Touch interrupts the performance.

A footswitch gives us another input channel:

```
Hands
  │
  └── perform music
```

```
Foot
  │
  └── control recording function
```

That separation can be extremely useful.

Good Footswitch Jobs

A footswitch is particularly suitable for actions that are:

- performance-related;
- time-sensitive;
- simple;
- safe to trigger without looking.

The best assignments depend entirely on the workflow.

For one person, a footswitch may belong to recording.

For another, it may trigger a transport or navigation function.

For someone else, it may never be needed.

Again:

Customisation follows the work.

Clip Based Looper

One particularly interesting assignable function is Clip Based Looper.

This connects directly with the recording concepts from Chapter 19.

DrivenByMoss uses the currently selected MIDI clip slot.

If that slot is empty, a new clip is created using the configured New Clip Length, and playback begins.

The footswitch then controls overdub according to whether it is held.

Conceptually:

```
Selected Clip Slot
        │
        ▼
Is there a clip?
   │           │
   │ no        │ yes
   ▼           ▼
Create Clip    Use Clip
   │           │
   └───────────┘
         │
         ▼
      Playback
         │
         ▼
Hold Footswitch
         │
         ▼
      Overdub
         │
         ▼
Release Footswitch
         │
         ▼
Stop Overdubbing
```

That is considerably more than a simple button assignment.

It creates a performance workflow.

Hands-Free Loop Building

Imagine playing a MIDI keyboard.

Both hands are occupied.

The basic loop exists, and you want to add another layer.

With an appropriately assigned footswitch:

```
Play
  ↓
Hold Footswitch
  ↓
Overdub
  ↓
Release
  ↓
Listen
```

The hands remain on the instrument.

The recording state is controlled by the foot.

This can feel much more natural than reaching away from the keyboard every time the loop changes state.

New Clip Length Matters Here

Clip Based Looper also demonstrates why the configuration choices from Chapter 21 matter.

If the selected slot is empty, the new clip uses the configured New Clip Length.

So:

```
Configuration
      │
      ▼
New Clip Length
      │
      ▼
Clip Based Looper
      │
      ▼
Performance behaviour
```

A setting made before the session affects what happens during the session.

This is exactly what we meant by:

Configure beforehand so that you have less to configure while making music.

A Simple Looper Workflow

Suppose New Clip Length is set to four bars.

A workflow might be:

1. Select the destination track and slot

Establish where the loop belongs.

2. Begin the Clip Based Looper operation

If necessary, the new four-bar clip is created.

3. Play the foundation

Record the basic musical idea.

4. Let it loop

Listen.

5. Hold the footswitch

Overdub becomes active.

6. Add another part

Continue playing.

7. Release the footswitch

Overdub stops.

8. Listen again

Decide whether the clip needs another layer.

This is the kind of workflow where hardware customisation becomes genuinely musical.

Customisation Can Reduce Mode Switching

A useful custom control can sometimes avoid a journey through several other controls.

Without an assignment:

```
leave current task
      ↓
find required function
      ↓
perform it
      ↓
return
```

With an assignment:

```
press
```

This does not mean that modes are bad.

Modes are what make the X-Touch powerful.

But if one operation repeatedly forces you out of the mode where you actually want to work, a custom button may provide a useful shortcut.

Expansion

Customisation changes what the existing hardware does.

Expansion changes how much physical hardware is available.

The Mackie MCU model was designed to support additional channel strips.

DrivenByMoss therefore supports multi-device configurations.

For the X-Touch family, the obvious companion is the X-Touch Extender.

Why Add an Extender?

A single X-Touch gives us eight channel strips.

That has shaped much of this guide:

```
Tracks 1–8
     ↓
BANK
     ↓
Tracks 9–16
```

An Extender gives us more physical strips simultaneously.

Conceptually:

```
X-Touch

1   2   3   4   5   6   7   8


+

Extender

9  10  11  12  13  14  15  16
```

Now sixteen channels can be available without banking.

Expansion Does Not Change the Mental Model

This is important.

An Extender does not invalidate everything we learned about banking and context.

It merely makes the visible window wider.

With one X-Touch:

```
Project
────────────────────────────────────────

        ┌────────────────┐
        │   8 channels   │
        └────────────────┘
```

With an additional eight-channel surface:

```
Project
────────────────────────────────────────

        ┌────────────────────────────────┐
        │          16 channels           │
        └────────────────────────────────┘
```

The idea remains:

The physical surface is a window onto the project.

The window is simply larger.

Main and Extender Roles

DrivenByMoss distinguishes between controller roles.

A device can be configured as:

- Main
- Extender
- MCU Extender

The Main device provides the full main-controller role, including controls such as the Master fader and transport functionality.

An Extender primarily contributes additional channel control.

Conceptually:

```
MAIN
  │
  ├── Channel strips
  ├── Master
  ├── Transport
  └── Main commands


EXTENDER
  │
  └── Additional channel strips
```

This division reflects the physical design of MCU-style systems.

MCU Extender

DrivenByMoss also distinguishes an MCU Extender role for devices that use the original Mackie MCU Extender protocol.

This matters because not every additional controller communicates in exactly the same way as the main MCU unit.

For a straightforward X-Touch plus X-Touch Extender setup, choose the role appropriate to the actual hardware and connection rather than assuming that every expansion device should be configured identically.

Multiple Main Devices

DrivenByMoss also supports multiple devices configured as Main.

That is a more specialised configuration.

For most Project XTC readers, the simpler mental model is sufficient:

```
Main X-Touch
     +
additional channel surface
```

More elaborate multi-device layouts should be configured only when there is a clear reason for them.

Restart After Extender Changes

Changes to the Extender Setup require the DrivenByMoss extension to be restarted before the new configuration takes effect.

This is worth remembering because otherwise the settings may appear not to have worked.

A sensible workflow is:

```
Change Extender Setup
        ↓
Restart extension
        ↓
Test banking and channel order
```

Do not repeatedly alter settings simply because the old state remains active before the required restart.

What Does More Hardware Actually Buy Us?

It is easy to think:

Sixteen faders must be twice as good as eight.

Not necessarily.

More physical channels offer some clear advantages.

Less Banking

The obvious advantage is fewer bank changes.

For a sixteen-track project:

One X-Touch

```
Tracks 1–8
    ↓ BANK
Tracks 9–16
```

X-Touch + eight-channel Extender

```
Tracks 1–16
simultaneously
```

That can make broad mixing considerably more immediate.

More Simultaneous Faders

Two hands can move several faders.

With more physical strips visible at once, more of the mix remains physically accessible.

This can be especially useful for:

- live mixing;
- automation passes;
- balancing Groups;
- performance-oriented work.

More Immediate Mute and Solo

The advantage is not only faders.

Additional strips also mean more simultaneously available:

- MUTE buttons;
- SOLO buttons;
- SELECT buttons;
- ARM buttons;
- V-Pots;
- display information.

The physical representation of the mixer becomes broader.

But More Hardware Also Means More Space

An Extender costs:

- money;
- desk space;
- another connection;
- more configuration;
- more physical reach.

The correct question is not:

Would sixteen faders be nice?

Of course they would.

The better question is:

Does banking interrupt my workflow often enough that more physical channels are worth the cost and space?

That is a much more useful decision.

Expansion and the Dub Mixing Desk

Our Chapter 20 dub workflow gives an obvious example.

With eight strips:

```
Drums
Bass
Skank
Organ
Percussion
Vocal
FX 1
FX 2
```

already fits rather neatly onto one X-Touch.

But a larger performance mix might contain:

```
Drums
Bass
Skank
Organ
Piano
Percussion
Vocal 1
Vocal 2
FX Return 1
FX Return 2
FX Return 3
FX Return 4
```

...

An Extender could keep considerably more of that surface physically available at once.

For a performance-style mix, that may genuinely change how the controller feels.

You bank less.

You react more.

More Controls Are Not Automatically Better

There is a useful parallel with custom buttons.

More functionality is only useful when it remains understandable.

A single well-learned X-Touch may be more effective than a huge surface whose layout you have not internalised.

Likewise:

```
4 useful custom assignments
```

may be better than:

```
20 assignments you cannot remember
```

The goal is not maximum capability.

It is maximum fluency.

Build a Personal Layer on Top of the Standard One

Project XTC has spent most of its time describing a common baseline.

That matters.

If every user begins with a completely different control map, documentation becomes impossible and troubleshooting becomes much harder.

So think of customisation as a second layer:

```
DrivenByMoss standard mapping
            │
            ▼
     Learn the system
            │
            ▼
   Personal customisation
```

Do not erase the standard mental model.

Build on it.

Keep the Obvious Things Obvious

If a button labelled PLAY plays, that is easy to remember.

If a custom button labelled F2 performs some personal action, that is also manageable.

Problems arise when familiar controls are remapped so aggressively that the surface no longer makes visual sense.

The printed legends are useful.

Use them where possible.

Save custom assignments for places intended to be custom.

Document Your Personal Setup

Once a controller becomes customised, your own documentation becomes valuable.

A simple file might record:

```
Project XTC Personal Configuration

F1  → ...
F2  → ...
F3  → ...

Footswitch 1 → ...
Footswitch 2 → ...

New Clip Length → ...

Track Navigation → Hierarchical
```

If the system ever needs to be recreated, this turns a reconstruction exercise into a checklist.

Customisation and Updates

DrivenByMoss evolves.

Bitwig evolves.

Available actions or configuration options may change.

After a major update, verify custom assignments just as you would verify normal controller behaviour.

A simple test might be:

```
F-buttons
   ✓
Footswitches
```

```
   ✓

Clip Based Looper
   ✓

Extender
   ✓
```

If something has changed, consult the current DrivenByMoss documentation before rebuilding the setup from scratch.

Leave Room for Discovery

One reason Project XTC grew from its original form was that the X-Touch could do more than we had initially realised.

An apparently obscure command such as:

```
OPTION + MARKER
```

led us to audit the complete DrivenByMoss MCU feature set.

That audit uncovered:

- Marker Mode;
- deeper modifier functions;
- Mixer Edit Modes;
- Automation;
- layers and drum pads;
- Master Mode;
- advanced recording;
- assignable controls;
- Extenders.

There will probably be further discoveries.

That is not a flaw.

It is what happens when a flexible controller meets actively developed software.

Don't Customise Away the Possibility of Learning

There is one subtle risk with customisation.

If we immediately assign a custom button to solve every unfamiliar task, we may never learn the standard DrivenByMoss workflow that already solves it.

So when you encounter friction:

```
Problem
   ↓
Is there already a normal X-Touch method?
   │
```

```
├── yes → learn it
│
└── no / awkward
         ↓
     customise
```

This preserves the common vocabulary of the controller while still allowing personal refinement.

A Sensible Customisation Process

A disciplined approach might be:

1. Use the normal setup

Work with it long enough to identify genuine friction.

2. Write the problem down

For example:

I repeatedly need to perform X and the current route interrupts me.

3. Look for an existing X-Touch workflow

There may already be one.

4. If necessary, choose a custom control

Use an F-button, footswitch or assignable Action.

5. Test it in a real session

Does it actually help?

6. Keep or remove it

Do not preserve a customisation merely because you spent time creating it.

7. Document anything important

Future you will be grateful.

Your X-Touch Does Not Have to Be My X-Touch

This guide needs a common baseline so that instructions remain meaningful.

But beyond that baseline, two users may reasonably develop different surfaces.

One might prioritise:

```
Automation
```

```
Mixing
```

```
Device control
```

Another:

```
Launcher recording

Clip Based Looper

Footswitch control
```

Another:

```
Large mixer

Extender

Many simultaneous faders
```

Another may change almost nothing.

All of those are valid.

The controller serves the workflow.

Not the other way around.

From Fixed Hardware to Adaptable Surface

When we first looked at the X-Touch, the hardware appeared fixed:

```
buttons
faders
knobs
displays
```

Now we can see several dimensions of adaptability.

```
Modes
   ↓
change what controls mean

Modifiers
   ↓
temporarily extend controls

Configuration
   ↓
change controller behaviour

Custom assignments
   ↓
add personal shortcuts

Extenders
   ↓
increase physical capacity
```

So although the hardware itself is fixed, the surface as experienced by the user is not.

The Important Idea

Customisation is not about making the X-Touch more complicated.

It should do the opposite.

A good customisation removes friction.

A good assignment shortens a repeated path.

A good footswitch keeps your hands on the instrument.

A useful Extender keeps the channels you care about physically available.

The test is simple:

Does this make the work easier to perform and easier to understand?

If yes, keep it.

If not, remove it.

End of the Numbered Chapters

We began with a piece of hardware.

Eight motor faders.

Eight channel strips.

Eight V-Pots.

A collection of buttons whose labels came from the Mackie Control world.

At first, the surface could look almost overwhelming.

But the underlying ideas turned out to be much simpler.

```
BANKS
   → move the window

SELECT
   → establish focus

MODES
   → change the view

MODIFIERS
   → temporarily extend a control

DISPLAYS
   → tell us what the controls mean

MOTOR FADERS
   → communicate in both directions

HIERARCHY
   → move between levels
```

```
CUSTOMISATION
   → adapt the surface to the work
```

Those ideas are more useful than memorising hundreds of commands.

Because when an unfamiliar function appears, we now have a way to understand it.

From Controller to Companion

The title of this project is X-Touch Companion.

That word matters.

A useful companion does not demand constant attention.

It does not make you think about its internal machinery every moment.

It becomes familiar.

Predictable.

Available when needed.

The same should eventually be true of the X-Touch.

At first:

```
Which button?
```

```
Which mode?
```

```
Which modifier?
```

Later:

```
I want the chorus.
```

```
I want the delay.
```

```
I want that fader.
```

```
I want to record this.
```

```
I want the snare quieter.
```

And your hands know what to do.

The Destination

The ultimate aim was never to become an expert in Mackie MCU button combinations.

It was never to avoid the mouse at all costs.

And it was never to use every feature simply because DrivenByMoss provides it.

The aim was to shorten the distance between:

```
musical intention
       │
       ▼
physical action
       │
       ▼
musical result
```

When that distance becomes small enough, the controller itself begins to disappear from conscious attention.

You stop thinking:

I am operating the X-Touch.

You think:

The vocal needs more delay.

And then you do it.

That is where Project XTC has been heading all along.

What Comes After the Chapters?

The numbered teaching chapters are now complete.

But a guide such as this also needs something different:

a place to look things up.

The next section of Project XTC will therefore be the Quick Reference.

It will organise the verified DrivenByMoss/X-Touch commands in two ways:

• by physical control;
• by task.

The teaching journey ends here.

The reference begins next.